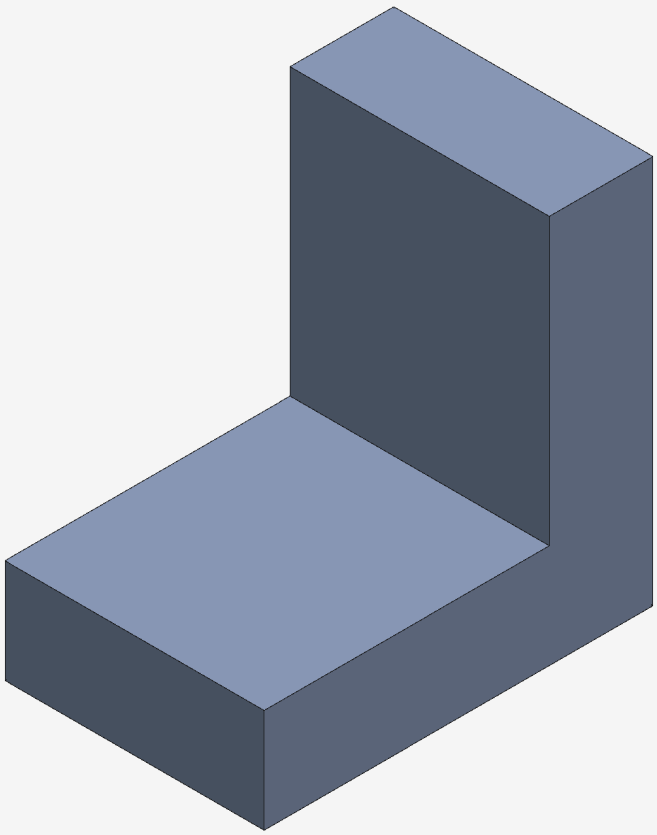


USER MANUAL

# Saulitwerx

Open-source parametric 3D CAD

| | |
|---|---|
| Describes | `main` at commit `1e89727` |
| Built | 2026-11-12 |
| Status | Pre-alpha: no desktop app yet |

# How to use this manual

This manual teaches you to use Saulitwerx, an open-source program for designing mechanical parts on a computer. You don't need to know anything about computer-aided design (CAD) to read it. Where a word has a special meaning, the manual explains it the first time it appears, and again in the glossary at the back.

## Saulitwerx is young

Saulitwerx is being built in the open, one small step each night. Today it is **pre-alpha**: the parts that do the real geometry work exist and are tested, but there is no desktop app with windows and buttons yet. That changes what this edition of the manual can offer:

- The **Concepts** part is for everyone. It explains how parametric CAD works, with pictures, so you will be ready when the app arrives.
- The **Tasks** part shows what you can already do, by running real commands and small programs. These sections carry a purple **For developers** band, because they need a terminal and some comfort with installing software.
- Nothing in this manual describes a feature as working unless it works on the version named on the cover. Plans are kept apart, in **Coming next** at the back.

## How the book is organized

| Part | Read it when you want to |
|---|---|
| I. Getting started | Know what Saulitwerx is, install it, and make your first picture of a part. |
| II. Concepts | Understand sketches, constraints, dimensions, parameters, features and solids. |
| III. Tasks | Do one specific thing. Each section is one task, with numbered steps. |
| IV. Reference | Look up an exact option, default or allowed value. |
| V. Help | Fix a problem, or understand an error message. |
| Back matter | Look up a word, see what changed, or find a topic in the index. |

## Conventions

| You see | It means |
|---|---|
| `swx-qa render` | Text you type exactly, or text the program prints. |
| Enter | A key on your keyboard. |
| *sketch* | A term defined at this spot. It is also in the glossary and the index. |
| 1 | A numbered step. Steps start with what to do, then say what you should see. |
| **RESULT** | What you have when a task is done. |

> **TIP**
>
> Callouts like this one hold shortcuts and good habits.

> **CAUTION**
>
> Cautions warn about a mistake that is easy to make, and how to avoid it.

> **FOR DEVELOPERS**
>
> Sections with this band run programs from a terminal. You can skip them and still follow every concept.

## Which version this describes

The cover names the commit of the `main` branch this edition describes, and the date it was built. The latest edition is always at github.com/glitchwizard/saulitwerx, in `docs/manual/`. **What's new**, near the back, lists what each edition added.

# Contents


**1 Welcome to Saulitwerx ..... 2**
- 1.1 What Saulitwerx is ..... 2
- 1.2 What it can do today ..... 2
- 1.3 Who should use it now ..... 3

**2 Installing Saulitwerx ..... 5**
- 2.1 What gets installed ..... 5
- 2.2 Install the tools ..... 5
- 2.3 Get the source and build it ..... 6
- 2.4 Other systems ..... 7

**3 Your first part ..... 8**
- 3.1 Draw a sample part ..... 8
- 3.2 Build a plate with a hole ..... 9
- 3.3 What you just did ..... 10

**4 How parametric CAD works ..... 12**
- 4.1 Solids ..... 12
- 4.2 Units ..... 13
- 4.3 Sketches ..... 13
- 4.4 Constraints ..... 13
- 4.5 Dimensions ..... 14
- 4.6 Degrees of freedom ..... 15
- 4.7 Profiles and regions ..... 15
- 4.8 Parameters ..... 16
- 4.9 Features ..... 16
- 4.10 History ..... 17
- 4.11 Putting it together ..... 17

**5 Sketching ..... 19**
- 5.1 Solve a sketch ..... 19
- 5.2 How the sample sketch is built ..... 20
- 5.3 Check how fully a sketch is defined ..... 21
- 5.4 Change a dimension ..... 22
- 5.5 Keep a hole centered ..... 22
- 5.6 Draw a slot ..... 24
- 5.7 Find the region to extrude ..... 26
- 5.8 Let Saulitwerx add constraints as you draw ..... 27
- 5.9 Common mistakes ..... 29

**6 Solids and STEP files ..... 30**
- 6.1 Make a solid and save it as STEP ..... 30
- 6.2 Combine solids ..... 31
- 6.3 Faces, edges and vertices of a solid ..... 31
- 6.4 Find the faces a Boolean made ..... 33
- 6.5 Parts with several bodies ..... 34
- 6.6 Open a STEP file from another program ..... 37

**7 Extruding a profile ..... 38**
7.1 Extrude a profile ..... 39
7.2 Choose the direction and the workplane ..... 40
7.3 Join, cut or intersect with an extrude ..... 41
7.4 Cut through the whole part ..... 43
7.5 Find the faces an extrude made ..... 45
7.6 Extrude up to a face ..... 47
7.7 Common mistakes ..... 49

**8 Revolving a profile ..... 51**
8.1 Revolve a profile ..... 52
8.2 Cut or join with a revolve ..... 53
8.3 Common mistakes ..... 54

**9 Rounding and bevelling edges ..... 56**
9.1 Fillet and chamfer edges ..... 56
9.2 Chamfer with two sizes ..... 58
9.3 Round with a changing radius ..... 60
9.4 Common mistakes ..... 62

**10 Hollowing a part ..... 64**
10.1 Shell a solid ..... 64
10.2 Common mistakes ..... 66

**11 Making holes ..... 67**
11.1 Drill holes in a plate ..... 68
11.2 Drill holes for screws ..... 70
11.3 Common mistakes ..... 73

**12 Tilting faces with draft ..... 75**
12.1 Draft the sides of a block ..... 75
12.2 Common mistakes ..... 77

**13 Lofting between sections ..... 78**
13.1 Loft through sections ..... 79
13.2 Common mistakes ..... 80

**14 Sweeping along a path ..... 81**
14.1 Sweep a profile along a path ..... 82
14.2 Sweep around a helix: springs and coils ..... 83
14.3 Common mistakes ..... 86

**15 Work planes, axes and points ..... 87**
15.1 Build work features ..... 88
15.2 Place a feature on a work plane ..... 89
15.3 Take work features from a part ..... 90
15.4 Midplanes, tangent planes and midpoints ..... 93
15.5 Common mistakes ..... 95

**16 Pictures of parts and sketches ..... 97**
16.1 Draw a STEP file ..... 97
16.2 Choose the size ..... 97
16.3 Choose which edges are outlined ..... 97
16.4 Draw a built-in sample ..... 98
16.5 Draw a sketch ..... 98

**17 Parameters and history ..... 100**
17.1 See what a change rebuilds ..... 100
17.2 How the sample is built ..... 101

17.3 Roll back and insert a feature ... 101
17.4 Name and rename parameters ... 103
17.5 Drive a sketch with parameters ... 104
17.6 Work out an expression ... 105
17.7 Make one size follow another ... 107
17.8 Common mistakes ... 109

**18 Saulitwerx document files ... 110**
18.1 Write and read a document folder ... 111
18.2 What is read and written ... 113
18.3 Common mistakes ... 113

**19 Reference ... 115**
19.1 `swx-qa render` ... 115
19.2 `swx-qa sketch` ... 115
19.3 Build options ... 116
19.4 Geometry kernel operations ... 118
19.5 Thread sizes ... 121
19.6 Profiles, extrude and revolve ... 121
19.7 Work features ... 123
19.8 Sketch entities and constraints ... 124
19.9 Solver settings and report ... 125
19.10 Profiles ... 125
19.11 Degree-of-freedom report ... 126
19.12 Inference ... 127
19.13 History ... 128
19.14 Parameters ... 128
19.15 Expressions ... 129
19.16 File formats and units ... 130
19.17 Document files ... 130
19.18 Keyboard and mouse ... 131

**20 Help ... 133**
20.1 Troubleshooting ... 133
20.2 Error messages ... 134
20.3 Questions ... 141
20.4 Known limitations ... 141

**Glossary ... 144**

**What's new ... 148**
2026-11-12: springs and coils, and Saulitwerx document files ... 148
2026-11-11: countersunk and tapped holes, and a thread table ... 148
2026-11-10: several bodies, midplanes and tangent planes, safer sweeps ... 148
2026-11-09: loft, sweep, and work features from the part ... 148
2026-11-08: holes, draft, work features, and a fillet that changes size ... 149
2026-11-07: shells, and rolling back the history ... 149
2026-11-06: uneven chamfers, and faces through a Boolean ... 149
2026-11-05: up to a face, fillets and chamfers ... 149
2026-11-04: which face is which ... 150
2026-11-03: safer revolves ... 150
2026-11-02: faces, edges and vertices ... 150
2026-11-01: revolve, and cuts through all ... 150
2026-10-31: extrude modes ... 150
2026-10-30: extrude ... 150
2026-10-29: formulas in parameters, and inference ... 151

2026-10-28: dimensions as parameters, expressions, and joined arcs ..... 151
2026-10-26: parameter table, and named conflicts ..... 151
2026-10-25: more constraints, and profiles ..... 151
2026-10-24: degrees of freedom and sketch pictures ..... 151
2026-10-22: first edition ..... 152
**Coming next ..... 153**
**Credits and licenses ..... 155**
**Index ..... 156**

PART I

# Getting started

What Saulitwerx is, what it can do today, how to install it, and a first session that ends with a picture of a real part.

CHAPTER 1

# Welcome to Saulitwerx

## 1.1 What Saulitwerx is

Saulitwerx is a program for designing solid mechanical parts on a computer: brackets, plates, shafts, housings, and anything else you might machine, cast or 3D-print. Programs like this are called *CAD* (computer-aided design). Saulitwerx is free and open source, so anyone can use it, study it, change it, and build it into other software.

Saulitwerx is a *parametric* modeler. You don't sculpt a shape by pushing points around. Instead you describe how to build it, step by step, with measurements: "draw a 40 by 25 mm rectangle, push it up 5 mm, cut a 10 mm hole through the middle." Saulitwerx remembers those steps and their numbers. Change the hole to 12 mm later, and the part rebuilds itself to match. Chapter 4 explains how that works.

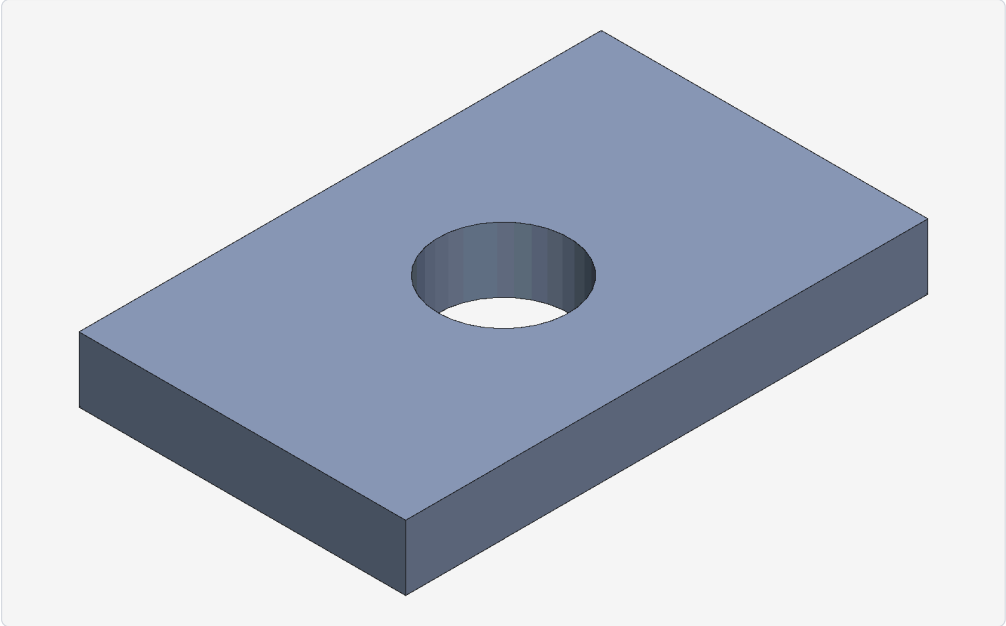


**Figure 1.1** A plate with a hole, made by Saulitwerx's geometry engine and drawn by its renderer. You will make this part yourself in Chapter 3.

## 1.2 What it can do today

Saulitwerx is built in small, tested steps, in the order of a public plan (the *roadmap*). Here is where it stands for this edition.

| Area | What exists | Use it |
|---|---|---|
| Solids | Make boxes, cylinders and spheres; combine them (add, subtract, keep the overlap), one tool or many at a time; count and separate the bodies of a part; measure volume, surface area and size. | NOW |
| Exchange | Save a solid as a STEP file, and read STEP files back. | NOW |
| Pictures | Draw a solid or a STEP file to a PNG picture with `swx-qa render`, and a sketch with `swx-qa sketch`. | NOW |
| Sketching | Lines, circles, arcs and points, with thirteen kinds of constraint (from horizontal and coincident to tangent and symmetric), a solver that moves the shape to satisfy them, a check of how fully a sketch is defined, and the regions a sketch encloses, ready to become solids. | NOW |
| History | Keep features in order with named parameters, work out which features a change affects, and roll back to insert features earlier in the history. | NOW |
| Extrude | Push a flat outline of lines, arcs and circles, with holes, into a solid, on any plane, one way or both, or cut right through a part; end it at a flat face of the part instead of a distance. | NOW |
| Revolve | Turn a flat outline around an axis into a round solid, by a full turn or part of one. | NOW |
| Edges | Round edges of a solid with a fillet, constant or changing along the edge, or bevel them with a chamfer. | NOW |
| Shell | Hollow a solid into walls of one thickness, open at the faces you choose. | NOW |
| Holes | Drill simple, counterbored, countersunk and tapped holes, blind or through all, in any direction. | NOW |
| Draft | Tilt the sides of a part so it comes out of a mold. | NOW |
| Loft, sweep | Blend a solid through several outlines, such as a square into a circle; move an outline along a bent path, such as a rod with an elbow, or around a helix, such as a coil spring. | NOW |
| Work features | Build work planes, axes and points to place features on, from numbers, halfway between two planes, or from a part's faces, edges and corners. | NOW |
| Document files | Write and read Saulitwerx document folders and check their manifest; the design itself can't be stored in them yet. | NOW |
| Desktop app | Windows, menus, a 3D view and mouse control. | LATER |
| Part files | Saving your design with its full history in a Saulitwerx part file, and opening it again. | LATER |
| Features | Extrude and revolve driven by sketches and recorded in the history; modeled screw threads, and more. | LATER |
| Assemblies, drawings | Putting parts together, and making 2D shop drawings. | LATER |

Everything marked NOW is a building block that programmers can call today. The **Tasks** part shows each one with a real command or a short program you can run. Everything marked LATER is described in **Coming next** at the back, and nowhere else.

## 1.3 Who should use it now

- **If you are new to CAD,** read Parts I and II. You will learn the ideas every parametric modeler is built on, and you will be ready the day the desktop app arrives.
- **If you are comfortable in a terminal,** install Saulitwerx (Chapter 2) and follow Chapter 3. It takes about 15 minutes and ends with a picture of a part you built.
- **If you write software,** the **For developers** sections show how to call each building block from Rust.

> **NOTE**
>
> Saulitwerx reads and writes only open, documented file formats. It will never open another program's own native files. To bring in a design from elsewhere, export it as STEP first.

CHAPTER 2

# Installing Saulitwerx

There is no installer yet. Until the first release, you build Saulitwerx from its source code. That sounds harder than it is: three tools to install, then one command builds everything. Plan on about 10 minutes, most of it waiting for downloads.

| | |
|---|---|
| **YOU WILL** | Install the tools Saulitwerx needs, download its source code, build it, and check that it works. |
| **YOU NEED** | A computer running Linux (these steps were tested on Ubuntu 24.04), an internet connection, and permission to install software ( `sudo` ). |
| **TIME** | About 10 minutes. |

## 2.1 What gets installed

| Tool | Why Saulitwerx needs it | Required? |
|---|---|---|
| Git | Downloads the source code and keeps it up to date. | Yes |
| Rust (1.80 or newer) | The programming language Saulitwerx is written in. Its tool `cargo` builds and runs the program. | Yes |
| C++ compiler | Builds a small bridge between Saulitwerx and the geometry engine. | For solids and STEP |
| OpenCascade (OCCT) 7.6 or newer | The open-source geometry engine that does the solid modeling. | For solids and STEP |

Without OpenCascade you can still sketch, try the document model, and draw the built-in sample parts. To make your own solids or read and write STEP files, you need it.

## 2.2 Install the tools

1. Open a terminal. On Ubuntu, press `Ctrl` + `Alt` + `T`. A window with a command prompt appears.
2. Install Git, a C++ compiler and OpenCascade:

   TERMINAL

   ```
   sudo apt-get update
   sudo apt-get install -y git build-essential curl \
      libocct-foundation-dev libocct-modeling-data-dev \
      libocct-modeling-algorithms-dev libocct-visualization-dev \
      libocct-data-exchange-dev
   ```

   Type your password when asked. The terminal prints a list of packages and ends at the prompt again.
3. Install Rust with its official installer, and accept the default choices when it asks:

   TERMINAL

   ```
   curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
   ```

It ends with `Rust is installed now. Great!`

4. Close the terminal and open a new one, so it finds the new tools. Then check them:

```
TERMINAL
cargo --version
```

You should see a version line such as `cargo 1.97.0 (…)`. Any version 1.80 or newer works.

## 2.3 Get the source and build it

1. Download the source code into a folder named `saulitwerx`, and go into it:

```
TERMINAL
git clone https://github.com/glitchwizard/saulitwerx
cd saulitwerx
```

2. Build Saulitwerx and run its own checks:

```
TERMINAL
cargo test --workspace
```

The first build downloads a few small libraries and takes a minute or two. It ends with several lines that start `test result: ok.` and none that say `FAILED`.

3. Build the picture tool with OpenCascade support:

```
TERMINAL
cargo build --release -p swx-qa --features occt
```

It ends with a line that starts `Finished`. The tool is now at `target/release/swx-qa`.

4. Check that the picture tool runs:

```
TERMINAL
target/release/swx-qa --help
```

You should see its help text, which starts `Usage: swx-qa render`.

**RESULT** Saulitwerx is built, its tests pass, and the `swx-qa` picture tool is ready. Go on to Chapter 3.

> **TIP Staying up to date**
>
> Saulitwerx changes most nights. To get the newest version, go into the `saulitwerx` folder and run `git pull`, then build again.

> **CAUTION Commands run from the `saulitwerx` folder**
>
> Every command in this manual assumes your terminal is in the `saulitwerx` folder. If a command says it can't find a file, run `cd saulitwerx` first. Chapter 20 lists other problems and their fixes.

## 2.4 Other systems

The build is regularly checked only on Ubuntu Linux. Saulitwerx aims to run on macOS and Windows too, and the same steps should work there once Rust and OpenCascade are installed, but they are untested. Without OpenCascade, leave out `--features occt` in step 3.

CHAPTER 3

# Your first part

In this tutorial you draw a sample part, then build a real one, a plate with a hole through it, measure it, save it in a standard file format, and draw it. Along the way you will meet the ideas the rest of the manual builds on.

| | |
|---|---|
| **YOU WILL** | Draw a built-in sample part, build a plate with a hole, check its measurements, save it as a STEP file, and draw it. |
| **YOU NEED** | Saulitwerx built with OpenCascade support (Chapter 2), and a terminal in the `saulitwerx` folder. |
| **TIME** | About 15 minutes. |

## 3.1 Draw a sample part

Saulitwerx comes with three sample parts built in: a cube, a cylinder and an L-shaped bracket. They need nothing else, so they are a quick way to check that drawing works.

1. Draw the cube to a picture file named `cube.png`:

   TERMINAL

   ```
   target/release/swx-qa render --fixture cube -o cube.png
   ```

   The tool answers with one line:

   OUTPUT

   ```
   wrote cube.png (800x600, 27669 bytes)
   ```

2. Open `cube.png` in any image viewer. On Ubuntu, type `xdg-open cube.png`. You should see a grey-blue cube on a light grey background, like Figure 3.1.

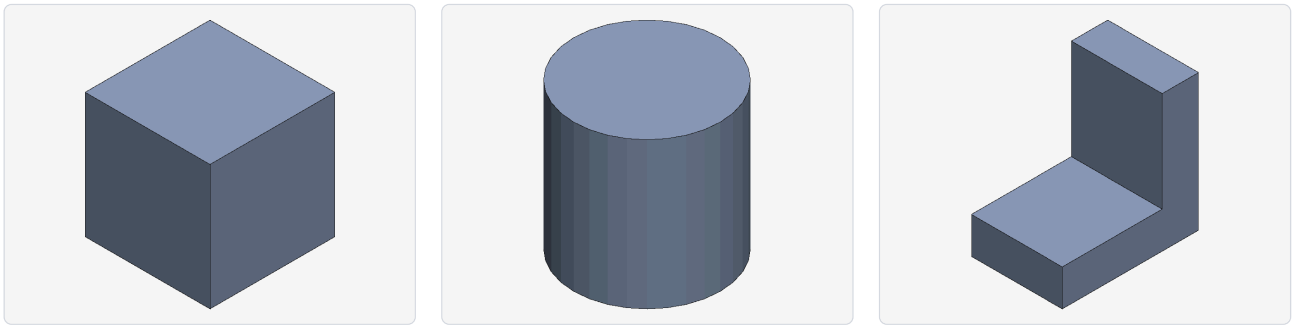


**Figure 3.1** The three built-in sample parts: `cube`, `cylinder` and `l-bracket`.

The picture is always drawn from the same corner, looking down at an angle, so that three sides of a box show at once. Each flat face gets one even shade, and sharp edges get a dark outline. This kind of view is called *isometric*.

> **TIP**
>
> Try `--fixture cylinder` and `--fixture l-bracket` too. Add `--size 1600x1200` for a bigger, sharper picture.

## 3.2 Build a plate with a hole

Now build a real part. Saulitwerx doesn't have a window to draw in yet, so this part comes from a small program that ships with Saulitwerx. The program asks the geometry engine for three things:

1. a *solid* block 40 mm long, 25 mm wide and 5 mm thick;
2. a cylinder 10 mm across, standing where the hole should go, a little taller than the plate;
3. the block **minus** the cylinder. Taking one solid away from another is called a *cut*.

1. Run the program, and tell it to save the part as `plate.step`:

   TERMINAL

   ```
   cargo run -p swx-kernel --features occt --example plate_with_hole -- plate.step
   ```

   After a few seconds of building, it prints the part's measurements. The geometry engine also prints a short report about writing the file, framed in rows of `*`; you can ignore it.

   OUTPUT

   ```
   volume        4607.30 mm³
   surface area  2650.00 mm²
   faces         7
   bounding box  [0.0, 0.0, 0.0] to [40.0, 25.0, 5.0]
   …  (the geometry engine's file report)
   wrote plate.step (read back: 4607.30 mm³, 7 faces)
   ```

2. Check the numbers. The block holds $40 \times 25 \times 5 = 5000$ mm³. The hole removes $\pi \times 5^2 \times 5 \approx 392.7$ mm³. $5000 - 392.7 = 4607.3$, which is what the program reports. The part has 7 faces: top, bottom, four sides and the inside of the hole.

3. Draw the part:

   TERMINAL

   ```
   target/release/swx-qa render --step plate.step --size 1600x1000 -o plate.png
   ```

   The tool answers `wrote plate.png (1600x1000, …)`.

4. Open `plate.png`. You should see the plate of Figure 3.2, with the hole showing light background through it.

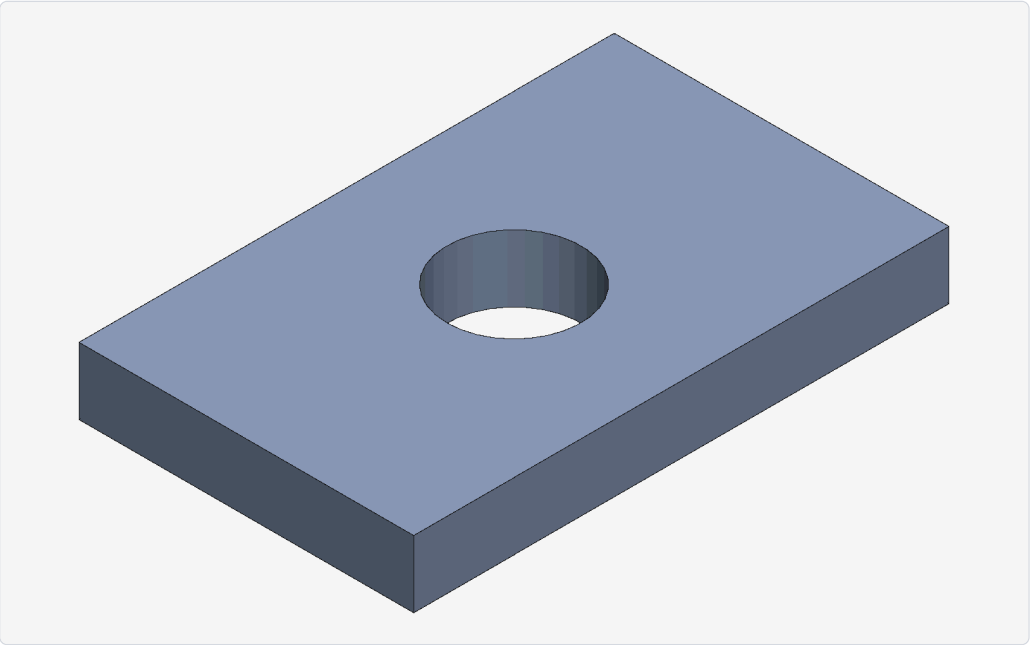


**Figure 3.2** The plate you built, drawn from the STEP file the program saved.

**RESULT** You built a solid part, checked its volume, saved it as `plate.step`, and drew it as `plate.png`.

## 3.3 What you just did

The file `plate.step` is in the *STEP* format, an international standard (ISO 10303) for exchanging 3D parts. Almost every CAD program can open it, so you can take this plate into another tool, send it to a machine shop, or prepare it for 3D printing.

What you did **not** see yet is the parametric part of Saulitwerx: the numbers 40, 25, 5 and 10 are typed into the program, not stored as named, editable parameters. That comes with the part features and part files on the roadmap. Chapter 4 explains the ideas behind it, and Chapter 17 shows the piece of it that already works.

> **FOR DEVELOPERS**
>
> The program you ran is `crates/swx-kernel/examples/plate_with_hole.rs`. Change `5.0` (the hole radius) in the `make_cylinder` line to `8.0`, run step 1 again, and the volume drops to 3994.69 mm³. Chapter 6 explains each call.

PART II

# Concepts

The ideas behind parametric CAD, in plain words. Read this before the task chapters; every procedure later relies on these terms.

CHAPTER 4

# How parametric CAD works

Every parametric modeler, Saulitwerx included, is built on a handful of ideas. Once you know them, the commands make sense on their own. This chapter explains each idea in the order you meet it when you design a part. For each one it says what it is, why it matters, and where it stands in Saulitwerx today.

## 4.1 Solids

A *solid* is a closed 3D shape with an inside and an outside, like a real object. Because Saulitwerx knows what is inside, it can tell you how much material a part holds (its *volume*), how much surface it has to paint (its *surface area*), and how big a box it would fit in (its *bounding box*).

The outside of a solid is made of *faces*: flat or curved patches of surface. Where two faces meet there is an *edge*, and where edges meet there is a corner point, a *vertex* (plural: vertices). The plate from Chapter 3 has 7 faces: top, bottom, four sides, and the round inside of the hole.

Faces, edges and vertices are what you will later pick to say "round this edge" or "sketch on this face", so it helps to see how they hang together. Each face is bounded by edges, and each edge ends at vertices. A box has 6 faces, 12 edges and 8 vertices; every edge sits between exactly two faces.

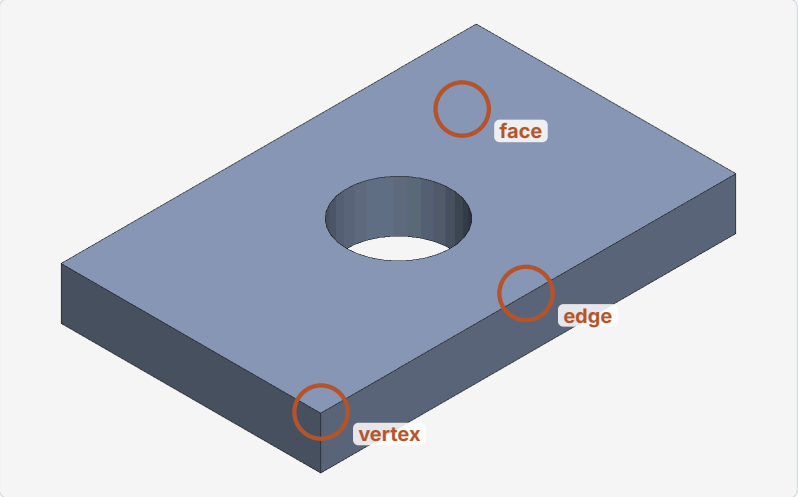



**Figure 4.1** A solid, with one face, one edge and one vertex ringed. Faces are shaded and edges outlined. Volume 4607.30 mm³, 7 faces.

**In Saulitwerx today:** the geometry engine makes boxes, cylinders and spheres, and measures volume, surface area, bounding box and face count. It can also count a solid's faces, edges and vertices, and list which ones touch (Section 6.3). Chapter 6 shows how.

## 4.2 Units

Every length in a part has a *unit*. In Saulitwerx the default is the millimeter (mm), and all of today's building blocks work in millimeters: a box 40 long is 40 mm long, and volumes come out in mm³. Angles are in degrees where you type them, and in radians inside the sketch geometry.

**In Saulitwerx today:** everything is in millimeters. STEP files are written in millimeters too. Choosing units per part comes with part files.

## 4.3 Sketches

Most solids start as a flat drawing. A *sketch* is a 2D drawing on a flat plane, made of simple pieces: points, straight lines, circles and arcs. Together these are called sketch *geometry* or *entities*.

A sketch for the plate would be a rectangle (four lines) and a circle. You don't need to draw it accurately. A rough shape is fine, because the next two ideas, constraints and dimensions, make it exact.

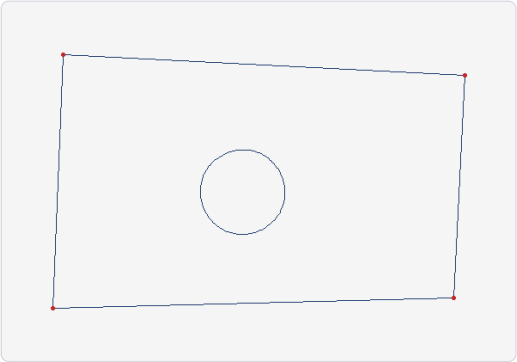


**Figure 4.2** A rough sketch of the plate: four lines and a circle, drawn by eye. The red dots are the ends of the lines.

**In Saulitwerx today:** sketches with points, lines, circles and arcs exist as a building block (Chapter 5). There is no drawing window yet, so sketches are written as a short program, and `swx-qa sketch` draws them to a picture (Section 16.5).

## 4.4 Constraints

A *constraint* is a rule about the shape of a sketch: "this line is horizontal," "these two corners touch," "this point stays at the origin." Constraints carry the **intent** of a design. If you later change a size, a line that must stay horizontal stays horizontal.

| Constraint | Rule it keeps |
|---|---|
| Coincident | Two points are at the same place. Use it to join the end of one line to the start of the next. |
| Horizontal | A line runs left–right; or two points are at the same height. |
| Vertical | A line runs up–down; or two points are directly above each other. |
| Fixed | A point stays at exact coordinates. Use it to anchor a sketch. |
| Parallel | Two lines point the same way, like opposite sides of a rectangle. |
| Perpendicular | Two lines meet at a right angle (90°), like the corner of a rectangle. |
| Equal | Two lines are the same length, or two circles or arcs are the same size. |
| Midpoint | A point sits exactly halfway along a line. |
| Symmetric | Two points are mirror images of each other across a line. |
| Tangent | A line just touches a circle or arc without cutting into it; or two circles or arcs just touch. |

Some constraints are about direction (horizontal, vertical, parallel, perpendicular), some about position (coincident, fixed, midpoint, symmetric), and some about how curves meet (tangent) or match (equal). Parallel and perpendicular are relative: a parallel pair can lean at any angle, as long as both lean the same way. That is why a rectangle usually gets one horizontal side, to say which way is level, and perpendicular and parallel for the rest.

You don't have to add every constraint by hand. Saulitwerx can add the ones your drawing already almost shows, such as a corner that nearly touches or a line that is nearly level (Section 5.8).

Saulitwerx satisfies constraints with a *solver*: a calculation that nudges every point until all the rules hold at once. You give it a rough drawing and the rules; it gives back the exact shape.

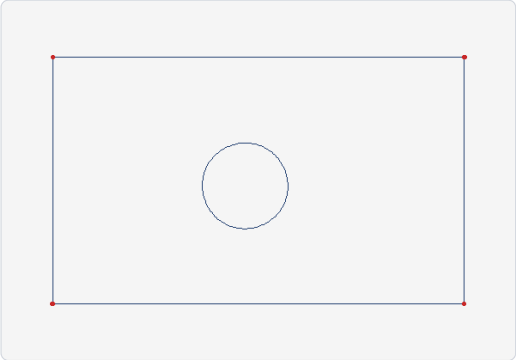


**Figure 4.3** The same sketch after the solver applied coincident, horizontal, vertical and fixed. It is now a true rectangle, but its size is still whatever the rough drawing happened to be.

## 4.5 Dimensions

A *dimension* is a constraint with a number: a length, a distance, a radius or a diameter. "The bottom line is 40 mm long." "The hole is 10 mm across." Dimensions fix the size of a sketch, the way the plain constraints above fix its shape.

| Dimension | Number it sets |
|---|---|
| Distance | The straight-line distance between two points, for example the two ends of a line. |
| Radius | The radius of a circle or arc. |
| Diameter | The diameter of a circle or arc: twice the radius. |

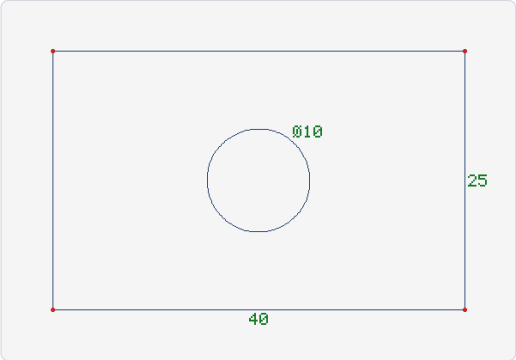



**Figure 4.4** With dimensions added, the plate sketch is exactly 40 × 25 mm with a ⌀10 mm hole. Each green number is a dimension.

## 4.6 Degrees of freedom

Each way a piece of a sketch can still move is called a *degree of freedom*, or DOF for short. A free point on a sheet has two (left–right and up–down). A free line has four (two for each end); a free circle has three (its center's two, and its size). Each constraint takes some away: fixed takes two from a point, horizontal takes one from a line.

Counting what is left tells you how finished a sketch is. A sketch is in one of four states: *under-constrained*, *fully constrained*, *over-constrained* or *conflicting*.

| State | What it means |
|---|---|
| Under-constrained | Something can still move. Add constraints or dimensions until the count reaches zero. |
| Fully constrained | Nothing can move, and no rule repeats another. This is the goal. |
| Over-constrained | At least one rule repeats what other rules already say, but they all agree. The repeated rule is *redundant*: harmless today, but it will fight you as soon as you change one of the sizes it repeats. |
| Conflicting | Redundant rules that disagree, such as two sides of a rectangle dimensioned 40 and 45 mm. No shape can satisfy them, so the solver stops short. |

The plate sketch starts with 19 degrees of freedom: four lines and a circle. Figure 4.3 still has 5 (the rectangle's width and height, and the circle's position and size); Figure 4.4 has none. Section 5.3 shows how to get these numbers.

> **TIP**
>
> Constrain the shape first (coincident, horizontal, vertical), then add dimensions. Then anchor one point with fixed, so the whole sketch can't slide around.

**In Saulitwerx today:** all of the constraints and dimensions in these tables work, and the solver reports whether it found a shape that meets them. A degree-of-freedom report gives the count and the state, and names each redundant constraint (Section 5.3). Constraints can name every point of a sketch: a point, a line's ends, a circle's or arc's center, and an arc's two ends, so lines and arcs join into closed outlines such as a slot (Section 5.6).

## 4.7 Profiles and regions

A sketch on its own is just lines. To become a solid, it has to enclose an area that a feature can push into 3D. A closed chain of sketch pieces, where each one ends exactly where the next begins, is a *loop*. The flat area a loop encloses is a *region*, and the regions of a sketch together are its *profile*.

A region can have holes. In the plate sketch the rectangle is one loop and the circle is another, inside it. Saulitwerx works out that the circle is a hole, so the plate's region is the rectangle **minus** the circle: 1000 − 78.54 = 921.46 mm². Pushed up 5 mm, that region is exactly the plate of Chapter 3, hole included, with no separate cut.

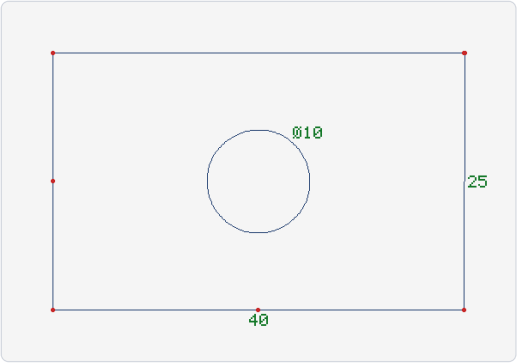



**Figure 4.5** Two loops, one region: the rectangle is its outside edge and the circle its hole.

Pieces that don't close a loop, like a line with a free end, are *open edges*. They are left out of every region and listed separately, so you can find the gap. Pieces that cross each other are an error: Saulitwerx won't guess which of the shapes they form you meant.

**In Saulitwerx today:** profiles are worked out from any sketch of lines, circles and arcs, with holes, nested islands and open edges (Section 5.7). The geometry engine can extrude an outline with holes into a solid (Chapter 7), or revolve one around an axis (Chapter 8), but the outline is typed in by hand: a sketch's regions can't be passed to either yet.

## 4.8 Parameters

A *parameter* is a named number, like `thickness = 5` or `hole_dia = 10`. Features and dimensions refer to parameters by name instead of holding a bare number. Change the parameter, and everything that uses it follows.

This is the "parametric" in parametric CAD. One plate design, with its thickness as a parameter, is really a whole family of plates: set `thickness` to 3, 5 or 8 and rebuild.

Parameters come in two kinds. Each dimension owns a *model parameter*, named `d0`, `d1`, … in the order the dimensions were made. You add *user parameters* yourself, with names you choose. Rename `d0` to `thickness`, and everything that uses it follows.

An *expression* is a small formula with units, such as `d1 / 2 + 5 mm`: half of `d1`, plus 5 millimeters. Saulitwerx checks that the units make sense, so adding a length to an angle is an error, not a silent wrong answer.

**In Saulitwerx today:** parts keep a parameter table with both kinds, units and comments, and features refer to parameters by name (Section 17.4). A sketch's distance, radius and diameter dimensions can each make their own model parameter and follow it when it changes (Section 17.5). A parameter can hold an expression, and follows the parameters it names (Section 17.7).

## 4.9 Features

A *feature* is one step in building a part: a sketch, or something done to the solid, such as pushing a sketch out into 3D or cutting a hole. Each feature records **what** it does and **which** sketch and parameters it uses, not just the shape that resulted.

Figure 4.6 shows the idea of a cut, done directly with the geometry engine: a block, minus a cylinder, gives the plate. An **extrude** can make the same plate in one step, from the outline of the rectangle and its hole (Chapter 7). Recording those steps as features in a part's history comes later.

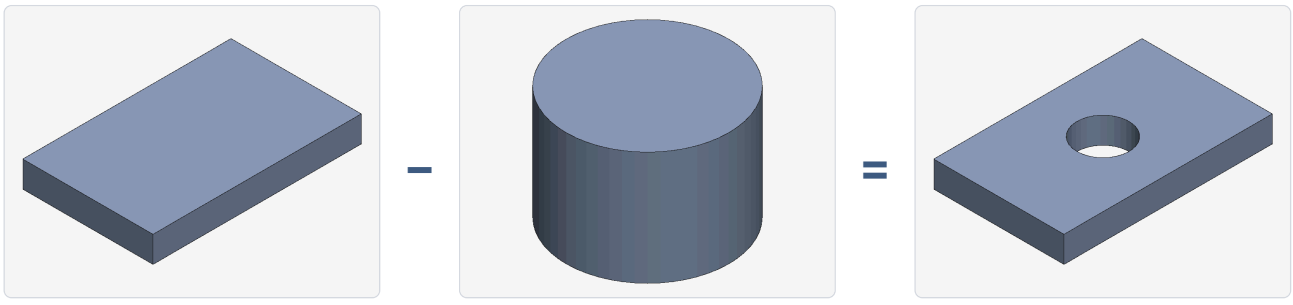



**Figure 4.6** A cut takes one solid away from another. Each picture is scaled to fill its frame, so the cylinder looks larger than it is.

There are three ways to combine two solids. Together they are called *Boolean operations*:

- **Union** (join): keep everything that is in either solid.
- **Cut** (subtract): keep the first solid, minus anything the second one overlaps.
- **Intersect**: keep only the part that is in both.

## 4.10 History

A part's *history*, also called its *feature tree*, is its list of features in the order they were made. Each feature can use only what came before it: a cut needs a solid to cut into, and an extrude needs a sketch to push.

| # | Feature | Uses |
|---|---|---|
| 0 | Sketch1 | nothing: a rectangle on a plane |
| 1 | Extrude1 | Sketch1, and the parameter `thickness` |
| 2 | Sketch2 | nothing: a profile for a pin |
| 3 | Revolve1 | Sketch2, and the parameter `sweep` |

Because the history remembers how each feature was made, Saulitwerx can **rebuild** a part after a change. It doesn't rebuild everything. It works out which features depend on what changed, and redoes only those. Change `thickness`, and only Extrude1 rebuilds; edit Sketch2, and Sketch2 and Revolve1 rebuild.

A history also has an **end-of-part marker**, normally after the last feature. Move it up, and the features below it are set aside, as if the part stopped there; new features then go in at the marker, in the middle of the history (Section 17.3).

**In Saulitwerx today:** the history, the end-of-part marker and the "what needs rebuilding" logic exist (Chapter 17), using the exact example in the table above. Running the rebuild on real geometry comes with the part features.

## 4.11 Putting it together

Here is how the ideas line up when you design a part, once the app exists:

1. Make a **sketch** on a plane, roughly the shape you want.
2. Add **constraints** to fix its shape, and **dimensions** to fix its size, until no degrees of freedom are left.
3. Turn the sketch into a **solid** with a **feature**, such as extrude.
4. Add more features (cuts, holes, rounded edges) to finish the part. They form its **history**.
5. Change a **parameter** or a dimension later, and the part rebuilds to match.

PART III

# Tasks

How to do things, one task per section. Until the desktop app arrives, every task here runs a real command or a small program, and is marked for developers.

CHAPTER 5

# Sketching

> **FOR DEVELOPERS**
>
> Until the desktop app's sketch mode arrives, sketches are written as a short Rust program against the `swx-sketch` library. This chapter runs one that ships with Saulitwerx.

## 5.1 Solve a sketch

The sketch for the plate of Chapter 3 is a 40 × 25 mm rectangle with a 10 mm circle in the middle. In this task you run a program that draws that sketch **roughly**, adds the constraints and dimensions, and asks the solver for the exact shape.

| | |
|---|---|
| **YOU WILL** | Run the sample sketch, read the solver's report, and see the rough shape become exact. |
| **YOU NEED** | Saulitwerx built (Chapter 2). OpenCascade is not needed. |
| **TIME** | 5 minutes. |

1. Run the sample sketch:

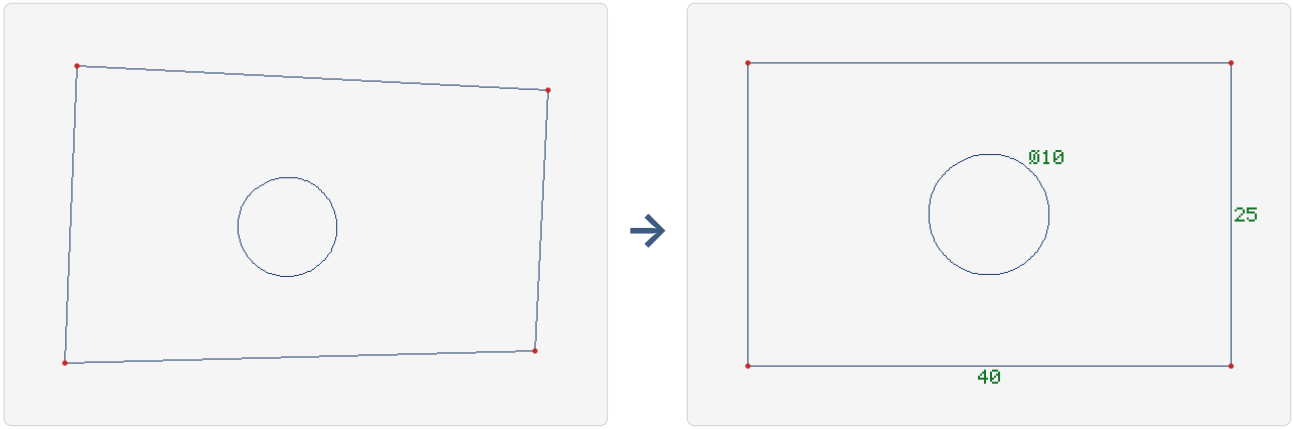
   TERMINAL
   ```
   cargo run -p swx-sketch --example plate_sketch
   ```

   It prints the solver's report, then each line and the circle:

   OUTPUT
   ```
   solved: true (Converged, 3 iterations, residual 5.3e-14)
   bottom (  0.00,   0.00) -> ( 40.00,   0.00)  length 40.00 mm
   right  ( 40.00,   0.00) -> ( 40.00,  25.00)  length 25.00 mm
   top    ( 40.00,  25.00) -> (  0.00,  25.00)  length 40.00 mm
   left   (  0.00,  25.00) -> (  0.00,   0.00)  length 25.00 mm
   hole   center (20.00, 12.50)  diameter 10.00 mm
   ```

2. Read the first line. `solved: true` and `Converged` mean the solver found a shape that meets every rule. `residual` is how far off it still is, here about 0.00000000000005 mm: exact for any practical purpose.
3. Compare the result with the rough drawing in the program (below). The bottom line was drawn 38 mm long and tilted; it is now exactly 40 mm and level.

**RESULT** A fully sized, square-cornered sketch: a 40 × 25 mm rectangle with its lower-left corner at the origin, and a 10 mm circle centered at (20, 12.5).




Before: the rough drawing After: solved

**Figure 5.1** The sample sketch before and after solving. Section 16.5 shows how to draw pictures like these.

## 5.2 How the sample sketch is built

The program is `crates/swx-sketch/examples/plate_sketch.rs`. It works in four stages, the same ones you will use in the app. The code below is shortened from the file.

**1. Draw roughly.** Four lines and a circle, deliberately off-size:

```
let bottom = s.add(Geometry::Line(Line::new(p(0.0, 0.0)?, p(38.0, 1.0)?)?))?;
let right = s.add(Geometry::Line(Line::new(p(38.0, 1.0)?, p(39.0, 22.0)?)?))?;
let top = s.add(Geometry::Line(Line::new(p(39.0, 22.0)?, p(1.0, 24.0)?)?))?;
let left = s.add(Geometry::Line(Line::new(p(1.0, 24.0)?, p(0.0, 0.0)?)?))?;
let hole = s.add(Geometry::Circle(Circle::new(p(18.0, 11.0)?, 4.0)?))?;
```

**2. Fix the shape.** Join each line's end to the next line's start with **coincident**, then make two lines horizontal and two vertical:

```
for (a, b) in [(bottom, right), (right, top), (top, left), (left, bottom)] {
    s.add_constraint(Constraint::Coincident(PointRef::End(a), PointRef::Start(b)))?;
}
s.add_constraint(Constraint::Horizontal(bottom))?;
s.add_constraint(Constraint::Vertical(left))?;   // and likewise top, right
```

**3. Anchor and size it.** Pin the lower-left corner to the origin with **fixed**, and set two **distances** and the hole's **diameter**:

```
s.add_constraint(Constraint::Fixed { point: PointRef::Start(bottom), x: 0.0, y: 0.0 })?;
s.add_constraint(Constraint::Distance {
    a: PointRef::Start(bottom), b: PointRef::End(bottom), value: 40.0 })?;
s.add_constraint(Constraint::Diameter { geom: hole, value: 10.0 })?;
```

**4. Solve.** One call moves everything into place and returns a report:

```
let report = s.solve(SolveOptions::default())?;
```

A `PointRef` names a point the constraint acts on: `Start(line)`, `End(line)`, `Center(circle)`, `ArcStart(arc)`, `ArcEnd(arc)` or `Point(point)`. Section 19.8 lists every entity and constraint with its exact form.

## 5.3 Check how fully a sketch is defined

A sketch that solves is not necessarily finished: parts of it may still be free to move, or some rules may repeat each other. A degree-of-freedom report (Section 4.6) tells you which. In this task you run a program that builds the plate sketch one group of rules at a time and checks it after each.

| | |
|---|---|
| **YOU WILL** | Watch the degrees of freedom of a sketch fall to zero, then see what a repeated or contradicting dimension looks like. |
| **YOU NEED** | Saulitwerx built (Chapter 2). OpenCascade is not needed. |
| **TIME** | 5 minutes. |

1. Run the sample:

   TERMINAL

   ```
   cargo run -p swx-sketch --example sketch_dof
   ```

   It prints one line per stage, with the number of equations the rules make, the degrees of freedom left, and the state:

   OUTPUT

   ```
   1 rough drawing            equations  0  dof 19  UnderConstrained
   2 + coincident x4          equations  8  dof 11  UnderConstrained
   3 + horizontal, vertical   equations 12  dof  7  UnderConstrained
   4 + fixed corner           equations 14  dof  5  UnderConstrained
   5 + two distances          equations 16  dof  3  UnderConstrained
     solve: Converged
   6 + hole, then solve       equations 19  dof  0  FullyConstrained
     solve: Converged
   7 + distance top 40        equations 20  dof  0  OverConstrained
                              redundant: distance top 40
     solve: Stagnated
   7 + distance top 45        equations 20  dof  0  Conflicting
                              redundant: distance top 45
   ```

2. Read stages 1 to 6. The four lines and the circle start with 19 degrees of freedom. Each group of rules takes some away, and after the hole is placed and sized none are left: `FullyConstrained`.
3. Read the first stage 7. A third distance, 40 mm on the top line, repeats what the bottom line and the horizontal and vertical rules already say. The solver still succeeds, but the report says `OverConstrained` and names the repeated rule.
4. Read the second stage 7. With 45 mm on the top line instead, no rectangle can meet every rule. The solver stops with `Stagnated`, and the report says `Conflicting` and names the 45 mm distance.

**RESULT** You can tell an unfinished sketch (`UnderConstrained`, with the number of degrees of freedom still open) from a finished one (`FullyConstrained`), and spot a repeated or contradicting rule.

### How the check works

The program is `crates/swx-sketch/examples/sketch_dof.rs`. One call on the sketch makes the report:

```rust
use swx_sketch::dof::DofOptions;

let r = s.dof(DofOptions::default())?;
println!("dof {} {:?}", r.report.dof, r.report.state);
for id in &r.redundant_constraints {
    println!("redundant: {}", names[id.0 as usize]);
}
```

`r.report` holds `dof` and the `state`. `r.redundant_constraints` lists the repeated rules as constraint ids: the number `add_constraint` gave each one, counting from 0 in the order you added them. The sample keeps a list of names in the same order, so `names[id.0]` prints the rule in words. A rule that adds two equations, such as a repeated coincident, is listed once. Section 19.11 lists every field and setting.

> **TIP** **Solve first, then check**
>
> The difference between over-constrained and conflicting is whether the rules are met. Run the check after `solve`: on a rough drawing that hasn't been solved yet, a harmless repeated rule can show as `Conflicting`.

> **CAUTION** **Only the later rule is named**
>
> When two rules repeat each other, the report names the one you added **later**, because the earlier one was still independent when it arrived. The fix may be to delete either one.

## 5.4 Change a dimension

1. Open `crates/swx-sketch/examples/plate_sketch.rs` in a text editor.
2. Find `value: 40.0` and change it to `value: 60.0`. Save the file.
3. Run the sample again (step 1 above). The bottom and top lines now report `length 60.00 mm`; the right line moves to x = 60. The hole stays at (20, 12.5), because it is fixed there, not centered.
4. Change the value back to `40.0` when you are done.

> **TIP**
>
> The hole stayed put because it is fixed at (20, 12.5). To keep it centered whatever the width, use midpoint instead (Section 5.5).

## 5.5 Keep a hole centered

A good sketch says what you **mean**. If the hole belongs in the middle of the plate, the sketch should say "in the middle", not "at x = 20". Then a wider plate keeps its hole centered with no extra work. The tool for this is the *midpoint* constraint, which keeps a point exactly halfway along a line.

| | |
|---|---|
| **YOU WILL** | Center a hole with midpoint constraints, then change the plate's width and watch the hole follow. |
| **YOU NEED** | Saulitwerx built (Chapter 2). OpenCascade is not needed. |
| **TIME** | 5 minutes. |

1. Run the centered-plate sample:

   TERMINAL

   ```
   cargo run -p swx-sketch --example centered_plate
   ```

   It solves the sketch, checks its degrees of freedom, and prints the sizes and the region an extrude would use:

OUTPUT

```
solved: true (Converged), dof 0 FullyConstrained
bottom length 40.00 mm
right  length 25.00 mm
hole   center (20.00, 12.50)  diameter 10.00 mm
regions: 1
  region 0: outer edges 4, holes 1, area 921.46 mm²
open edges: 0
```

2. Check the first line. `FullyConstrained` with `dof 0` means every piece is pinned down, including the hole, even though no rule gives the hole's coordinates.

3. Run it again for a plate 60 mm wide. The two numbers after `--` are the width and the height:

TERMINAL

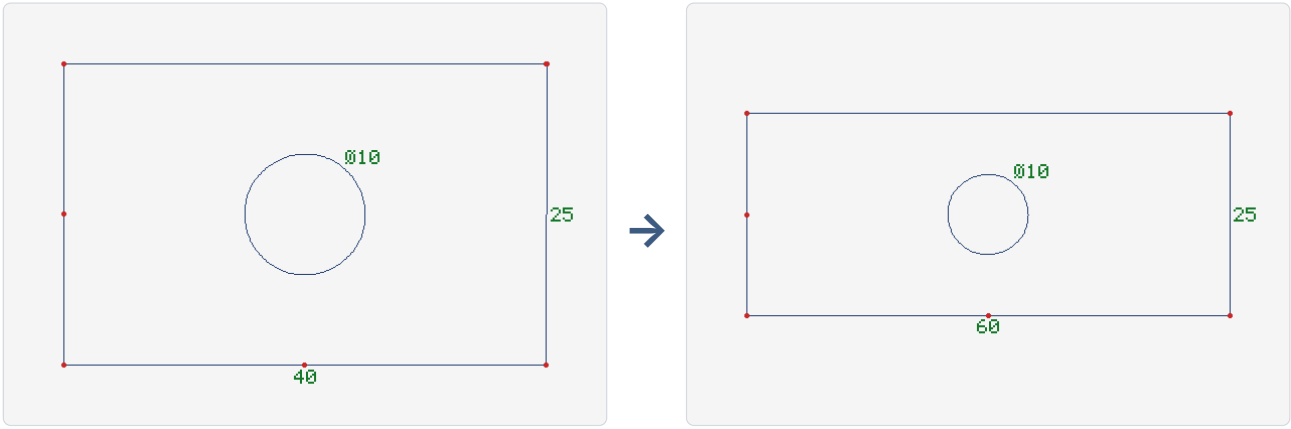
```
cargo run -p swx-sketch --example centered_plate -- 60 25
```

OUTPUT

```
solved: true (Converged), dof 0 FullyConstrained
bottom length 60.00 mm
right  length 25.00 mm
hole   center (30.00, 12.50)  diameter 10.00 mm
regions: 1
  region 0: outer edges 4, holes 1, area 1421.46 mm²
open edges: 0
```

The hole has moved to x = 30, the middle of the new width.

**RESULT** A plate whose hole stays in the middle whatever its width or height, and a sketch that is fully constrained.




40 mm wide 60 mm wide

**Figure 5.2** The same sketch with only the width changed. The red dots at the middle of the bottom and left sides are the helper points that center the hole.

## How the centered plate is built

The program is `crates/swx-sketch/examples/centered_plate.rs`. It starts from the same rough drawing as the plate sketch, and changes two things.

**Square corners without four direction rules.** Only the bottom is horizontal. The left side is *perpendicular* to it, and each side is *parallel* to the one opposite:

```
s.add_constraint(Constraint::Horizontal(bottom))?;
s.add_constraint(Constraint::Perpendicular(bottom, left))?;
```

```
s.add_constraint(Constraint::Parallel(top, bottom))?;
s.add_constraint(Constraint::Parallel(right, left))?;
```

**Center the hole with two helper points.** A point entity sits at the middle of the bottom side, and another at the middle of the left side. The hole's center is lined up straight above the first and level with the second:

```
let mid_bottom = s.add(Geometry::Point(p(19.0, 0.0)?))?;
let mid_left = s.add(Geometry::Point(p(0.0, 12.0)?))?;
s.add_constraint(Constraint::Midpoint { point: PointRef::Point(mid_bottom), line: bottom })?;
s.add_constraint(Constraint::Midpoint { point: PointRef::Point(mid_left), line: left })?;
s.add_constraint(Constraint::VerticalPoints(
    PointRef::Point(mid_bottom), PointRef::Center(hole)))?;
s.add_constraint(Constraint::HorizontalPoints(
    PointRef::Point(mid_left), PointRef::Center(hole)))?;
```

Helper points are safe to add: they are not part of any outline, so the profile ignores them.

> **TIP Why not a diagonal?**
>
> Drawing the rectangle's diagonal and putting the hole's center at its midpoint would also work for the solver. But the diagonal would cut through the hole and the plate, and profiles can't be found from pieces that cross (Section 5.7). Helper points avoid that.

The *symmetric* constraint is the other way to say "centered": `Symmetric { a, b, line }` makes points `a` and `b` mirror images across `line`. It suits pairs, such as two holes placed evenly either side of a center line.

## 5.6 Draw a slot

A slot is a rounded-end opening, like the hole a bolt slides along. Its outline is two straight sides and two half circles, and the sides must run smoothly into the curves, with no corner. Three constraints say that. Coincident joins each side's ends to the arcs' ends, so the outline closes. The *tangent* constraint makes a line touch a circle or arc without cutting into it, so there is no corner where they meet. The *equal* constraint keeps both ends the same size, so you set the radius only once.

| | |
|---|---|
| **YOU WILL** | Close a roughly drawn slot with coincident, tangent and equal constraints, check that it is fully defined, and find its region. |
| **YOU NEED** | Saulitwerx built (Chapter 2). OpenCascade is not needed. |
| **TIME** | 5 minutes. |

1. Run the slot sample:

   TERMINAL

   ```
   cargo run -p swx-sketch --example slot_sketch
   ```

   It prints the slot twice: as drawn, then after solving.

OUTPUT

```
1 as drawn
  left   arc center (  0.00,  0.00)  radius 4.00 mm  from   90° to  260°
  right  arc center ( 28.00,  1.00)  radius 6.00 mm  from  -90° to   90°
  bottom (  0.00, -4.00) -> ( 28.00, -5.00)
  regions: 0
  open edges: [0, 1, 2, 3]
2 solved: true
  dof 0 FullyConstrained
  left   arc center (  0.00,  0.00)  radius 5.00 mm  from   90° to  270°
  right  arc center ( 30.00,  0.00)  radius 5.00 mm  from  -90° to   90°
  bottom (  0.00, -5.00) -> ( 30.00, -5.00)
  regions: 1
    outer edges 4, holes 0, area 378.54 mm²
  open edges: []
```

2. Read stage 1. The slot was drawn carelessly on purpose: the ends have radii 4 and 6, the slot is tilted, and the left arc stops at 260°, 10° short of the bottom side (Figure 5.3). Because of that one gap the outline doesn't close, so there is no region and all four pieces are listed as open edges.
3. Read stage 2. Both radii are now 5 mm, the centers are 30 mm apart and level, and the left arc runs the full half circle from 90° to 270°. The solver turned the arc's end until it met the bottom side.
4. Read `dof 0 FullyConstrained`. Every piece of the slot is pinned down: its size, its shape and where it sits. Nothing is left to chance.
5. Read the region. The closed slot encloses one region of 378.54 mm²: a 30 × 10 mm rectangle plus two half circles of radius 5 mm.

**RESULT** A fully defined 30 mm slot with 5 mm round ends, and one region ready to become a solid.

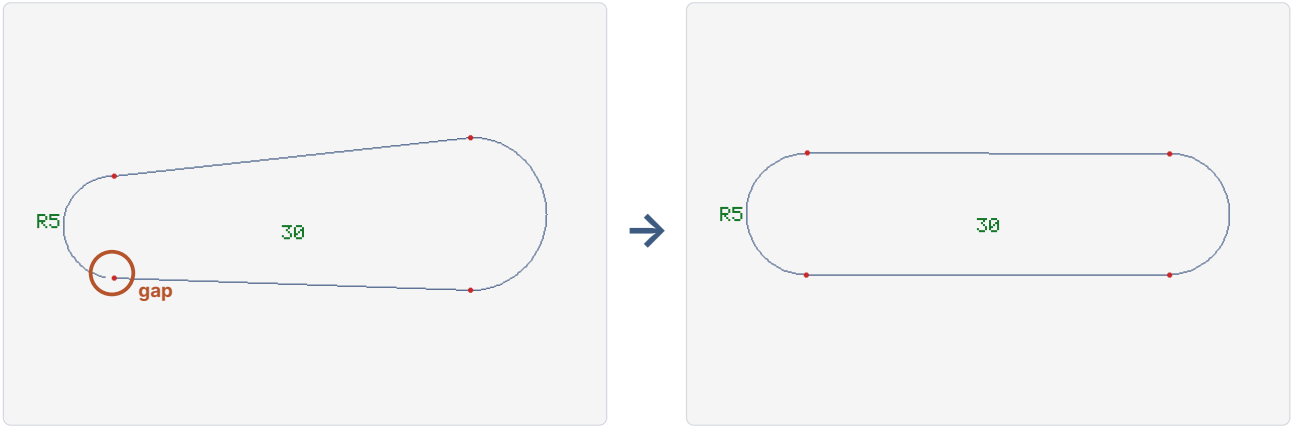



**Figure 5.3** The slot before and after solving. As drawn, the ends differ in size, the slot is tilted, and the left arc stops short of the bottom side (ringed). Solved, it is level, with equal 5 mm ends joined all the way round.

## How the slot is built

The program is `crates/swx-sketch/examples/slot_sketch.rs`. An arc's two ends are named with `PointRef::ArcStart(arc)` and `PointRef::ArcEnd(arc)`. An arc always runs counterclockwise from its start to its end, so the left arc, drawn from the top (90°) round to the bottom (270°), ends where the bottom side starts:

```
for (a, b) in [
    (PointRef::Start(bottom), PointRef::ArcEnd(left)),
    (PointRef::End(bottom), PointRef::ArcStart(right)),
    (PointRef::Start(top), PointRef::ArcEnd(right)),
```

```
    (PointRef::End(top), PointRef::ArcStart(left)),
] {
    s.add_constraint(Constraint::Coincident(a, b))?;
}
for (line, arc) in [(bottom, left), (bottom, right), (top, left), (top, right)] {
    s.add_constraint(Constraint::Tangent(line, arc))?;
}
s.add_constraint(Constraint::Equal(left, right))?;
s.add_constraint(Constraint::Radius { geom: left, value: 5.0 })?;
```

Then the bottom side is made horizontal, the left center is fixed at the origin, and the centers are set 30 mm apart. That is all the slot needs.

> **TIP** **Which end is which**
>
> Picture walking round the arc counterclockwise, the way the angles grow. Where you set off is `ArcStart`; where you stop is `ArcEnd`. If you join the wrong end, the solver still tries to obey, and you get a different shape from the one you meant. Check the angles in the output, or plot the sketch (Section 16.5).

> **TIP**
>
> A tangent line stays on the side of the circle it started on, and two tangent circles stay touching on the outside or the inside, whichever they were closer to when you added the constraint. Draw roughly the right arrangement and the solver keeps it. When a line's end is joined to an arc's end, tangent there means the line meets the arc square to its radius, which is exactly "no corner".

## 5.7 Find the region to extrude

Before a sketch can become a solid, Saulitwerx has to know which areas it encloses (Section 4.7). Both samples above end by asking for exactly that. This section shows the call, and how to read the answer.

| | |
|---|---|
| **YOU WILL** | Get the regions of a sketch, with their holes and areas, and find the open edges when a sketch doesn't close. |
| **YOU NEED** | The two samples above, run once each. |
| **TIME** | 5 minutes. |

1. Look at the end of the centered-plate output (Section 5.5): `regions: 1`, `holes 1`, `area 921.46 mm²`. The rectangle is the region's outside edge and the circle its hole; the area is the rectangle's 1000 mm² minus the hole's 78.54 mm².
2. Look at stage 2 of the slot output (Section 5.6): one region with `outer edges 4` (two lines, two arcs) and no holes.
3. Look at stage 1: `regions: 0` and `open edges: [0, 1, 2, 3]`. The numbers are the pieces in the order they were added to the sketch: 0 and 1 are the arcs, 2 and 3 the lines. When a sketch has no region, the open edges show where to look for the gap.

**RESULT** You can tell whether a sketch is ready to become a solid, and how much area it encloses. Chapter 7 shows how an outline like this becomes a solid.

### How it works

One call returns every region and the open edges:

```rust
use swx_sketch::profile::{profile_report, ProfileOptions};

let profile = profile_report(&s, &ProfileOptions::default())?;
for r in &profile.regions {
    println!("holes {}, area {:.2} mm²", r.holes.len(), r.area());
}
println!("open edges: {:?}", profile.open_edges);
```

`profiles(&s, &options)` returns only the regions. Ends of pieces closer together than a small tolerance count as joined; Section 19.10 lists the setting, every field, and the rules for loops inside loops.

> **CAUTION** **Crossing pieces stop the search**
>
> If two pieces cross, overlap or touch away from their ends, `profile_report` returns an error naming both, such as `geometry 2 and 5 intersect`, and no regions. Saulitwerx doesn't split pieces where they cross. Move or shorten one of them so they meet only end to end.

## 5.8 Let Saulitwerx add constraints as you draw

Most of a sketch's constraints say what your drawing already almost shows: this corner joins that one, this line is level, this circle touches that line. Adding each of them by hand is slow. *Inference* does it for you. After you add a piece, Saulitwerx compares it with everything already in the sketch, suggests the constraints it nearly satisfies, adds them and solves. It never adds a dimension: sizes stay yours to set.

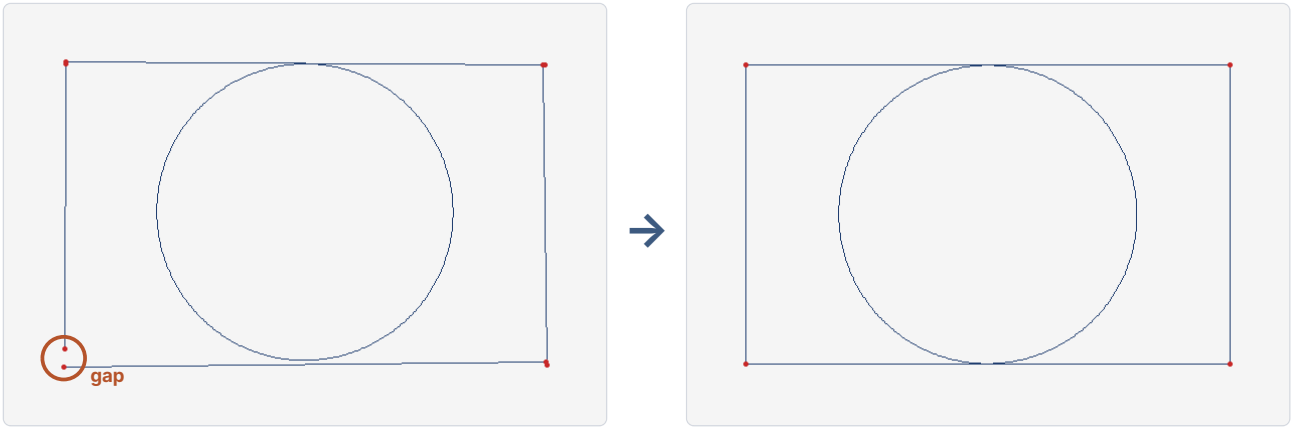



**Figure 5.4** The sketch before and after inference. As drawn, the sides lean slightly and the left side stops short of the bottom-left corner (ringed). After inference and solving, the sides are level and upright, every corner is joined, and the circle touches the top and bottom.

| | |
|---|---|
| **YOU WILL** | Draw a rough rectangle and a circle one piece at a time, and see which constraints inference adds after each. |
| **YOU NEED** | Saulitwerx built (Chapter 2). OpenCascade is not needed. |
| **TIME** | 5 minutes. |

1. Run the inference sample:

   TERMINAL

   ```
   cargo run -p swx-sketch --example inferred_sketch
   ```

   It draws four sides and then a circle, and after each one lists what inference suggested.

OUTPUT

```
bottom: 1 suggested
  horizontal: bottom
right: 2 suggested
  coincident: start of right = end of bottom
  vertical: right
top: 2 suggested
  coincident: start of top = end of right
  horizontal: top
left: 2 suggested
  coincident: start of left = end of top
  vertical: left
  (missed the first corner by 1.31 mm; the default point_tol is 0.047 mm)
again with point_tol 2 mm:
left: 3 suggested
  coincident: start of left = end of top
  coincident: end of left = start of bottom
  vertical: left
corners: (0.13, 0.52) (39.90, 0.52) (39.90, 25.07) (0.13, 25.07)
circle: 2 suggested
  tangent: circle and bottom
  tangent: circle and top
```

2. Read the first three pieces. The bottom was drawn 0.6° off level, so it got horizontal. The right side starts a few hundredths of a millimeter from the bottom's end, so its start was joined to that end, and it got vertical. The top went the same way. No piece was made parallel or perpendicular to another: once a line is horizontal or vertical, those would only repeat the rule.
3. Read the first try at the left side. Its start was joined to the top, but its end stopped 1.31 mm short of the first corner. That is far more than the default reach of 0.047 mm, so that corner was left open.
4. Read the second try. With a reach of 2 mm the gap was closed, and the rectangle is complete. Its corners are now exactly square (Figure 5.4).
5. Read the circle. It was drawn 0.01 mm short of touching the top and bottom, so it got tangent to both.

**RESULT** A closed rectangle with a circle touching its top and bottom, held by ten constraints you didn't have to add yourself.

Inference doesn't make a sketch fully defined. This rectangle has no sizes yet, and its corner is not fixed, so it still has degrees of freedom (Section 5.3). Add a fixed point and the dimensions next.

### How the sample is built

The program is `crates/swx-sketch/examples/inferred_sketch.rs`. After each `add`, one call infers, adds and solves:

```rust
use swx_sketch::infer::{add_inferred, infer, InferOptions};

let right = s.add(Geometry::Line(Line::new(start, end)?))?;
let opts = InferOptions::for_sketch(&s);
let suggested = infer(&s, right, &opts)?;      // look only
let added = add_inferred(&mut s, right, &opts)?; // add and solve
```

`infer` only looks and changes nothing, so you can show the suggestions first and let the user choose. `add_inferred` adds every suggestion and solves. Two settings decide how near is near enough. `point_tol` is the reach for joining points and for tangency; by default it is a thousandth of the sketch's overall size, its diagonal. `angle_tol_deg` is the reach for directions; by default 1°. Section 19.12 gives the full rules.

> **TIP** **Draw from where you stopped**
>
> Inference joins only points that are very close by default. Start each new piece where the last one ended, as precisely as you can, and the corners join themselves. When a drawing is coarser, widen `point_tol`, but not so far that it joins points you meant to keep apart.

> **CAUTION** **It adds what it sees**
>
> A line drawn half a degree off level becomes level, even if you meant it to lean, and a constraint can't be taken out of a sketch yet. When that matters, call `infer` first, and add only the suggestions you want with `add_constraint`. A constraint already in the sketch is never suggested again.

## 5.9 Common mistakes

- **A value of zero or less.** Distance, radius and diameter must be greater than zero. Adding `value: 0.0` fails with `value must be > 0, got 0`.
- **The wrong kind of point.** `PointRef::Start` and `End` work only on lines, `ArcStart` and `ArcEnd` only on arcs, and `Center` only on circles and arcs. Using `Center` on a line fails with `geometry 0 has a different kind`.
- **Rules that fight each other.** Two fixed points 30 mm apart with a distance of 40 mm between them can't both hold. The solver then stops with `solved: false` and a reason of `Stagnated`, and moves the points as close to meeting the rules as it can. A degree-of-freedom check then says `Conflicting` and names the clashing constraint (Section 5.3).
- **The wrong kind of entity.** Parallel and perpendicular take two lines. Equal takes two lines or two round pieces (circles and arcs), not one of each. Tangent takes a line and a round piece, or two round pieces, never two lines. Each of these fails with `geometry N has a different kind`.
- **An outline with a gap.** Pieces that look joined but aren't leave no region. Join every corner with coincident, including where a line meets an arc (Section 5.6).
- **Expecting inference to set sizes.** Inference adds coincident, horizontal, vertical, parallel, perpendicular and tangent only. Add distances, radii and diameters yourself (Section 5.8).
- **Too few rules.** A sketch can solve and still be under-constrained: the solver keeps unconstrained pieces near where you drew them. Check that the degrees of freedom reach zero before you rely on the sizes.

CHAPTER 6

# Solids and STEP files

> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

The part of a CAD program that does the 3D math is its *geometry kernel*. Saulitwerx talks to its kernel through one interface, the `Kernel` trait, so the rest of the program never depends on which kernel is underneath. Today that kernel is OpenCascade, reached through `OcctKernel`.

## 6.1 Make a solid and save it as STEP

| | |
|---|---|
| **YOU WILL** | Build the plate with a hole, measure it, save it as STEP and read it back. |
| **YOU NEED** | Saulitwerx built with OpenCascade (Chapter 2). |
| **TIME** | 5 minutes. |

You already ran this program in Chapter 3. Here is what each part of it does. The file is `crates/swx-kernel/examples/plate_with_hole.rs`.

1. **Make the two starting solids.** A box is given by its lowest corner and its three sizes; a cylinder stands on +Z, given by the center of its base, its radius and its height:

```
let plate = k.make_box([0.0, 0.0, 0.0], [40.0, 25.0, 5.0])?;
let tool  = k.make_cylinder([20.0, 12.5, -1.0], 5.0, 7.0)?;
```

2. **Cut one from the other.** The cylinder starts 1 mm below the plate and ends 1 mm above it, so the hole goes cleanly all the way through:

```
let part = k.cut(&plate, &tool)?;
```

3. **Measure the result** with `volume`, `surface_area`, `face_count` and `bounding_box`. The program prints them; Chapter 3 checks them by hand.

4. **Save and read back.** `export_step` writes the file, in millimeters; `import_step` reads it back as one solid. Matching volume and face count show nothing was lost:

```
k.export_step(&part, &out)?;
let back = k.import_step(&out)?;
```

**RESULT** A STEP file of the plate, checked by reading it back: `wrote plate.step (read back: 4607.30 mm³, 7 faces)`.

> **TIP** **Make the tool overlap**
>
> When you cut a hole, make the tool solid a little longer than the part on both sides. A tool that ends exactly on a face can leave a paper-thin skin, or confuse the kernel.

## 6.2 Combine solids

The kernel has three Boolean operations (Section 4.9). Each takes two solids and returns a new one; the inputs are left unchanged.

| Call | Result |
|---|---|
| `k.union(&a, &b)` | Everything in `a` or `b`. |
| `k.cut(&a, &b)` | `a` minus whatever `b` overlaps. |
| `k.intersect(&a, &b)` | Only what is in both. |

> **CAUTION**
>
> Sizes must be finite and greater than zero, and positions finite. A box with a zero size is refused before it reaches the kernel, with `invalid input: box dx must be finite and > 0, got 0`.

## 6.3 Faces, edges and vertices of a solid

A solid's faces, edges and vertices (Section 4.1) are its *sub-shapes*. Later features will need them: a fillet rounds an edge, a sketch sits on a face. Today you can count them and walk from one to its neighbours. Two sub-shapes are *adjacent* when one bounds the other: a face and the edges around it, or an edge and the two vertices at its ends.

The kernel numbers the sub-shapes of each kind from 0. `SubShape { kind, index }` names one of them, such as face 6, and `TopoKind` is the kind: `Face`, `Edge` or `Vertex`.

| | |
|---|---|
| **YOU WILL** | Count the plate's faces, edges and vertices, and find the faces around the hole. |
| **YOU NEED** | Saulitwerx built with OpenCascade (Chapter 2). |
| **TIME** | 5 minutes. |

The program is `crates/swx-kernel/examples/part_topology.rs`. It builds the same plate as above, then asks the kernel about its parts.

1. **Count each kind** with `sub_shape_count`:

```rust
for kind in [TopoKind::Face, TopoKind::Edge, TopoKind::Vertex] {
    println!("{kind:?} count: {}", k.sub_shape_count(&part, kind)?);
}
```

   A shared sub-shape counts once: the edge between two faces is one edge.

2. **Go down from a face to its edges** with `adjacent`. Give it the sub-shape to start from and the kind you want back:

```rust
let face = SubShape { kind: TopoKind::Face, index };
let edges = k.adjacent(&part, face, TopoKind::Edge)?;
```

   The list comes back sorted by index, with no repeats.

3. **Go up from an edge to its faces** the same way, asking for `TopoKind::Face`. The program does this for each edge of the hole's wall, the only face with 3 edges.
4. **Run it:**

TERMINAL

```
cargo run -p swx-kernel --features occt --example part_topology
```

You should see:

OUTPUT

```
Face count: 7
Edge count: 15
Vertex count: 10

face 0: 4 edges [0, 1, 2, 3]
face 1: 4 edges [0, 4, 5, 6]
face 2: 5 edges [1, 6, 7, 8, 9]
face 3: 4 edges [2, 7, 10, 11]
face 4: 5 edges [3, 4, 10, 12, 13]
face 5: 4 edges [5, 8, 11, 12]
face 6: 3 edges [9, 13, 14]

hole wall is face 6
  edge 9: faces [2, 6]
  edge 13: faces [4, 6]
  edge 14: faces [6]

face 7: invalid input: face index 7 is out of range: the shape has 7 faces
```

**RESULT** The plate has 7 faces, 15 edges and 10 vertices. Faces 2 and 4 have 5 edges each, four sides plus the rim of the hole: they are the top and the bottom, the two faces the hole passes through. The hole's wall, face 6, meets them along edges 9 and 13.

> **NOTE** **Why 15 edges, and an edge with one face**
>
> A box has 12 edges; the hole adds its two rims, which makes 14. The fifteenth is a *seam*: the kernel makes a closed round face, like the hole's wall, from a sheet rolled up until its two sides meet, and the line where they meet is an edge. It doesn't show in pictures. Because the wall lies on both sides of it, the seam, edge 14, lists only one face.

> **CAUTION** **A number belongs to one shape**
>
> Indices count the sub-shapes of one shape, so face 6 of this plate means nothing for another shape, even the same plate after a change. Ask again after every operation. An extrude and a Boolean can tell you which of their faces is which (Section 7.5, Section 6.4); names that last through changes are on the way, see **Coming next**.

### Common mistakes

- **Asking for the same kind you start from,** such as the edges of an edge. That gives: `invalid input: adjacent needs two different kinds, got edge and edge`.
- **Counting from 1.** The last face of a 7-face part is face 6. Face 7 gives: `invalid input: face index 7 is out of range: the shape has 7 faces`.

## 6.4 Find the faces a Boolean made

A Boolean renumbers the faces of its result (Section 6.3), so face 5 of a block need not be face 5 after a cut. To keep track of a face, such as the top you want to round next, ask for the Boolean *with history*. Next to the new solid you get one entry for each face of both inputs: first every face of `a`, then every face of `b`, each with the faces of the result it became.

An entry can list:

- **One face.** The face lives on in the result. It may be trimmed: the cutting tool's bottom becomes a groove's floor, cut to the block's width.
- **Several faces.** The face was *split*: a groove across the top leaves two pieces of top, one each side.
- **No faces.** The face was deleted: nothing of it is left on the result's surface.

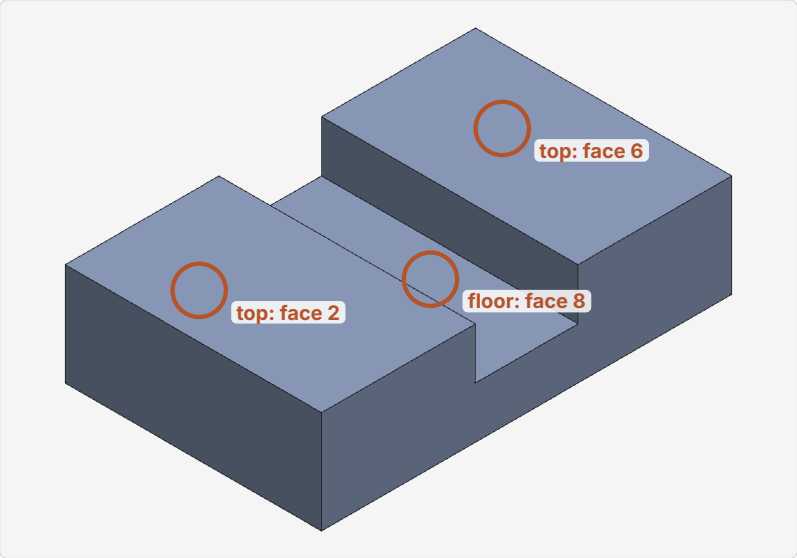



**Figure 6.1** A groove cut across a block, with history. The block's top, face 5, was split into faces 2 and 6 of the result. The tool's bottom became the groove's floor, face 8.

| | |
|---|---|
| YOU WILL | Cut a groove across a block with history, and find what became of each face. |
| YOU NEED | Saulitwerx built with OpenCascade (Chapter 2). |
| TIME | 5 minutes. |

1. Run the sample program, and tell it to save the part as `groove.step`:

TERMINAL

```
cargo run -p swx-kernel --features occt --example boolean_history -- groove.step
```

It prints the face counts, one line for each face of the block and of the tool, then the groove's floor:

OUTPUT

```
block 6 faces, tool 6 faces, part 10 faces
block face 0  x 0..0  y 0..25  z 0..10     one face   -> [0]
block face 1  x 40..40  y 0..25  z 0..10   one face   -> [5]
block face 2  x 0..40  y 0..0  z 0..10     one face   -> [1]
block face 3  x 0..40  y 25..25  z 0..10   one face   -> [3]
block face 4  x 0..40  y 0..25  z 0..0     one face   -> [4]
block face 5  x 0..40  y 0..25  z 10..10   split in 2 -> [2, 6]
tool  face 0  x 15..15  y -1..26  z 5..11  one face   -> [9]
tool  face 1  x 25..25  y -1..26  z 5..11  one face   -> [7]
tool  face 2  x 15..25  y -1..-1  z 5..11  deleted    -> []
tool  face 3  x 15..25  y 26..26  z 5..11  deleted    -> []
tool  face 4  x 15..25  y -1..26  z 5..5   one face   -> [8]
tool  face 5  x 15..25  y -1..26  z 11..11 deleted    -> []
groove floor is part face 8  x 15..25  y 0..25  z 5..5
part volume 8750.00 mm³
…  (the geometry engine's file report)
wrote groove.step
```

2. Read the line `block face 5`. It lies flat at z = 10: the block's top. The groove split it in two, faces 2 and 6 of the part.
3. Read the `tool` lines. The tool's two walls at x = 15 and x = 25 became the groove's walls, faces 9 and 7. Its bottom at z = 5 became the floor, face 8. Its two ends and its top stuck out of the block, so nothing of them is left.
4. Count the part's faces: the block's 6, plus one more piece of top, plus the tool's 3 that survive, make 10.

**RESULT** Every face of the grooved block traced back to the face it came from.

The sample calls `boolean_tracked` with the operation to do, `BooleanOp::Union`, `Cut` or `Intersect`, and the two solids. It returns the solid, the same as `union`, `cut` or `intersect` would, and an `OpHistory`:

```
let (part, history) = k.boolean_tracked(BooleanOp::Cut, &block, &tool)?;
for (operand, face, images) in &history.modified {
    // operand: Operand::A (the block) or Operand::B (the tool)
    // face: a face of that input; images: the faces of `part` it became
}
```

**In Saulitwerx today:** history covers faces only. Edges are found through their faces (Section 6.3). `history.generated` is always empty for a Boolean, because a Boolean makes no new faces, only the edges where the two solids meet. A cut or join made by an extrude (Section 7.3) doesn't report a history yet, and the part's history doesn't keep it.

## 6.5 Parts with several bodies

A solid doesn't have to be in one piece. Cut a slot right across a plate and you get two pieces that don't touch, but they are still one result, one shape. Each separate piece is a *body*. A part with more than one is a *multi-body part*. You meet them when a cut splits a part, and you can build them on purpose: to make two halves that fit together, or to shape several pieces side by side and pull them apart at the end.

Three calls work with bodies:

- `body_count` says how many bodies a shape has.
- `bodies` hands you each body as its own shape, to measure, save or use on its own.

- `combine` applies one Boolean to a target with a whole list of tools, in order: one call drills every hole, or joins every boss. `combine_tracked` does the same and says what became of every face, like a Boolean with history (Section 6.4).

| | |
|---|---|
| **YOU WILL** | Drill two holes in one call, see what became of each face, split the plate in two, and take the two bodies apart. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its part in a folder called `bodies`:

   TERMINAL
   ```
   cargo run -p swx-kernel --features occt --example multi_body -- bodies
   ```

   It prints the drilled plate and the history of its faces, the slotted plate and its two bodies, three errors on purpose, and the file it wrote:

   OUTPUT
   ```
   drilled plate: 13796.81 mm³, 8 faces, 1 body
   plate face 0 -> [0]
   plate face 1 -> [5]
   plate face 2 -> [1]
   plate face 3 -> [3]
   plate face 4 -> [4]
   plate face 5 -> [2]
   pin 0 faces -> [[6], [], []]
   pin 1 faces -> [[7], [], []]
   slotted plate: 12836.81 mm³, 2 bodies
   body 0: 6418.41 mm³, x 0..28
   body 1: 6418.41 mm³, x 32..60
   no tools -> error: invalid input: combine needs at least one tool body
   flat tool -> error: invalid input: combine tool 0 has no solid body
   far tool -> error: empty result: the combined result encloses no volume
   …  (the geometry engine's file report)
   wrote bodies/two-bodies.step
   ```

2. Read the first line. The plate is 60 × 40 × 6 = 14400 mm³, and each hole 8 mm across takes π × 4² × 6 = 301.59 mm³: 14400 − 603.19 = 13796.81 mm³. Still one body.
3. Read the `plate face` lines. Each of the plate's 6 faces lives on, renumbered. The top and the bottom now have holes in them, but each is still one face.
4. Read the `pin` lines. Each pin is a cylinder with three faces: its round side, then its two ends. The round side became the wall of its hole, face 6 or 7. The ends were outside the plate, so nothing of them is left.
5. Read the `slotted plate` lines. The slot removed 4 × 40 × 6 = 960 mm³, and the plate fell into 2 bodies. `bodies` gives them in order along X: the left half, from x = 0 to 28, then the right half, from 32 to 60. Each holds half the volume.
6. Read the errors. A combine needs at least one tool, every tool must be a solid (not a flat face), and an intersect with a tool that doesn't touch the plate leaves nothing.
7. Draw the part:

   TERMINAL
   ```
   target/release/swx-qa render --step bodies/two-bodies.step -o two-bodies.png
   ```

   Open `two-bodies.png`. You should see the two halves of Figure 6.2.

**RESULT** Two holes from one call, a part split into two bodies, and each body in hand on its own.

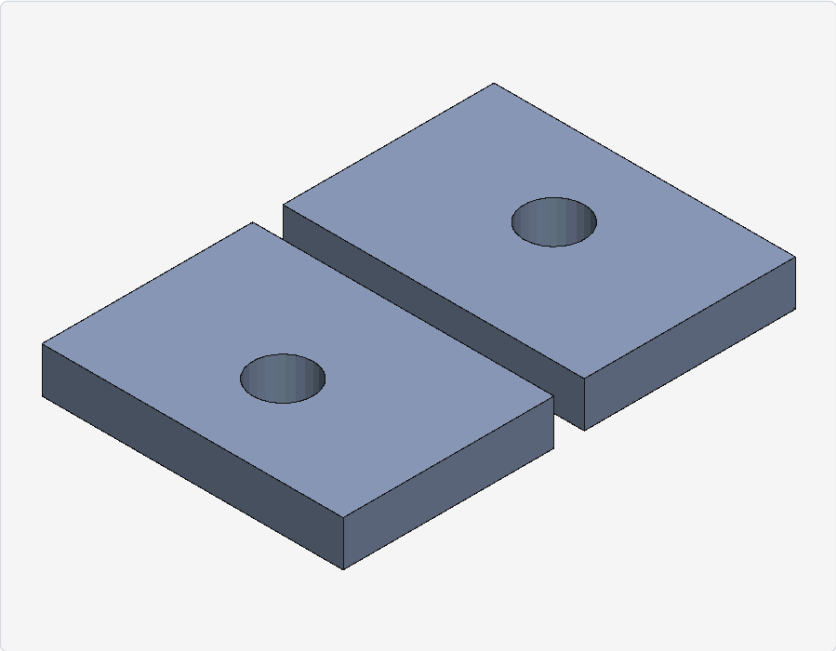


**Figure 6.2** A 60 × 40 × 6 mm plate with two holes, drilled in one combine call, then cut by a 4 mm slot across the middle. The result is one shape with two bodies.

### How the sample is built

The program is `crates/swx-kernel/examples/multi_body.rs`. It does three things.

1. **Combine with history.** The tools go in a list. The result is the same as cutting with each pin in turn:

```
let (drilled, history) = k.combine_tracked(BooleanOp::Cut, &plate, &pins)?;
// history.target[f]: the faces of `drilled` that face f of the plate became
// history.tools[t][f]: the same for face f of tool t
```

2. **Split.** A plain `combine` with one tool, the slot, cuts the plate in two:

```
let split = k.combine(BooleanOp::Cut, &drilled, &[slot])?;
```

3. **Count and take apart:**

```
let n = k.body_count(&split)?;      // 2
for body in k.bodies(&split)? { /* measure, save or use each one */ }
```

> **TIP** **One call for a row of features**
>
> To drill ten holes or join ten bosses, build the ten tools and pass them to one `combine`. It's shorter than ten Booleans, and `combine_tracked` still tells you which hole wall came from which tool.

> **CAUTION** **Count the bodies after a cut**
>
> A cut can split a part without warning: a slot or a hole that runs right across leaves two pieces in one shape. If a later feature should touch the whole part, check `body_count` first.

**In Saulitwerx today:** the bodies of a shape can be counted, taken apart and combined, but not named, moved or picked, and a part's history doesn't keep them. `bodies` orders them by the lowest corner of each one's bounding box, so the order follows where they are, not how they were made.

## 6.6 Open a STEP file from another program

`import_step` reads any STEP file, and `swx-qa render --step` draws one (Chapter 16). Everything in the file comes in as one shape. Saulitwerx can't edit an imported shape yet; using it as a starting feature is on the roadmap.

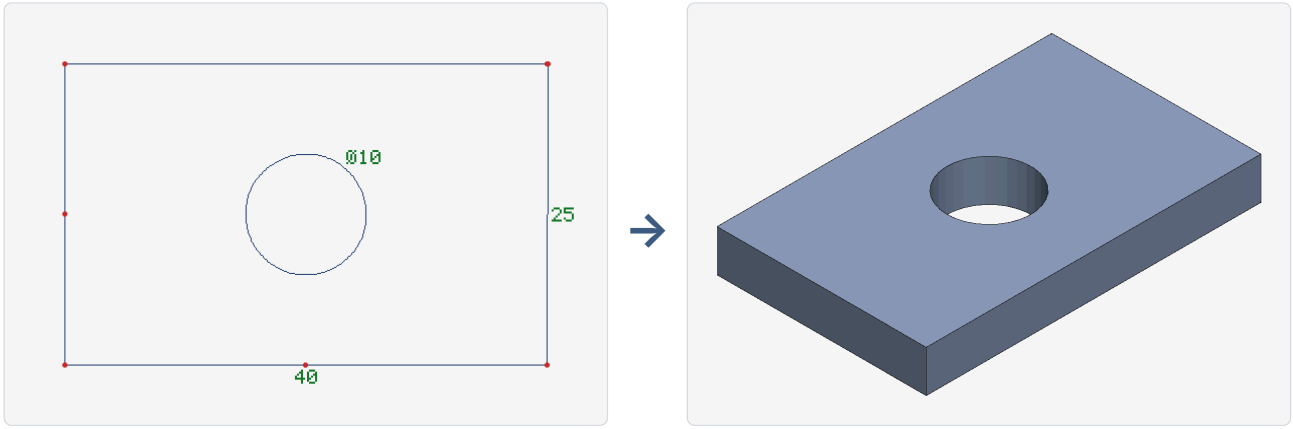
CHAPTER 7

# Extruding a profile

> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

An *extrude* takes a flat shape and pushes it straight out into 3D, the way a cookie cutter pressed into dough makes a shape with straight sides. It is the most common way to start a part: draw the outline, then give it a thickness.

Think back to the plate of Chapter 3. There you made it in two steps: a block, then a cylinder cut out of it. With an extrude you draw the plate's outline once, rectangle and hole together, and push it up 5 mm. The hole comes with it, with no separate cut (Figure 7.1).




Profile: rectangle with a hole — Extruded 5 mm

**Figure 7.1** An extrude pushes a flat profile straight out. The circle inside the rectangle becomes a hole all the way through.

Two more words come with it:

- The *workplane* is the flat plane the outline is drawn on. It has its own left–right and up–down directions, and a third direction straight out of it, called its *normal*. The extrude goes along the normal.
- The *extent* says how far the extrude goes: a distance one way, a length split evenly on both sides of the workplane, or up to a face of the part.

**In Saulitwerx today:** the geometry engine can make a flat face from outlines of lines, arcs and circles on any workplane, and extrude it by a distance or symmetrically. The result can be a new solid, or be joined to a part, cut from it or intersected with it in the same step (Section 7.3). A cut can also go through the whole part, however thick it is (Section 7.4), an extrude can end on a face of the part instead of at a distance (Section 7.6), and a plain extrude can report which face each piece of the outline became (Section 7.5). You type the outline's pieces into a small program; a sketch's regions (Section 5.7) can't be handed to the extrude yet, and the extrude isn't recorded in a part's history.

## 7.1 Extrude a profile

| | |
|---|---|
| YOU WILL | Make the plate with a hole from its outline, in one extrude, and save it as STEP. |
| YOU NEED | Saulitwerx built with OpenCascade (Chapter 2). |
| TIME | 5 minutes. |

1. Run the sample program, and tell it to save the part as `extruded-plate.step`:

   TERMINAL

   ```
   cargo run -p swx-kernel --features occt --example extrude_plate -- extruded-plate.step
   ```

   It prints the solid's measurements, then the geometry engine's report about writing the file:

   OUTPUT

   ```
   volume        4607.30 mm³
   faces         7
   bounding box  [0.0, 0.0, 0.0] to [40.0, 25.0, 5.0]
   …  (the geometry engine's file report)
   wrote extruded-plate.step
   ```

2. Compare the numbers with the plate you made in Chapter 3. They are the same: 4607.30 mm³, 7 faces, 40 × 25 × 5 mm. The region's area, 921.46 mm² (Section 5.7), times the 5 mm height gives the volume.

3. Draw the part:

   TERMINAL

   ```
   target/release/swx-qa render --step extruded-plate.step -o extruded-plate.png
   ```

   Open `extruded-plate.png`. You should see the plate of Figure 7.1, with the hole showing light background through it.

**RESULT** The plate with a hole, made from its outline in one step instead of a block and a cut.

### How the sample is built

The program is `crates/swx-kernel/examples/extrude_plate.rs`. It does three things.

1. **Describe each loop.** A `ProfileLoop` is a list of pieces, `Segment2`, in workplane coordinates. Each piece starts where the one before it ended, and the last one ends where the first began. A full circle is a loop on its own:

   ```
   let corners = [[0.0, 0.0], [40.0, 0.0], [40.0, 25.0], [0.0, 25.0]];
   let outer = ProfileLoop {
       segments: (0..4)
           .map(|i| Segment2::Line { start: corners[i], end: corners[(i + 1) % 4] })
           .collect(),
   };
   let hole = ProfileLoop {
       segments: vec![Segment2::Circle { center: [20.0, 12.5], radius: 5.0 }],
   };
   ```

2. **Make the face.** `make_face` takes the workplane, the outer loop, and a list of holes. The hole loops must lie inside the outer one:

   ```
   let face = k.make_face(&Workplane::XY, &outer, &[hole])?;
   ```

3. **Extrude it.** `Extent::Distance(5.0)` pushes the face 5 mm along the workplane's normal, which for the XY plane is up, +Z:

```rust
let part = k.extrude(&face, Extent::Distance(5.0))?;
```

> **TIP** **Direction doesn't matter**
>
> You can list a loop's pieces clockwise or counterclockwise, and any piece may run backwards. Saulitwerx puts them in order before the geometry engine sees them. What matters is that the ends meet: closer than 0.0000001 mm.

## 7.2 Choose the direction and the workplane

An extrude doesn't have to go up. A negative distance goes the other way, below the workplane. A symmetric extent splits the length evenly, half on each side, which is handy for a part whose middle should sit on the plane, like a bracket centered on its mounting plate.

| | |
|---|---|
| **YOU WILL** | Extrude a slot up, down and symmetrically, and see where each result sits. |
| **YOU NEED** | Saulitwerx built with OpenCascade. |
| **TIME** | 5 minutes. |

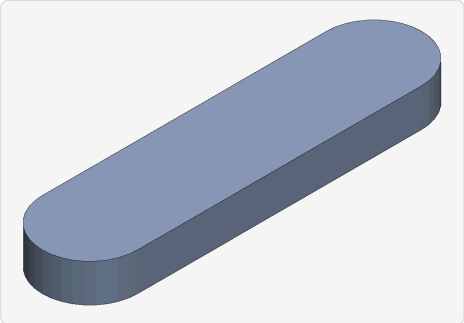


**Figure 7.2** The slot you will make: two lines and two half-circles, extruded 4 mm.

1. Run the slot sample:

   TERMINAL

   ```
   cargo run -p swx-kernel --features occt --example extrude_slot -- extruded-slot.step
   ```

   It extrudes the slot three ways, 4 mm each, and prints each result's volume and height:

   OUTPUT

   ```
   Distance(4.0)   1514.16 mm³, z from 0 to 4
   Distance(-4.0)  1514.16 mm³, z from -4 to 0
   Symmetric(4.0)  1514.16 mm³, z from -2 to 2
   …  (the geometry engine's file report)
   wrote extruded-slot.step
   ```

2. Read the three lines. The volume is the same each time; only the position changes. `Distance(4.0)` sits on the workplane and rises to 4 mm; `Distance(-4.0)` hangs below it; `Symmetric(4.0)` straddles it, 2 mm each side.

3. Draw the first one:

   TERMINAL

   ```
   target/release/swx-qa render --step extruded-slot.step -o extruded-slot.png
   ```

You should see the slot of Figure 7.2: flat sides and round ends.

**RESULT** You can place an extrude above, below or across its workplane.

The slot's outline mixes lines and arcs. An arc, `Segment2::Arc`, runs counterclockwise from `start_angle` to `end_angle`, in radians measured from the workplane's left–right axis, just as in a sketch (Section 5.6). The right-hand end of the slot is the arc from $-\pi/2$ to $\pi/2$ around (30, 5).

Three workplanes are built in. To draw on another plane, give your own `Workplane` with an `origin`, an `x_dir` and a `normal` at right angles to it (Section 19.6).

| Workplane | Left–right | Up–down | Extrude goes |
|---|---|---|---|
| `Workplane::XY` | +X | +Y | +Z |
| `Workplane::XZ` | +X | +Z | −Y |
| `Workplane::YZ` | +Y | +Z | +X |

**CAUTION** **XZ extrudes toward −Y**

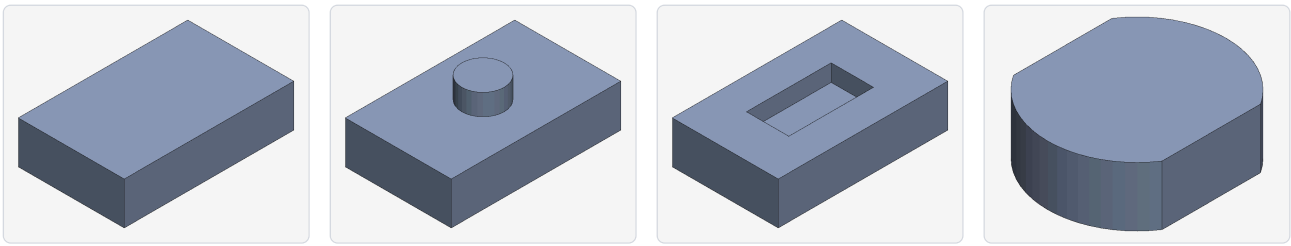
Looking down on the XZ plane so that X runs right and Z runs up, you are looking from the −Y side, and that is where a positive distance goes. If a part comes out on the wrong side, use a negative distance.

## 7.3 Join, cut or intersect with an extrude

Most extrudes change a part that already exists. A boss stands up from a plate; a pocket sinks into it. You could extrude a new solid and then combine it with a Boolean operation (Chapter 6), but an extrude can do both in one step. Its *extrude mode* says what happens to the extruded shape:

| Mode | What you get |
|---|---|
| New solid | The extruded shape on its own. The part is left as it was. |
| Join | The part with the extruded shape added to it. |
| Cut | The part with the extruded shape taken out of it. |
| Intersect | Only the part's material that lies inside the extruded shape. |

The part an extrude joins, cuts or intersects is its *target body*. A good habit: draw the outline on a face of the target, then extrude away from the part to add material, and into it to remove material. Figure 7.3 shows one block and the three combining modes.



Target: 40 × 25 × 10 mm block | Join: circle, 5 mm up | Cut: rectangle, 4 mm down | Intersect: circle, 10 mm down

**Figure 7.3** One target and three extrude modes, each outline drawn on the block's top face. The intersect keeps the part of the block inside a 30 mm circle: the block's long sides trim the circle flat. Each picture is scaled to fill its frame.

**YOU WILL** Add a boss to a block, sink a pocket into it, and trim it to a circle, each with one extrude.
**YOU NEED** Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2).
**TIME** 10 minutes.

1. Run the sample program, and tell it to save its parts in a folder called `modes`:

   TERMINAL
   ```
   cargo run -p swx-kernel --features occt --example extrude_modes -- modes
   ```

   It prints a line for the block and one for each mode, then two errors on purpose, then the files it wrote:

   OUTPUT
   ```
   target     10000.00 mm³   6 faces  z from 0 to 10
   join       10392.70 mm³   8 faces  z from 0 to 15
   cut         9200.00 mm³  11 faces  z from 0 to 10
   intersect   6505.89 mm³   7 faces  z from 0 to 10
   join, no target -> error: invalid input: extrude mode "join" needs a target body
   cut, all of it -> error: empty result: extrude cut leaves no solid (volume 0e0)
   …  (the geometry engine's file reports)
   wrote modes/modes-target.step
   wrote modes/modes-join.step
   wrote modes/modes-cut.step
   wrote modes/modes-intersect.step
   ```

2. Read the `join` line. The block grew by the boss: a 10 mm circle 5 mm tall adds 392.70 mm³, and the part now reaches 15 mm up.
3. Read the `cut` line. A 20 × 10 mm rectangle cut 4 mm deep removes 800 mm³. The height stays 10 mm, and the pocket's floor and four walls bring the faces to 11.
4. Read the `intersect` line. What is left is the block's material inside a 30 mm circle: less than either one on its own.
5. Draw the cut result:

   TERMINAL
   ```
   target/release/swx-qa render --step modes/modes-cut.step -o modes-cut.png
   ```

   Open `modes-cut.png`. You should see the block with a pocket in its top, as in Figure 7.3. Draw the other files the same way.

**RESULT** A boss, a pocket and a trimmed block, each made by one extrude against the same target. The block itself is never changed: each mode returns a new solid.

### How the sample is built

The program is `crates/swx-kernel/examples/extrude_modes.rs`. The outlines sit on the block's top face, a workplane at z = 10:

```
const TOP: Workplane = Workplane {
    origin: [0.0, 0.0, 10.0],
    x_dir: [1.0, 0.0, 0.0],
    normal: [0.0, 0.0, 1.0],
};
```

Each mode is one call to `extrude_feature`. It takes the face, the extent, the mode and the target. The join goes up, +5 mm, away from the block; the cut goes down, −4 mm, into it:

```
let joined = extrude_feature(&k, &boss, Extent::Distance(5.0),
    ExtrudeMode::Join, Some(&block))?;
let cut = extrude_feature(&k, &pocket, Extent::Distance(-4.0),
    ExtrudeMode::Cut, Some(&block))?;
```

For `ExtrudeMode::NewSolid` the target isn't used, so pass `None`. That gives the same solid as `extrude` (Section 7.1).

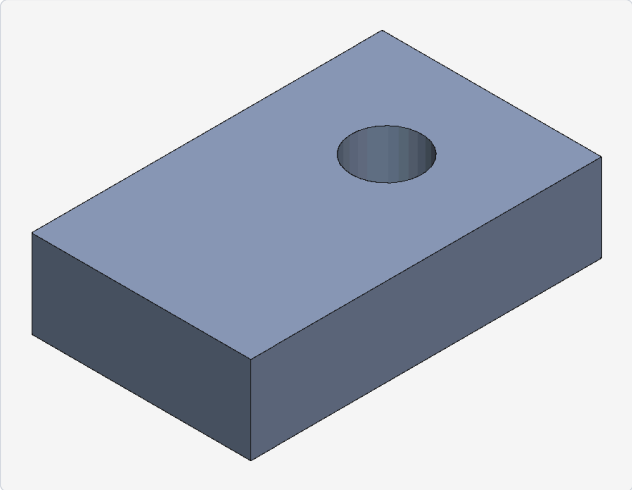
> **TIP** **Which way is "into" the part?**
>
> The extrude goes along the workplane's normal for a positive distance. On a top face whose normal points up, a cut needs a negative distance. If a cut leaves the part unchanged, it probably went the wrong way, into empty space.

> **CAUTION** **A result must have something left**
>
> A cut that removes the whole target, or an intersect whose shapes don't overlap, leaves nothing. That is an error, `empty result: …`, not an empty part. The sample's last line shows one: a rectangle larger than the block, cut all the way through.

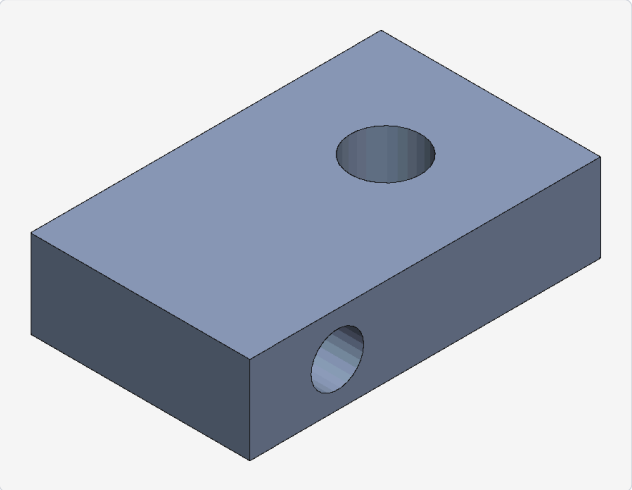
## 7.4 Cut through the whole part

Holes for bolts and pins usually go all the way through a part. You could measure the part and cut exactly that deep, but if the part later gets thicker, the hole would stop short. A *through-all* cut has no depth of its own: it goes right through the target body, whatever its size.

A through-all cut still needs a direction. It can go along the workplane's normal, against it, or both ways at once. Both ways is the one to use when the outline sits on a plane in the middle of the part (Figure 7.4).



Down from the top face (reversed)



Then both ways from the middle

**Figure 7.4** Two through-all cuts in a 40 × 25 × 10 mm block. The first hole is drawn on the top face and cut against its normal, down. The second is drawn on a plane through the middle of the block and cut both ways, out of the front and the back.

| | |
|---|---|
| **YOU WILL** | Cut two holes right through a block, one from its top face and one from a plane in its middle. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `through`:

   TERMINAL

   ```
   cargo run -p swx-kernel --features occt --example extrude_through_all -- through
   ```

   It prints a line for the block and one for each cut, then two errors on purpose, then the files it wrote:

OUTPUT

```
block            10000.00 mm³   6 faces
down (reversed)   9497.35 mm³   7 faces
both ways         8790.49 mm³   8 faces
up (normal) -> error: invalid input: the through-all cut misses the target: it removes nothing
join -> error: invalid input: a through-all extent works only with the cut mode, not "join"
…  (the geometry engine's file reports)
wrote through/through-down.step
wrote through/through-both.step
```

2. Read the `down` line. An 8 mm hole through 10 mm of block removes 502.65 mm³, and adds one face: the hole's wall.
3. Read the `both ways` line. A 6 mm hole through the block's 25 mm width removes another 706.86 mm³. The cut started in the middle of the block and went out through both sides.
4. Read the first error. From the top face, the normal points up, away from the block. A through-all cut that way finds nothing to remove, and says so.
5. Draw the result with both holes:

   TERMINAL

   ```
   target/release/swx-qa render --step through/through-both.step -o through-both.png
   ```

   Open `through-both.png`. You should see the block of Figure 7.4, with a hole down through its top and a smaller one through its side.

**RESULT** Two holes that go right through the block. If the block were 50 mm thick instead of 10, the same two cuts would still go through it.

### How the sample is built

The program is `crates/swx-kernel/examples/extrude_through_all.rs`. The extent is `Extent::ThroughAll`, with a direction:

| Direction | The cut goes |
|---|---|
| `ThroughDirection::Normal` | Along the workplane's normal. |
| `ThroughDirection::Reversed` | Against the normal. |
| `ThroughDirection::Both` | Both ways from the workplane. |

The first hole is drawn on the top face, so it goes against the normal, down into the block:

```
let drilled = extrude_feature(&k, &top_hole,
    Extent::ThroughAll(ThroughDirection::Reversed),
    ExtrudeMode::Cut, Some(&block))?;
```

The second is drawn on a workplane through the middle of the block, at y = 12.5, and cut from the result of the first:

```
let cross = extrude_feature(&k, &cross_hole,
    Extent::ThroughAll(ThroughDirection::Both),
    ExtrudeMode::Cut, Some(&drilled))?;
```

> **NOTE** **How far is “all”?**
>
> Saulitwerx measures the outline and the target, and makes the cut a little longer than the distance needed to clear the whole target. That is why a through-all extent works only with `extrude_feature` and the cut mode: it needs a target to measure. `extrude` on its own refuses it.

> **TIP** **A missed cut is an error**
>
> If a through-all cut removes nothing, Saulitwerx refuses it instead of handing back the part unchanged, so a cut that went the wrong way never passes unnoticed. Change the direction, or move the outline over the part.

## 7.5 Find the faces an extrude made

Most features after the first one point at something already in the part: round *this* edge, draw on *that* face, cut down to *this* face. The kernel numbers every face and edge (Section 6.3), but the numbers come from the geometry engine, not from you, so you can’t tell in advance which number the top of the plate gets.

An extrude can tell you. Ask for it *with history*, and next to the solid you get a record of where each face came from. Faces of an extrude come in two sorts:

- The two *caps*: the flat faces at each end of the push. On the plate, the bottom and the top.
- One *side face* for each edge of the outline: the wall it swept as it moved. A straight edge sweeps a flat wall; the hole’s circle sweeps a round one.

Follow one piece of your outline through both steps, and you know its face: the rectangle’s line at x = 40 became edge 1 of the flat face, and edge 1 swept face 1 of the plate (Figure 7.5).

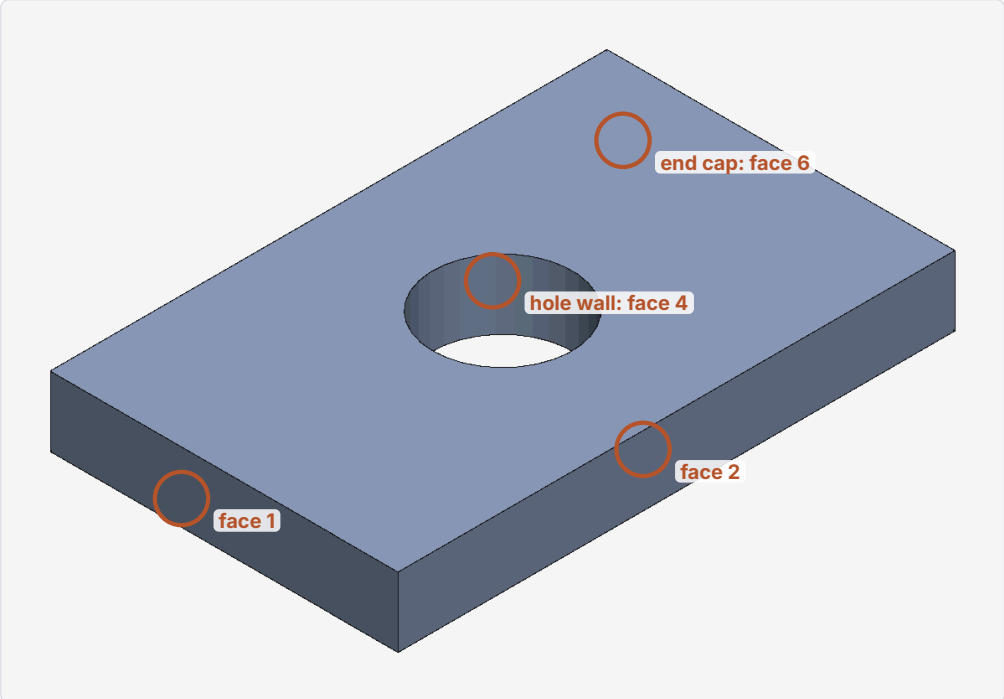



**Figure 7.5** The plate’s faces, as its extrude history names them. Face 1 was swept by the rectangle’s line at x = 40, face 2 by its line at y = 25, face 4 by the hole’s circle. The start cap, face 5, is underneath.

**In Saulitwerx today:** the geometry engine keeps this record for a plain extrude, the one that makes a new solid. You get it in your own program. The extrude with join, cut or intersect doesn't give one yet, the part's history doesn't keep it, and no later feature can use it to find a face yet.

| | |
|---|---|
| **YOU WILL** | Extrude the plate with history, and find which face each piece of its outline became. |
| **YOU NEED** | Saulitwerx built with OpenCascade (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program:

TERMINAL

```
cargo run -p swx-kernel --features occt --example extrude_history
```

It prints the two caps, then one line for each piece of the outline, then where the top is:

OUTPUT

```
start cap     face 5
end cap       face 6
outer 0  ->  edge 0  ->  face 0   x 0..40  y 0..0  z 0..5
outer 1  ->  edge 1  ->  face 1   x 40..40  y 0..25  z 0..5
outer 2  ->  edge 2  ->  face 2   x 0..40  y 25..25  z 0..5
outer 3  ->  edge 3  ->  face 3   x 0..0  y 0..25  z 0..5
hole 0  ->  edge 4  ->  face 4   x 15..25  y 7.5..17.5  z 0..5
end cap is    x 0..40  y 0..25  z 5..5
```

2. Read the line `outer 1`. The second line of the rectangle, from (40, 0) to (40, 25), became edge 1 of the face, and edge 1 swept face 1. The box after it shows where that face is: flat at x = 40, the full 25 mm wide and 5 mm tall. That is the short side at the front left in Figure 7.5.
3. Read the line `hole 0`. The circle swept face 4, the hole's wall: 10 mm across and 5 mm tall, around the hole's center.
4. Read the last line. The end cap lies flat at z = 5 mm: it is the top of the plate.

**RESULT** Every face of the plate, named by the piece of the outline that made it. Faces 0 to 4 are the sides, 5 the bottom and 6 the top.

## How the sample is built

The program is `crates/swx-kernel/examples/extrude_history.rs`. It uses the outline of `extrude_plate`, and three calls do the work.

1. **Make the face with history.** `make_face_tracked` takes the same inputs as `make_face`. It also returns, for each loop (the outer one first, then the holes), the edge each piece became, in the order you listed the pieces:

```
let (face, edges) = k.make_face_tracked(&Workplane::XY, &outer, &[hole])?;
```

Here `edges[0][1]` is the edge made from the outer loop's second piece.

2. **Extrude it with history.** `extrude_tracked` takes the same inputs as `extrude`, and returns an `ExtrudeHistory` with the solid:

```
let (part, history) = k.extrude_tracked(&face, Extent::Distance(5.0))?;
```

`history.start_cap` and `history.end_cap` are the two caps. `history.sides` holds one pair per edge of the face: the edge, then the side face it swept.

3. **Follow an edge to its face**, and ask where that face is. `sub_shape_bounding_box` gives the smallest axis-aligned box around one face, edge or vertex:

```
let side = history.sides.iter()
    .find(|(e, _)| e == edge)
    .map(|(_, f)| *f)
    .ok_or("edge not in history")?;
let bb = k.sub_shape_bounding_box(&part, side)?;
```

**CAUTION** **The start cap isn't always the face you drew**

The sweep runs from the start cap to the end cap, along the workplane's normal. With a positive distance, the start cap is where your outline was drawn. With a negative one, the sweep starts below it: the end cap is the face you drew, and the start cap is the far end. With a symmetric extent, neither cap is on the workplane. When it matters, check with `sub_shape_bounding_box`, as the sample does.

**NOTE** **Same plate, other numbers**

The plate of Section 6.3, made as a block with a cylinder cut out, has its hole's wall as face 6. The extruded plate has it as face 4. The two shapes are the same; the steps that made them differ, and so do the numbers. That is why a number is only good for the shape it came from, and why the history is worth having: it tells you which number is which.

**TIP** **Your order, not the engine's**

However you list a loop's pieces, clockwise or counterclockwise, the edges come back in your order: the second piece you gave is always `edges[loop][1]`. You don't have to know how Saulitwerx reordered the loop.

## 7.6 Extrude up to a face

Often the depth you want isn't a number but a place on the part: a post that should end level with the top, a pocket that should go down exactly to a step. You could measure the height and type it in, but if the part changes later, the number is wrong. An extent can name a face of the target body instead. A *to-face* extent goes from the workplane to that face. A *between-faces* extent goes from one face to another, wherever the outline was drawn.

The face has to be flat and parallel to the workplane, like a step or a top face. The extrude ends on the plane that face lies on, so the face can be on either side of the workplane: Saulitwerx works out which way to go.

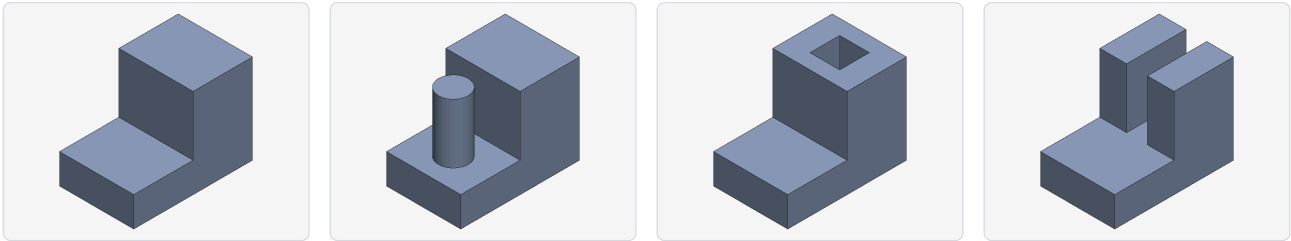



**Figure 7.6** Three extrudes whose depth comes from faces of the target. The step face is the top of the low part, 10 mm up; the top face is the top of the tall part, 30 mm up. Each picture is scaled to fill its frame.

**In Saulitwerx today:** you give the face by its number in the target (Section 6.3), so your program finds it first. The face must be flat; a round face, or one standing at an angle to the workplane, is refused. The face's number belongs to the target as it is, so if the target changes, find the face again.

| | |
|---|---|
| **YOU WILL** | Add a post that ends level with a part's top, a pocket that stops at its step, and a slot between the two, each with one extrude. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 10 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `toface`:

   TERMINAL
   ```
   cargo run -p swx-kernel --features occt --example extrude_to_face -- toface
   ```

   It prints the block and its two faces, a line for each extrude, then two errors on purpose, then the files it wrote:

   OUTPUT
   ```
   stepped block  20000.00 mm³  14 faces  z from 0 to 30
   step face is face 12, top face is face 7
   post to top    21570.80 mm³  16 faces  z from 0 to 30
   pocket to step 18000.00 mm³  19 faces  z from 0 to 30
   slot between   17200.00 mm³  20 faces  z from 0 to 30
   from 10 to 30   2800.00 mm³   6 faces  z from 10 to 30
   new solid -> error: invalid input: a to-face or between-faces extent needs a target body: it
   works with the join, cut and intersect modes, not "new solid"
   side face -> error: invalid input: target face 11 (normal [1.0, 0.0, -0.0]) is not parallel to
   the workplane (normal [0.0, 0.0, 1.0])
   …  (the geometry engine's file reports)
   wrote toface/toface-target.step
   wrote toface/toface-post.step
   wrote toface/toface-pocket.step
   wrote toface/toface-slot.step
   ```

2. Read the second line. The program looked through the block's faces and found the step face, number 12, and the top face, number 7.
3. Read the `post to top` line. The post is drawn on the step face and joined up to the top face: a 10 mm circle 20 mm tall adds 1570.80 mm³. Nobody typed 20 mm; it is the distance between the two faces.
4. Read the `pocket to step` line. The pocket is drawn on the top face and cut to the step face. The step face is below the outline, so the cut goes down, 20 mm, and removes 10 × 10 × 20 = 2000 mm³.
5. Read the `slot between` line. The slot is drawn on the XY plane at the very bottom, z = 0, but it is cut only between the step face and the top face: a 20 × 7 mm rectangle, 20 mm tall, removes 2800 mm³ and splits the tall part in two.
6. Draw the slot result:

   TERMINAL
   ```
   target/release/swx-qa render --step toface/toface-slot.step -o toface-slot.png
   ```

   Open `toface-slot.png`. You should see the block of Figure 7.6 with its tall part split by a slot. Draw the other files the same way.

**RESULT** Three extrudes whose depths come from the part itself. If the tall part were 50 mm high instead of 30, the post would still end level with its top, and the slot would still reach the step.

### How the sample is built

The program is `crates/swx-kernel/examples/extrude_to_face.rs`. It does two new things.

1. **Find the faces.** `face_plane(s, face)` gives a flat face's plane: a point on it and its *outward normal*, the direction pointing out of the solid. The sample's `find_face` goes through every face and keeps the one whose normal points up and whose plane is at the height it wants. A round face has no plane, and `face_plane` refuses it, so the search skips it:

```
for index in 0..k.sub_shape_count(s, TopoKind::Face)? {
    let face = SubShape { kind: TopoKind::Face, index };
    if let Ok((p, n)) = k.face_plane(s, face) {
        if dot(n, normal) > 0.999_999 && (dot(p, normal) - at).abs() < 1e-6 {
            return Ok(face);
        }
    }
}
```

2. **Name the faces in the extent.** `Extent::ToFace(face)` and `Extent::BetweenFaces(first, second)` take faces of the target, and work with `extrude_feature` in the join, cut and intersect modes:

```
let joined = extrude_feature(&k, &post, Extent::ToFace(top),
    ExtrudeMode::Join, Some(&stepped))?;
let slotted = extrude_feature(&k, &slot, Extent::BetweenFaces(step, top),
    ExtrudeMode::Cut, Some(&stepped))?;
```

Behind the scenes, Saulitwerx measures how far each face's plane is from the workplane, along its normal, and turns the face extent into a plain one. A to-face extent becomes a distance: +20 mm for the post, −20 mm for the pocket. A between-faces extent becomes a *from–to* extent, `Extent::FromTo(start, end)`: from 10 mm to 30 mm along the normal for the slot. You can use a from–to extent yourself, with no target at all, as the sample's `from 10 to 30` line does:

```
let block = k.extrude(&slot, Extent::FromTo(10.0, 30.0))?;
```

> **TIP** **The order of the two faces doesn't matter**
>
> Either order gives the same slot, from the step up to the top or from the top down to the step. The same goes for the two numbers of a from–to extent.

> **CAUTION** **A face extent needs a target**
>
> The faces belong to the target body, so a to-face or between-faces extent works only with `extrude_feature` and the join, cut or intersect mode. With the new-solid mode, as in the sample's first error, or with `extrude` or `extrude_tracked`, it is refused.

## 7.7 Common mistakes

- **The ends don't quite meet.** A loop whose last piece stops short of the first is refused with `outer loop: consecutive segments don't meet within 0.0000001`. Compute shared corners once and reuse them, as `extrude_plate` does with `corners`.
- **A circle mixed with other pieces.** A full circle must be a loop of its own. To round a corner, use an arc.

- **A hole outside the outline, or touching its edge.** The geometry engine refuses the face: `make_face failed: the face is invalid …`. Keep every hole wholly inside the outer loop.
- **An outline that crosses itself,** like a figure eight. Same message; or, when the two halves are the same size, `outer loop encloses no area`, because their areas cancel out. Split it into separate outlines.
- **A distance of zero.** An extrude has to go somewhere: `extrude distance must be finite and non-zero, got 0`.
- **Join, cut or intersect with no target.** These modes need a part to work on: `invalid input: extrude mode "cut" needs a target body`. Pass the part as `Some(&part)`, or use `ExtrudeMode::NewSolid`.
- **A cut that goes the wrong way.** It misses the part, and the part comes back unchanged. Check the sign of the distance (Section 7.3).
- **A cut that removes everything,** or an intersect that doesn't overlap: `empty result: extrude cut leaves no solid …`. Make the outline smaller than the face it cuts, or move it over the part.
- **A through-all extent with another mode.** Only a cut can go through all: `a through-all extent works only with the cut mode, not "join"`. Give a distance for a join or an intersect.
- **A through-all extent with history.** `extrude_tracked` has no target to measure, so it refuses through-all just as `extrude` does: `a through-all extent needs a target body to measure; use extrude_feature with the cut mode`.
- **A face number from another shape.** The numbers in an extrude history belong to that solid. After a cut, face 6 may be another face, or not exist: `sub_shape_bounding_box` then gives the wrong face's box, or `face index 7 is out of range: the shape has 7 faces` (Section 6.3). Take the numbers again from the step that made the shape you ask about.
- **A through-all cut the wrong way.** It misses the part: `the through-all cut misses the target: it removes nothing`. On a top face, use `ThroughDirection::Reversed`, or `Both` (Section 7.4).
- **A face that isn't parallel to the workplane.** A to-face extrude can only end on a face that faces the same way as the outline, or the opposite way: `target face 11 (normal …) is not parallel to the workplane (normal …)`. Pick a face whose normal matches the workplane's, or its reverse (Section 7.6).
- **A round face.** A to-face extrude can't end on a curved face: `face 2 is not planar`. Use a flat face.
- **A face in the workplane.** If the face lies on the same plane as the outline, there is nothing to extrude: `face 5 lies in the workplane: the to-face extrude has zero length`. The same goes for two faces on one plane: `faces 7 and 9 are coplanar: the between-faces extrude has zero length`.

CHAPTER 8

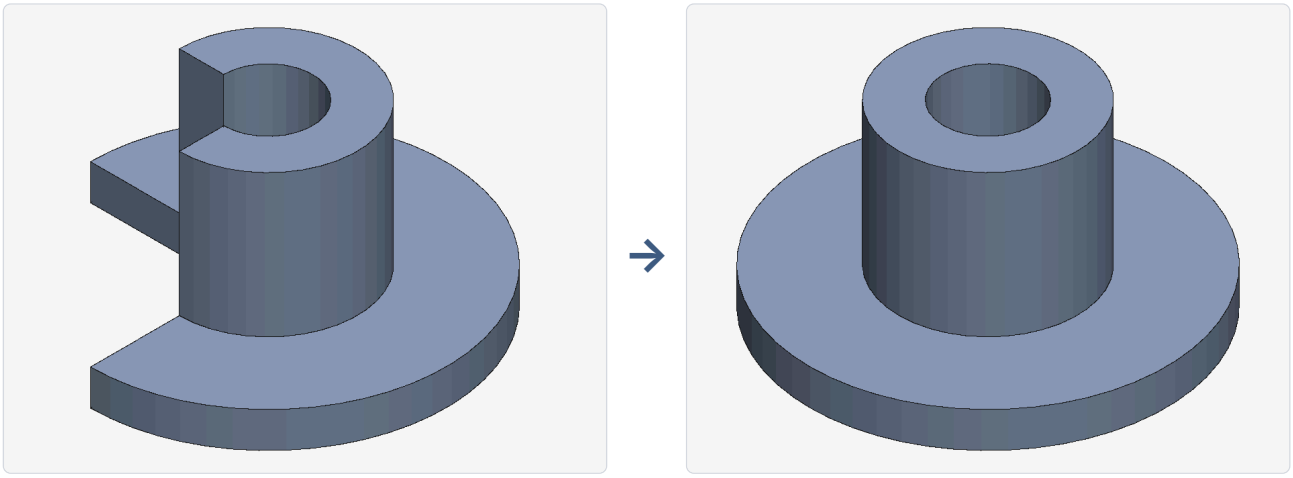
# Revolving a profile

> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

A *revolve* turns a flat shape around a line, the way a potter's wheel turns a lump of clay into a pot. Every point of the shape sweeps a circle around that line, so the result is round: a shaft, a knob, a washer, a bottle.

Where an extrude (Chapter 7) pushes an outline straight out, a revolve swings it around. You draw only half of the part's cross-section, the half on one side of the center line, and the revolve makes the rest.

The line the shape turns around is the *axis of revolution*. The *revolve angle* says how far it turns: a full turn makes a closed, round part; less than a full turn makes a wedge, like a slice of a cake.



Three quarters of a turn A full turn

**Figure 8.1** A flanged bushing made by one revolve. Stopped at three quarters of a turn (left), the open quarter shows the outline that was turned: an L-shape beside a hollow center. A full turn (right) closes the part.

**In Saulitwerx today:** the geometry engine can revolve a flat face from `make_face` about any axis, by any angle up to a full turn. Like an extrude, the result can be a new solid, or be joined to a part, cut from it or intersected with it in the same step. Before it turns anything, it checks that the axis lies in the outline's plane and stays clear of the outline, and says plainly when it doesn't (Section 8.3). You type the outline's pieces into a small program; a sketch's regions can't be handed to the revolve yet, and the revolve isn't recorded in a part's history.

## 8.1 Revolve a profile

| | |
|---|---|
| **YOU WILL** | Make a flanged bushing from half its cross-section, with a full turn and with three quarters of a turn. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 10 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `revolve`:

   TERMINAL
   ```
   cargo run -p swx-kernel --features occt --example revolve_bushing -- revolve
   ```

   It prints a line for each part, then five errors on purpose, then the files it wrote. The two longest error lines are broken here to fit the page; on your screen each is one line:

   OUTPUT
   ```
   full turn   8482.30 mm³   6 faces  x from -20 to 20
   270°        6361.73 mm³   8 faces  x from -20 to 20
   grooved     8362.92 mm³  10 faces  x from -20 to 20
   360 -> error: invalid input: revolve angle must be in (0, 2π] radians, got 360
   zero axis -> error: invalid input: revolve axis direction must be non-zero
   cut, no target -> error: invalid input: revolve mode "cut" needs a target body
   axis through the outline -> error: invalid input: the profile crosses
     the revolve axis (origin [12.0, 0.0, 0.0], direction [0.0, 0.0, 1.0]):
     it reaches 7 on one side and 8 on the other
   axis out of the plane -> error: invalid input: revolve axis (origin
     [0.0, 0.0, 0.0], direction [0.0, 1.0, 0.0]) is not in the profile's
     plane (through [0.0, 0.0, 0.0], normal [0.0, -1.0, 0.0])
   …  (the geometry engine's file reports)
   wrote revolve/bushing.step
   wrote revolve/bushing-270.step
   wrote revolve/bushing-grooved.step
   ```

2. Read the `full turn` line. The outline reaches 20 mm from the axis, so the part runs from x = −20 to 20 mm: 40 mm across. The six faces are the bore, the tube's outside, the flange's rim, and the top, the flange's top and the bottom.

3. Read the `270°` line. Three quarters of a turn gives three quarters of the volume, 6361.73 mm³. The two flat ends of the wedge add two faces.

4. Draw the three-quarter part:

   TERMINAL
   ```
   target/release/swx-qa render --step revolve/bushing-270.step -o bushing-270.png
   ```

   Open `bushing-270.png`. You should see the left picture of Figure 8.1: the open quarter shows the L-shaped outline that was turned. Draw `revolve/bushing.step` the same way to see the full part.

**RESULT** A round part made from one flat outline. The bushing's bore, tube and flange came from a single six-sided shape.

### How the sample is built

The program is `crates/swx-kernel/examples/revolve_bushing.rs`. It does three things.

1. **Draw half the cross-section.** The outline sits on the XZ workplane, so left–right is X, the distance from the axis, and up–down is Z, the height. It starts 5 mm from the axis, which leaves a 10 mm bore down the middle:

```
let outline = polygon(&[
    [5.0, 0.0], [20.0, 0.0], [20.0, 4.0],
    [10.0, 4.0], [10.0, 20.0], [5.0, 20.0],
]);
let section = k.make_face(&Workplane::XZ, &outline, &[])?;
```

`polygon` is a small helper in the sample that joins the points with lines, the last back to the first.

2. **Give the axis.** An `Axis` is a point on the line and the line's direction. Here it is the Z axis, through the origin and pointing up:

```
let z_axis = Axis { origin: [0.0, 0.0, 0.0], dir: [0.0, 0.0, 1.0] };
```

3. **Revolve it.** `revolve_feature` takes the face, the axis, the angle in radians, the mode and the target. `TAU` is a full turn, 2π; `1.5 * PI` is three quarters of one:

```
let bushing = revolve_feature(&k, &section, z_axis, TAU,
    ExtrudeMode::NewSolid, None)?;
let cutaway = revolve_feature(&k, &section, z_axis, 1.5 * PI,
    ExtrudeMode::NewSolid, None)?;
```

> **NOTE** **How close is close enough?**
>
> The checks allow 0.0000001 mm ( `REVOLVE_AXIS_TOLERANCE` ), the same small gap the geometry engine treats as zero. An axis that misses the plane by less than that still counts as in it, and an outline that pokes past the axis by less than that still counts as touching it.

> **TIP** **Which way does it turn?**
>
> Point your right thumb along the axis direction: your fingers curl the way a positive angle turns. Here the axis points up, so the outline swings from +X towards +Y, counterclockwise seen from above. A full turn looks the same either way.

> **CAUTION** **Radians, not degrees**
>
> The geometry engine takes the angle in radians: a full turn is 2π, about 6.283, and 360 is refused. To work in degrees, multiply by π/180, or write `360.0_f64.to_radians()` .

## 8.2 Cut or join with a revolve

A revolve uses the same four modes as an extrude (Section 7.3): new solid, join, cut and intersect, with a target body for the last three. A revolve cut is how you make a round groove, such as the seat for a sealing ring around a shaft (Figure 8.2).

| | |
|---|---|
| **YOU WILL** | Cut a groove around the bushing with a second revolve. |
| **YOU NEED** | The sample run of Section 8.1. |
| **TIME** | 5 minutes. |

1. Read the `grooved` line of the output in Section 8.1. The groove took 119.38 mm³ from the bushing, and its floor and two walls added three faces, bringing the bushing's six faces to nine, plus one more where the groove splits the tube's outside in two: ten.
2. Draw the grooved part:

```
TERMINAL
target/release/swx-qa render --step revolve/bushing-grooved.step -o bushing-grooved.png
```

   Open `bushing-grooved.png`. You should see Figure 8.2: a narrow groove around the tube, halfway up.

**RESULT** A groove around a round part, made by turning a small rectangle around the same axis as the part itself.

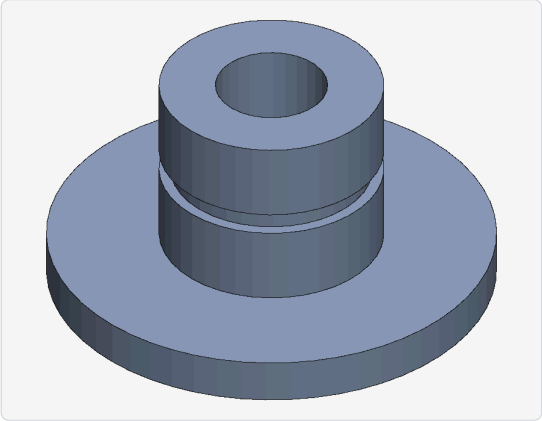


**Figure 8.2** The bushing with a groove around its tube: a small rectangle, revolved a full turn and cut from the part.

The groove's outline is a 2 × 2 mm square that straddles the tube's outer wall, at x = 10 mm. Only the half inside the tube removes material; the other half turns through empty space. Letting a cutting outline overlap the edge like this makes sure the groove opens cleanly onto the surface:

```
let groove = k.make_face(&Workplane::XZ,
    &polygon(&[[9.0, 11.0], [11.0, 11.0], [11.0, 13.0], [9.0, 13.0]]), &[])?;
let grooved = revolve_feature(&k, &groove, z_axis, TAU,
    ExtrudeMode::Cut, Some(&bushing))?;
```

The rules are the extrude's: join, cut and intersect need a target, a result with nothing left is an `empty result` error, and the target itself is never changed. `ExtrudeMode` is the name of the mode type for both features.

## 8.3 Common mistakes

- **Degrees instead of radians.** `revolve angle must be in (0, 2π] radians, got 360`. Convert the angle (Section 8.1).
- **An angle of zero, or a negative one.** A revolve has to turn: the same message. To turn the other way, reverse the axis direction.
- **An axis with no direction.** `revolve axis direction must be non-zero`. Give the axis a direction, such as `[0.0, 0.0, 1.0]`.
- **The axis runs through the outline.** The outline would sweep through itself, so the revolve is refused: `the profile crosses the revolve axis (…): it reaches 7 on one side and 8 on the other`. The two numbers say how far, in millimeters, the outline reaches on each side; the sample's flange runs from 5 to 20 mm, and an axis at 12 mm splits it 7 and 8. Move the axis, or the outline, so the whole outline is on one side. Touching the axis is fine, and makes a solid center with no bore.

- **The axis isn’t in the outline’s plane.** An axis that is tilted out of the plane, stands straight out of it, or runs beside it is refused: `revolve axis (…) is not in the profile's plane (through …, normal …)`. The message gives a point on the plane and its normal, the direction straight out of it; the axis direction must be at right angles to that normal, and the axis point must lie on the plane. Draw the outline on a plane that contains the axis, as the sample does with the XZ plane and the Z axis.
- **A shape that isn’t one flat face.** `revolve needs a single planar face`. Give the revolve a face from `make_face`, not a solid.
- **Join, cut or intersect with no target.** `revolve mode "cut" needs a target body`. Pass the part as `Some(&part)`, or use `ExtrudeMode::NewSolid`.

CHAPTER 9

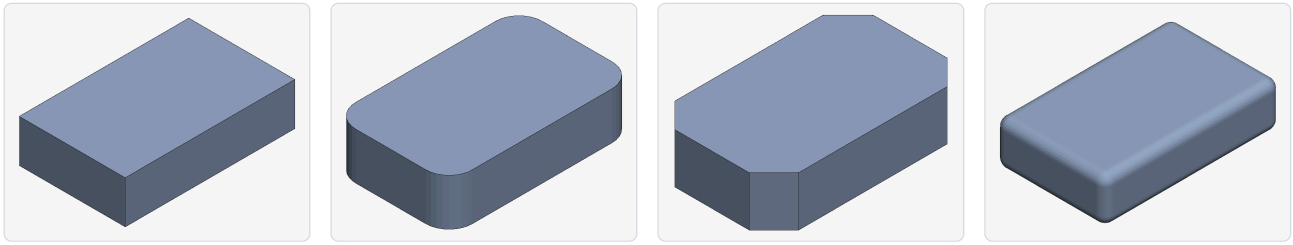
# Rounding and bevelling edges

> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

Sharp edges are rare on real parts. They cut hands, chip easily, and crack under load. So designers soften them, in one of two ways:

- A *fillet* rounds an edge into a smooth curve, like the corners of a phone. Its size is a *radius*: the radius of the curve that replaces the edge.
- A *chamfer* cuts an edge off at an angle, leaving a narrow flat face, like the bevel on the edge of a tabletop. Its size is a *distance*: how far the cut reaches back along each of the two faces that meet at the edge. It can also reach back further on one face than the other (Section 9.2).

Both work on edges of a solid you already have. You pick the edges, give one size, and get back a new solid with those edges softened (Figure 9.1).



Block: 40 × 25 × 10 mm | Fillet 5 mm, 4 edges | Chamfer 5 mm, 4 edges | Fillet 2 mm, all edges

**Figure 9.1** One block, three ways to soften its edges. The fillet and the chamfer take the same four upright corner edges and the same size, 5 mm. The last picture rounds every edge at once. Each picture is scaled to fill its frame.

**In Saulitwerx today:** the geometry engine can fillet edges with one radius, and chamfer them with one distance, the same on both faces. A fillet of one edge can also change its radius from one end to the other (Section 9.3), and a chamfer can take two different distances, or a distance and an angle (Section 9.2). You give the edges, and for an uneven chamfer the face to measure from, by their numbers in the solid (Section 6.3), so your program finds them first. A fillet or a chamfer isn't recorded in a part's history yet.

## 9.1 Fillet and chamfer edges

| | |
|---|---|
| **YOU WILL** | Round the four corners of a block, bevel them instead, and round every edge at once. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `edges`:

   TERMINAL
   ```
   cargo run -p swx-kernel --features occt --example fillet_chamfer -- edges
   ```

   It prints the block, the edges it picked, one line for each result, then four errors on purpose, then the files it wrote:

   OUTPUT
   ```
   block              10000.00 mm³   6 faces  12 edges
   upright edges: [0, 2, 4, 6]
   fillet r 5          9785.40 mm³  10 faces  24 edges
   chamfer 5           9500.00 mm³  10 faces  24 edges
   fillet all, r 2     9753.19 mm³  26 faces  56 edges
   r 15 -> error: kernel failure: fillet failed
   no edges -> error: invalid input: the edge list is empty
   a face -> error: invalid input: expected an edge, got face 0
   r 0 -> error: invalid input: fillet radius must be finite and > 0, got 0
   …  (the geometry engine's file reports)
   wrote edges/edges-fillet.step
   wrote edges/edges-chamfer.step
   wrote edges/edges-all.step
   ```

2. Read the second line. Of the block's 12 edges, numbers 0, 2, 4 and 6 are the four that stand upright at its corners.
3. Read the `fillet r 5` line. Each corner lost a 5 × 5 mm square less a quarter circle, 5.37 mm², over the block's 10 mm height. Four corners make 214.60 mm³. Each rounded corner is a new face, so the block now has 10.
4. Read the `chamfer 5` line. Each corner lost a triangle with two 5 mm sides, 12.5 mm², so four corners take 500 mm³. The new faces are flat.
5. Read the `r 15` line. A 15 mm fillet doesn't fit: on the block's 25 mm side, the two rounded corners would need 15 + 15 = 30 mm. The geometry engine can't build it and says so. The block is unchanged.
6. Draw the filleted block:

   TERMINAL
   ```
   target/release/swx-qa render --step edges/edges-fillet.step -o edges-fillet.png
   ```

   Open `edges-fillet.png`. You should see the block of Figure 9.1 with its four corners rounded. Draw the other files the same way.

**RESULT** Three softened blocks, each made by one call. The block itself is left as it was.

### How the sample is built

The program is `crates/swx-kernel/examples/fillet_chamfer.rs`. It does two things.

1. **Find the edges.** An edge is a `SubShape` of kind `Edge`, numbered from 0. To find the upright ones, the sample asks for each edge's bounding box (Section 7.5). An upright edge's box has no width and no depth, only height:

   ```
   let bb = k.sub_shape_bounding_box(&block, edge)?;
   if bb.max[0] - bb.min[0] < 1e-6 && bb.max[1] - bb.min[1] < 1e-6 {
       corners.push(edge);
   }
   ```

2 **Fillet or chamfer them.** `fillet` takes the solid, a list of edges and a radius; `chamfer` takes the same with a distance. Every edge in the list gets the same size:

```
let rounded = k.fillet(&block, &corners, 5.0)?;
let bevelled = k.chamfer(&block, &corners, 5.0)?;
let soft = k.fillet(&block, &all, 2.0)?;
```

**TIP Round many edges in one call**

Give all the edges that should be rounded in one list, as the sample does for `all`. Where the rounded edges meet at a corner, the geometry engine blends them into one smooth corner. Listing an edge twice is harmless; it counts once.

**CAUTION The numbers change**

A fillet or a chamfer makes new faces and edges, so the result is numbered afresh: the block had 12 edges, the filleted one has 24. Edge 0 of the result may be a different edge. To soften more edges later, find them again in the new solid.

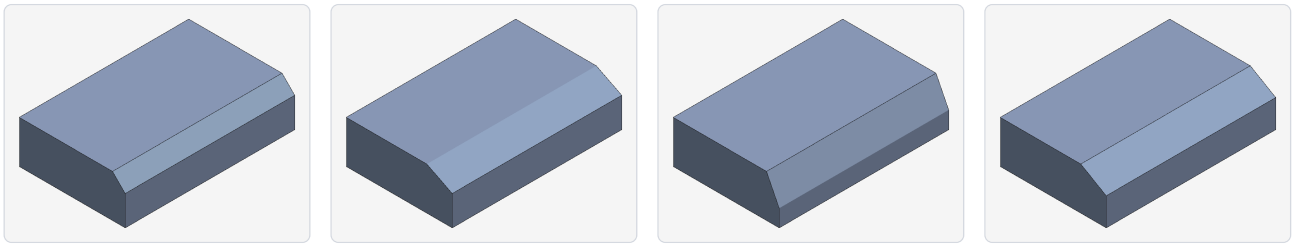
## 9.2 Chamfer with two sizes

An even chamfer cuts back the same distance on both faces, at 45° where they meet square. Sometimes you want it uneven: a long, shallow lead-in on the top of a part so a mating part slides on, or a bevel at a set angle to match a tool. Then one number isn't enough, and the chamfer needs to know which face each number belongs to. That face is the *reference face*: one of the two faces that meet at the edge.

You can size an uneven chamfer in two ways:

- **Two distances.** The first is measured on the reference face, the second on the other face.
- **A distance and an angle.** The distance is measured on the reference face. The angle is between the reference face and the new sloping face, so a smaller angle makes a flatter bevel. Where the faces meet square, the setback on the other face is the distance times the tangent of the angle: 6 mm at 30° reaches 3.46 mm down the side.

Swap the reference face and the same two numbers give a different part (Figure 9.2).



Even, 3 mm | 6 and 3 mm, top first | 6 and 3 mm, side first | 6 mm at 30°, top first

**Figure 9.2** Four chamfers on the same edge of the 40 × 25 × 10 mm block: where the top meets the long side nearest you. The middle two use the same two distances, 6 and 3 mm, measured from different reference faces.

| | |
|---|---|
| YOU WILL | Bevel one edge of a block evenly, with two distances from each face in turn, and with a distance and an angle. |
| YOU NEED | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| TIME | 5 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `edges`:

   TERMINAL

   ```
   cargo run -p swx-kernel --features occt --example chamfer_sizes -- edges
   ```

   It prints the block, the faces and the edge it found, one line for each chamfer, four errors on purpose, then the files it wrote:

   OUTPUT

   ```
   block                  10000.00 mm³
   top face 5, side face 3, edge 11
   equal 3                 9820.00 mm³  slope y 22..25  z 7..10
   6 and 3, top first      9640.00 mm³  slope y 19..25  z 7..10
   6 and 3, side first     9640.00 mm³  slope y 22..25  z 4..10
   6 at 30°, top first     9584.31 mm³  slope y 19..25  z 6.54..10
   6 and 30 -> error: kernel failure: chamfer failed
   6 at 90° -> error: invalid input: chamfer angle must be finite and
       strictly between 0 and π/2, got 1.5707963267948966
   bottom face -> error: invalid input: face 4 does not bound edge 11
   both faces -> error: invalid input: edge 11 is listed with two
       different faces (3 and 5)
   …  (the geometry engine's file reports)
   wrote edges/chamfer-equal.step
   wrote edges/chamfer-top.step
   wrote edges/chamfer-side.step
   wrote edges/chamfer-angle.step
   ```

   Two long error lines are wrapped here to fit the page.

2. Read the second line. The top of the block is face 5, the long side at y = 25 is face 3, and the edge where they meet is edge 11.

3. Read the `slope` part of each chamfer line. It is where the new sloping face lies: how far it reaches across the top (in y, from the edge at 25) and down the side (in z, from the top at 10). The even chamfer reaches 3 mm each way.

4. Compare the two `6 and 3` lines. With the top as reference face, the slope reaches 6 mm across the top (y 19..25) and 3 mm down the side (z 7..10). With the side as reference face, it is the other way round. Both take away the same 360 mm³: a triangle of 6 × 3 ÷ 2 = 9 mm² along the 40 mm edge.

5. Read the `6 at 30°` line. The slope reaches 6 mm across the top and 3.46 mm down the side, to z = 6.54.

6. Draw the chamfers:

   TERMINAL

   ```
   target/release/swx-qa render --step edges/chamfer-top.step -o chamfer-top.png
   ```

   Open `chamfer-top.png`. You should see the second block of Figure 9.2. Draw the other files the same way.

**RESULT** One edge bevelled four ways. The block itself is left as it was.

### How the sample is built

The program is `crates/swx-kernel/examples/chamfer_sizes.rs`. It does three things.

1. **Find the faces and the edge.** The sample finds the top and the side by their bounding boxes, the way Section 9.1 found the upright edges. The edge it wants is the one both faces list as theirs (Section 6.3):

```
let top_edges = k.adjacent(&block, top, TopoKind::Edge)?;
let edge = k.adjacent(&block, side, TopoKind::Edge)?
    .into_iter()
    .find(|e| top_edges.contains(e))
    .expect("shared edge");
```

2. **Pair the edge with its reference face.** An `EdgeOnFace` holds an edge and the face to measure from. The face must be one of the two that meet at the edge:

```
let on_top = EdgeOnFace { edge, face: top };
let on_side = EdgeOnFace { edge, face: side };
```

3. **Give the size, and chamfer.** `ChamferSize::TwoDistances` takes `d1` (on the reference face) and `d2` (on the other face). `ChamferSize::DistanceAngle` takes a `distance` and an `angle` in radians, so convert degrees with `to_radians()`:

```
let two = ChamferSize::TwoDistances { d1: 6.0, d2: 3.0 };
let angled = ChamferSize::DistanceAngle {
    distance: 6.0,
    angle: 30f64.to_radians(),
};
let from_top = k.chamfer_asymmetric(&block, &[on_top], two)?;
let from_side = k.chamfer_asymmetric(&block, &[on_side], two)?;
```

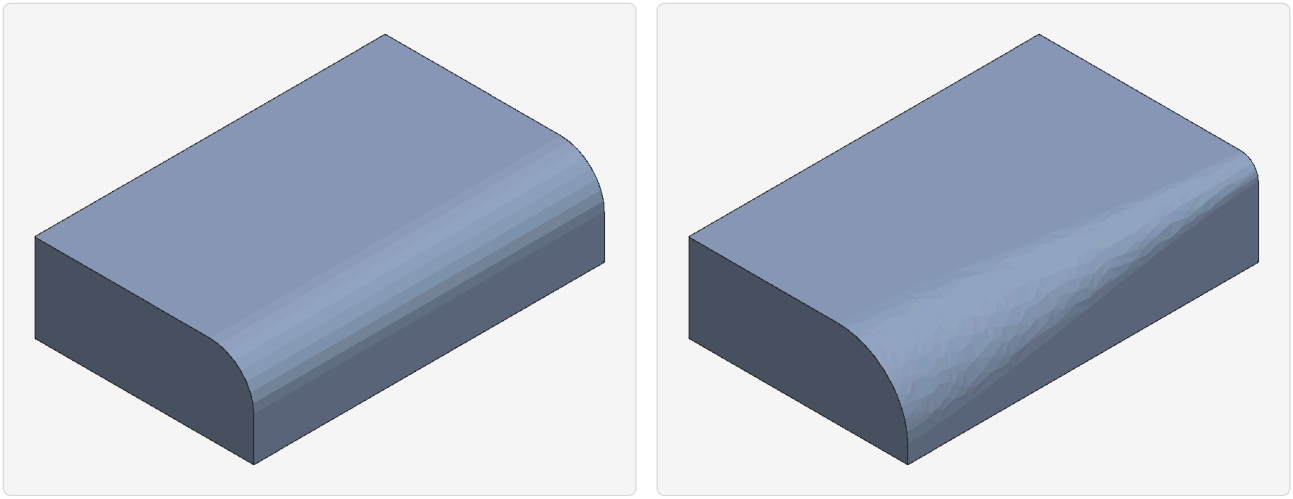
> **TIP** **Many edges, one size**
>
> `chamfer_asymmetric` takes a list, like `chamfer`. Every edge in it gets the same size, each measured from its own reference face. Listing the same edge and face twice is harmless; it counts once.

> **CAUTION** **Angles are in radians**
>
> The angle must be strictly between 0 and $\pi/2$ (90°). Typing `30.0` means 30 radians and is refused; write `30f64.to_radians()`, or 0.5236.

## 9.3 Round with a changing radius

A constant fillet is the same size all along its edge. Sometimes the rounding should grow instead: a handle that is gentle at one end and generous where the hand rests, or a blend that must be small where it meets a narrow part and large where there is room. A *variable fillet* does this. You give a radius for each end of the edge, and the radius changes evenly from one to the other along the way (Figure 9.3).

The geometry engine needs to know which end is which. An edge runs between two *vertices*, its end points. You name one of them as the *start*: the first radius belongs there, and the second radius to the other end.



Before: constant 5 mm After: 2 mm to 8 mm

**Figure 9.3** The same edge of the 40 × 25 × 10 mm block, rounded two ways: with one 5 mm radius, and with a radius that grows from 2 mm at the far end (x = 0) to 8 mm at the near end (x = 40). The faint ripples on the curved face come from the picture tool's triangles, not from the part.

| | |
|---|---|
| **YOU WILL** | Round one edge of a block from 2 mm at one end to 8 mm at the other, and compare it with a constant fillet. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `edges`:

TERMINAL

```
cargo run -p swx-kernel --features occt --example fillet_variable -- edges
```

It prints the block, the edge and its two ends, one line for each fillet, four errors on purpose, then the files it wrote:

OUTPUT

```
block                  10000.00 mm³   6 faces
edge 11: vertex 2 at x = 0, vertex 6 at x = 40
constant r 5            9785.40 mm³   7 faces
2 at x = 0, 8 at 40     9753.28 mm³   7 faces
2 at x = 40, 8 at 0     9753.28 mm³   7 faces
2 to 12 -> error: kernel failure: fillet failed
wrong vertex -> error: invalid input: vertex 1 is not on edge 11
a face -> error: invalid input: a variable fillet needs an edge and a
    vertex, got Face 0 and Vertex 2
0 to 8 -> error: invalid input: fillet start radius must be finite and
    > 0, got 0
…  (the geometry engine's file reports)
wrote edges/fillet-constant.step
wrote edges/fillet-variable.step
```

Two long error lines are wrapped here to fit the page.

2. Read the second line. The edge is number 11: the top edge of the long side at y = 25, running from vertex 2 at x = 0 to vertex 6 at x = 40.
3. Read the `constant r 5` line. A 5 mm fillet along the 40 mm edge takes away 214.60 mm³, as in Section 9.1, and adds one curved face.

4. Read the two variable lines. Starting at vertex 2 puts 2 mm at x = 0 and 8 mm at x = 40; starting at vertex 6 swaps the ends. Both take away the same 246.72 mm³. That is a little more than the constant 5 mm fillet, although 5 mm is the average radius: the material a rounding takes away grows with the square of its radius, so the large end takes more than the small end saves.
5. Read the `2 to 12` line. The side of the block is only 10 mm tall, so a 12 mm rounding can't fit on it. The geometry engine says so, and the block is unchanged.
6. Draw the variable fillet:

   TERMINAL
   ```
   target/release/swx-qa render --step edges/fillet-variable.step -o fillet-variable.png
   ```

   Open `fillet-variable.png`. You should see the right-hand block of Figure 9.3, with the 8 mm end (x = 40) nearest you.

**RESULT** One edge rounded with a radius that changes along it. The block itself is left as it was.

### How the sample is built

The program is `crates/swx-kernel/examples/fillet_variable.rs`. It does two things.

1. **Find the edge and its ends.** The sample finds the edge by its bounding box, as Section 9.1 does. `adjacent` lists the edge's two vertices (Section 6.3), and the sample tells them apart by where they are:

   ```rust
   for v in k.adjacent(s, edge, TopoKind::Vertex)? {
       if (k.sub_shape_bounding_box(s, v)?.min[0] - x).abs() < 1e-6 {
           return Ok(v);
       }
   }
   ```

2. **Round it.** `fillet_variable` takes the solid, one edge, the start vertex, the radius at the start and the radius at the other end:

   ```rust
   let growing = k.fillet_variable(&block, edge, left, 2.0, 8.0)?;
   ```

> **TIP** **Equal radii**
>
> Give the same radius at both ends and you get the constant fillet of Section 9.1. A variable fillet rounds one edge per call; to round several edges, call it once for each, finding the next edge again in the new solid.

> **CAUTION** **Edges that flow on**
>
> If the edge runs smoothly into the next one, as the edges round a filleted corner do, the geometry engine would have to round the whole smooth chain, so it refuses. Use a variable fillet on edges with a sharp corner at each end.

## 9.4 Common mistakes

- **A size too large for the part.** The rounding or the bevel has to fit on both faces next to each edge, and must not run into the next one: `kernel failure: fillet failed` (or `chamfer failed`). Make it smaller than the narrowest face it touches, and less than half of it where two softened edges share that face.
- **An empty list.** `invalid input: the edge list is empty`. Give at least one edge.
- **A face or a vertex in the list.** Only edges can be rounded: `invalid input: expected an edge, got face 0`. Ask for edges with `TopoKind::Edge`, or turn a face into its edges with `adjacent` (Section 6.3).

- **An edge number from another solid.**
  `invalid input: edge index 12 is out of range: the shape has 12 edges`, or worse, the wrong edge. Find the edges in the solid you are softening.
- **A size of zero or less.** `invalid input: fillet radius must be finite and > 0, got 0` (or `fillet start radius`, `fillet end radius`, `chamfer distance`, `chamfer d1`, `chamfer d2`).
- **An uneven chamfer that doesn't fit.** Each setback must fit on its own face:
  `kernel failure: chamfer failed`. The sample's 30 mm on the 10 mm side is far too much.
- **An angle of 90° or more, or in degrees.**
  `invalid input: chamfer angle must be finite and strictly between 0 and π/2, got 1.5707963267948966`. Give radians, between 0 and π/2.
- **A reference face that doesn't touch the edge.** `invalid input: face 4 does not bound edge 11`. Use one of the two faces that `adjacent` lists for the edge.
- **The same edge with both faces.** `invalid input: edge 11 is listed with two different faces (3 and 5)`. Each edge gets one reference face; pick the one to measure the first distance on.
- **A variable fillet's start that isn't on its edge.** `invalid input: vertex 1 is not on edge 11`. Ask `adjacent` for the edge's two vertices and pick one (Section 9.3).
- **A face or a vertex where the edge goes.**
  `invalid input: a variable fillet needs an edge and a vertex, got Face 0 and Vertex 2`. The first is the edge, the second its start vertex.
- **A variable fillet too large at one end.** The larger radius must fit as a constant fillet would:
  `kernel failure: fillet failed`.

CHAPTER 10

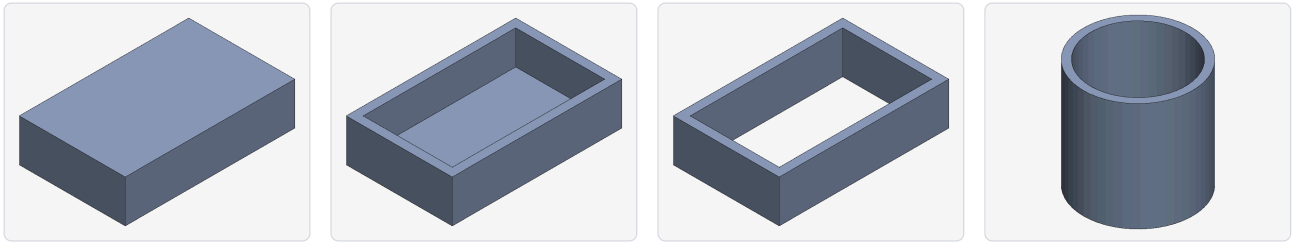
# Hollowing a part

> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

Many parts are hollow: a box for electronics, a tray, a cup, a cover. Making them solid would waste material, add weight, and in a molded or printed part cause sink marks and warping. You could model the inside as a second, smaller shape and cut it away, but there is a quicker way.

A *shell* hollows a solid in one step. It leaves walls of one thickness all round and takes away everything inside. The faces you choose to remove become the openings; they are the *open faces*. A box with its top as the open face becomes a tray. Open the top and the bottom, and it becomes a frame. Open the top of a cylinder, and it becomes a cup (Figure 10.1).

The walls grow *inward* from the faces of the solid. The outside of the part stays where it was, at the size you drew, and only the inside changes. That is usually what a design needs: the outside fits a space, and the walls are as thick as the material requires.



Block: 40 × 25 × 10 mm | Top open: a tray | Top and bottom open: a frame | Cylinder, top open: a cup

**Figure 10.1** Shells with 2 mm walls. The tray and the frame are made from the same block, opened at one face and at two. The cup is a cylinder opened at its top. Each part keeps its outside size. Each picture is scaled to fill its frame.

**In Saulitwerx today:** the geometry engine can shell a solid with one wall thickness, open at one or more faces. You give the open faces by their numbers in the solid (Section 6.3), so your program finds them first. Every wall has the same thickness, and a shell must have at least one opening. A shell isn't recorded in a part's history yet.

## 10.1 Shell a solid

| | |
|---|---|
| **YOU WILL** | Hollow a block into a tray and a frame, and a cylinder into a cup, all with 2 mm walls. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `shells`:

   TERMINAL

   ```
   cargo run -p swx-kernel --features occt --example shell_parts -- shells
   ```

   It prints the block and the faces it found, one line for each solid, then five errors on purpose, then the files it wrote:

   OUTPUT

   ```
   block            10000.00 mm³   6 faces  outside 40 × 25 × 10 mm
   top face 5, bottom face 4
   tray, wall 2      3952.00 mm³  11 faces  outside 40 × 25 × 10 mm
   frame, wall 2     2440.00 mm³  10 faces  outside 40 × 25 × 10 mm
   cylinder         21205.75 mm³   3 faces  outside 30 × 30 × 30 mm
   cup, wall 2       6339.73 mm³   5 faces  outside 30 × 30 × 30 mm
   wall 13 -> error: kernel failure: shell failed
   no faces -> error: invalid input: the face list is empty
   an edge -> error: invalid input: expected a face, got edge 0
   face 6 -> error: invalid input: face index 6 is out of range: the shape has 6 faces
   wall 0 -> error: invalid input: shell thickness must be finite and > 0, got 0
   …  (the geometry engine's file reports)
   wrote shells/shell-tray.step
   wrote shells/shell-frame.step
   wrote shells/shell-cup.step
   ```

2. Read the second line. The block's top is face 5 and its bottom is face 4.

3. Read the `tray` line. The outside is still 40 × 25 × 10 mm. The 2 mm walls leave a hollow 36 × 21 mm across and 8 mm deep, which is 6048 mm³ taken away. The tray has 11 faces: five outside, five inside, and the rim on top.

4. Read the `frame` line. With the bottom open too, the hollow goes all the way through, 36 × 21 × 10 mm, and only the four walls are left.

5. Read the `cup` line. The cylinder is 30 mm across and 30 mm tall. Its walls and floor are 2 mm thick, so the hollow is 26 mm across and 28 mm deep. The cup has 5 faces: the outside, the bottom, the inside wall, the floor and the rim.

6. Read the `wall 13` line. The block is 25 mm wide, and two walls of 13 mm would need 26 mm. The geometry engine can't build it and says so. The block is unchanged.

7. Draw the tray:

   TERMINAL

   ```
   target/release/swx-qa render --step shells/shell-tray.step -o shell-tray.png
   ```

   Open `shell-tray.png`. You should see the tray of Figure 10.1. Draw the other files the same way.

**RESULT** A tray, a frame and a cup, each made by one call. The solids you started from are left as they were.

### How the sample is built

The program is `crates/swx-kernel/examples/shell_parts.rs`. It does two things.

1. **Find the faces to open.** A face is a `SubShape` of kind `Face`, numbered from 0. The sample looks for the flat face whose outward normal points the way it wants, up for the top and down for the bottom. `face_plane` gives each flat face's normal (Section 7.6), and refuses curved faces, which the sample skips:

```
for index in 0..k.sub_shape_count(s, TopoKind::Face)? {
    let face = SubShape { kind: TopoKind::Face, index };
    if let Ok((_, n)) = k.face_plane(s, face) {
        if dot(n, normal) > 0.999_999 {
            return Ok(face);
        }
    }
}
```

2. **Shell the solid.** `shell` takes the solid, a list of open faces and the wall thickness in mm:

```
let tray = k.shell(&block, &[top], 2.0)?;
let frame = k.shell(&block, &[top, bottom], 2.0)?;
let cup = k.shell(&can, &[rim], 2.0)?;
```

> **TIP** **Choosing a wall thickness**
>
> Pick the thickness your material and process need, then check that the part still has room for it: every wall comes off the inside, so a part 25 mm wide with 2 mm walls is 21 mm wide inside. Listing a face twice is harmless; it counts once.

> **CAUTION** **The numbers change**
>
> A shell makes new faces and edges, so the result is numbered afresh: the block had 6 faces, the tray has 11. To round the tray's rim or open another face later, find the faces again in the new solid.

## 10.2 Common mistakes

- **A wall too thick for the part.** The walls from opposite sides must not meet: `kernel failure: shell failed`. Keep the thickness under half the part's narrowest width.
- **No open face.** `invalid input: the face list is empty`. Give at least one face to open. A part hollow inside with no opening can't be made yet.
- **An edge or a vertex in the list.** Only faces can be opened: `invalid input: expected a face, got edge 0`. Ask for faces with `TopoKind::Face`, or turn an edge into its faces with `adjacent` (Section 6.3).
- **A face number from another solid.** `invalid input: face index 6 is out of range: the shape has 6 faces`, or worse, the wrong face opened. Find the faces in the solid you are shelling.
- **A thickness of zero or less.** `invalid input: shell thickness must be finite and > 0, got 0`.

CHAPTER 11

# Making holes

> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

Almost every part has holes: for screws and bolts, for pins that line parts up, for shafts and wires to pass through. You could make each one by sketching a circle and cutting it (Section 7.4), but a hole is so common that it has its own feature. You say where it starts, which way it goes, how wide and how deep it is, and the geometry engine drills it.

A *hole* has three things to choose:

- **Where and which way.** A start point on the face of the part, and a direction pointing into the material, the way a drill would go. The direction can be any way at all, so a hole can go into the top, the side or a sloping face.
- **Its shape.** A *simple hole* is a plain round hole of one diameter. A *counterbore* adds a wider, flat-bottomed recess at the mouth of the hole, so that a screw head sits flush with the surface or below it. A *countersink* adds a cone-shaped recess instead, for a screw with a cone-shaped (flat) head. A *tapped hole* is drilled at the right size for a screw thread (Section 11.2).
- **How deep.** A *blind hole* stops at a depth you give, with a flat bottom. A *through hole* goes all the way through the part.

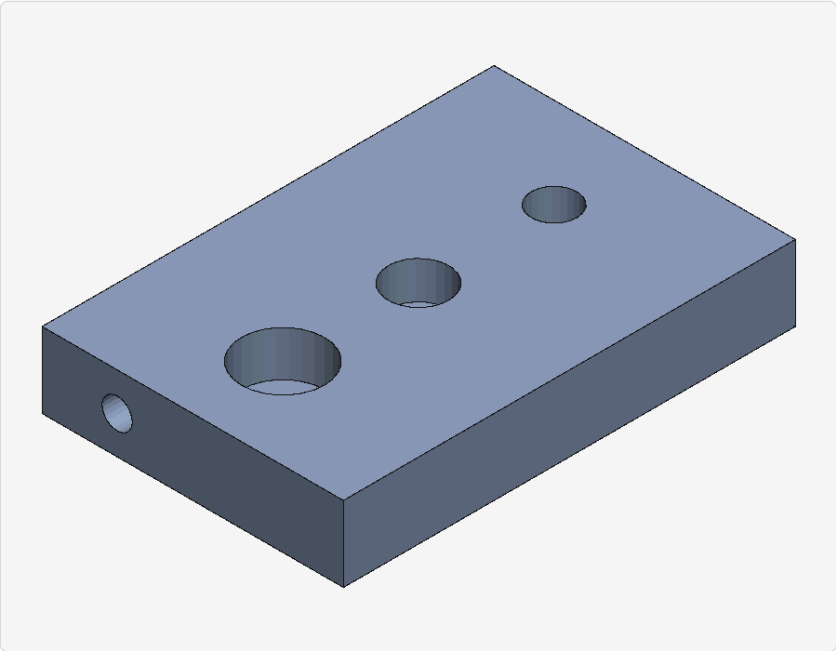


**Figure 11.1** A 60 × 40 × 10 mm plate with four holes. In its top, from the back: a 6 mm simple hole through the plate, an 8 mm blind hole 5 mm deep, and a 6 mm through hole with an 11 mm counterbore 6 mm deep. In the end nearest you, a 4 mm blind hole 20 mm deep, drilled sideways.

**In Saulitwerx today:** the geometry engine can drill simple, counterbored, countersunk and tapped holes, blind or through all, in any direction. Holes have flat bottoms, with no drill points. A tapped hole is drilled at its tap-drill size, but the thread itself isn't modeled. You give the start point and the direction as numbers, so your program works them out first. A hole isn't recorded in a part's history yet.

## 11.1 Drill holes in a plate

| | |
|---|---|
| **YOU WILL** | Drill a through hole, a blind hole, a counterbored hole and a sideways hole in a plate. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its part in a folder called `holes`:

   TERMINAL
   ```
   cargo run -p swx-kernel --features occt --example hole_plate -- holes
   ```

   It prints the plate, one line after each hole, four errors on purpose, then the file it wrote:

   OUTPUT
   ```
   plate              24000.00 mm³   6 faces
   6 through          23717.26 mm³   7 faces
   + 8 blind 5        23465.93 mm³   9 faces
   + counterbore      22782.63 mm³  12 faces
   + 4 side, 20 deep  22531.31 mm³  14 faces
   cb 5 on hole 6 -> error: invalid input: counterbore diameter 5 must be
       larger than the hole diameter 6
   blind 4, cb 6 -> error: invalid input: hole depth 4 must be larger than
       the counterbore depth 6
   off the plate -> error: invalid input: the hole misses the solid: it
       removes nothing
   diameter 0 -> error: invalid input: hole diameter must be finite and > 0,
       got 0
   …  (the geometry engine's file reports)
   wrote holes/hole-plate.step
   ```

   Four long error lines are wrapped here to fit the page.

2. Read the `6 through` line. A 6 mm hole through the 10 mm plate takes away a cylinder of $\pi \times 3^2 \times 10 = 282.74$ mm³. It adds one face: the wall of the hole.

3. Read the `8 blind 5` line. The 8 mm hole stops 5 mm down and takes away 251.33 mm³. It adds two faces: the wall and the flat bottom.

4. Read the `counterbore` line. The 6 mm hole goes through, and the 11 mm recess is 6 mm deep, so 683.30 mm³ goes in all. It adds three faces: the recess wall, the flat step at the recess's bottom, and the wall of the narrow hole.

5. Read the `4 side` line. This hole starts on the plate's end at x = 60 and points back along −X, so it goes in sideways, 20 mm deep.

6. Read the four errors. Each one says what is wrong, and the plate is unchanged after each.

7. Draw the plate:

   TERMINAL
   ```
   target/release/swx-qa render --step holes/hole-plate.step -o hole-plate.png
   ```

   Open `hole-plate.png`. You should see the plate of Figure 11.1.

**RESULT** A plate with four holes, each made by one call. Each call gives a new solid; the one before it is left as it was.

### How the sample is built

The program is `crates/swx-kernel/examples/hole_plate.rs`. Each hole is one call to `hole`, with four things: the solid, where it starts and which way it goes, its shape, and its depth.

1. **Say where it starts and which way it goes.** An `Axis` holds a point and a direction. The sample's top holes start on the top face, at z = 10, and point straight down. The direction doesn't need to be one unit long. The sample's helper `down_at(x, y)` builds this axis for any spot on the top:

```
let at = Axis { origin: [12.0, 20.0, 10.0], dir: [0.0, 0.0, -1.0] };
```

2. **Give its shape and depth, and drill.** `HoleKind::Simple` takes a diameter; `HoleKind::Counterbore` takes the hole's diameter, the recess's diameter and the recess's depth. `HoleDepth::Blind` takes a depth in mm; `HoleDepth::ThroughAll` takes nothing:

```
let one = k.hole(&plate, at, HoleKind::Simple { diameter: 6.0 },
                 HoleDepth::ThroughAll)?;
let cb = HoleKind::Counterbore {
    diameter: 6.0, cb_diameter: 11.0, cb_depth: 6.0,
};
let two = k.hole(&one, down_at(48.0, 20.0), cb, HoleDepth::ThroughAll)?;
```

> **TIP** **Sizes for a screw**
>
> A counterbore is sized for the screw it holds: the hole a little wider than the screw, the recess a little wider than its head, and the recess at least as deep as the head is tall. The sample's 6 mm hole, 11 mm recess and 6 mm depth are close to the usual sizes for a 5 mm cap screw. Check the sizes your screws need.

> **CAUTION** **Depths are measured from the start point**
>
> A blind depth and a counterbore depth both run from the start point along the direction. Put the start point on the face where the hole begins: start it 2 mm above the part, and a 5 mm hole is only 3 mm deep in the material.

## 11.2 Drill holes for screws

A screw holds two parts together in one of two ways. Either it passes through a plain hole in the top part and screws into the part below, or it passes through both and a nut holds it. The part it screws into needs a hole with a *thread* cut inside it: a spiral ridge that matches the screw's. A hole with a thread inside it is a tapped hole, because the thread is cut with a tool called a tap.

Threads come in standard sizes. The commonest are the ISO metric threads, named `M` and the screw's outside diameter in mm: an M6 screw is 6 mm across. Each size has a *pitch*, the distance from one ridge of the thread to the next. The "coarse" pitch is the one you get unless you ask for another; for M6 it is 1 mm. Before you tap a hole, you drill it narrower than the screw, so the tap has material to cut the thread into. That narrower size is the *tap drill*. A common workshop rule gives it: the screw's diameter minus its pitch, so 5 mm for M6.

Saulitwerx has the metric coarse thread sizes from M2 to M24 built in (Section 19.5). Ask it for a size by name, such as `"M6"`, and hand that size to a tapped hole. The hole is drilled at the tap-drill diameter.

A countersink is a cone at a hole's mouth, so a screw with a cone-shaped head sits flush with the surface. You give the hole's diameter, the cone's diameter at the surface, and the angle across the cone. Most metric flat-head screws have a 90° head.

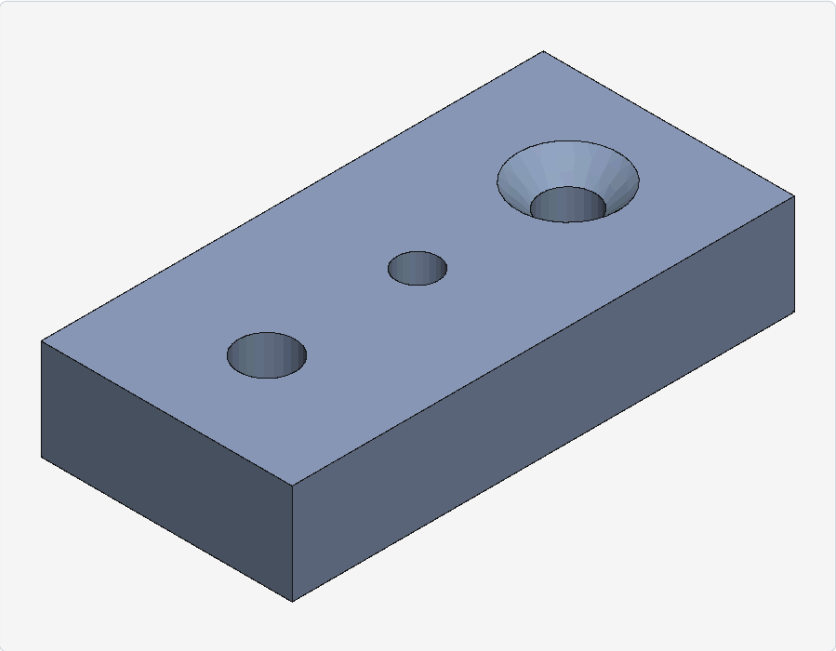


**Figure 11.2** A 60 × 30 × 12 mm plate with holes for screws. From the back: a 6.4 mm hole through the plate with a 90° countersink 12 mm across, a tapped M6 hole 10 mm deep (drilled 5 mm), and a tapped M8 hole through the plate (drilled 6.75 mm).

| | |
|---|---|
| **YOU WILL** | Drill a countersunk hole and two tapped holes in a plate, and look up thread sizes. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). The thread table needs no OpenCascade. |
| **TIME** | 5 minutes. |

1. Print the thread table:

TERMINAL

```
cargo run -p swx-kernel --example thread_table
```

It prints each size with its pitch, its minor diameter (the inside of the thread's ridges) and its tap drill, then four lookups by name:

OUTPUT

```
size   pitch  minor   tap drill
M2      0.40   1.57   1.60
…
M6      1.00   4.92   5.00
M8      1.25   6.65   6.75
…
M24     3.00  20.75  21.00
"M6" -> M6, pitch 1 mm, tap drill 5 mm
"m10" -> M10, pitch 1.5 mm, tap drill 8.5 mm
"M7" -> error: invalid input: no metric coarse thread of size M7
    (sizes: M2 to M24)
"6" -> error: invalid input: thread designation "6" must be M
    followed by a size, e.g. "M6"
```

Rows are left out here (…); the whole table is in Section 19.5. Two long error lines are wrapped to fit the page.

2. Read the lookups. A small `m` works as well as a capital. M7 isn't a standard coarse size, and a size needs its `M`.

3. Run the plate sample, and tell it to save its part in a folder called `holes`:

```
TERMINAL
cargo run -p swx-kernel --features occt --example hole_screw -- holes
```

It prints the plate, one line after each hole, four errors on purpose, then the file it wrote:

```
OUTPUT
plate            21600.00 mm³   6 faces
countersink      21112.16 mm³   8 faces
+ M6 blind 10     20915.81 mm³  10 faces
+ M8 through      20486.39 mm³  11 faces
csk 6 on hole 6.4 -> error: invalid input: countersink diameter 6 must
    be larger than the hole diameter 6.4
angle 180° -> error: invalid input: countersink angle 3.141592653589793
    (180°) must be strictly between 0 and π radians
blind 2 -> error: invalid input: hole depth 2 must be larger than the
    countersink depth 2.8
M7 -> error: invalid input: no metric coarse thread of size M7 (sizes:
    M2 to M24)
…  (the geometry engine's file reports)
wrote holes/hole-screw.step
```

Four long error lines are wrapped here to fit the page.

4. Read the `countersink` line. The cone narrows from 12 mm to 6.4 mm at 90°, so it is (12 − 6.4) ÷ 2 = 2.8 mm deep. With the hole below it, 487.84 mm³ goes. It adds two faces: the cone and the hole's wall.

5. Read the `M6 blind 10` line. The hole is 5 mm across, M6's tap drill, not 6 mm: π × 2.5² × 10 = 196.35 mm³ goes. It adds two faces, the wall and the flat bottom.

6. Read the `M8 through` line. The hole is 6.75 mm across, 8 mm minus the 1.25 mm pitch, and adds one face.

7. Read the errors. The last number in the third one is 2.8, printed with every digit the computer holds. The plate is unchanged after each.

8. Draw the plate:

```
TERMINAL
target/release/swx-qa render --step holes/hole-screw.step -o hole-screw.png
```

Open `hole-screw.png`. You should see the plate of Figure 11.2.

**RESULT** A plate with a countersunk hole for a flat-head screw and two holes ready to be tapped, each made by one call.

### How the sample is built

The programs are `crates/swx-kernel/examples/thread_table.rs` and `crates/swx-kernel/examples/hole_screw.rs`. They use the same `hole` call as Section 11.1, with two more shapes.

1. **Make a countersink.** `HoleKind::Countersink` takes the hole's diameter, the cone's diameter at the start point, and the angle across the cone in radians. `FRAC_PI_2` is $\pi/2$, that is 90°:

```
let countersink = HoleKind::Countersink {
    diameter: 6.4, csk_diameter: 12.0, csk_angle: FRAC_PI_2,
};
let csk = k.hole(&plate, down_at(12.0, 15.0), countersink,
                 HoleDepth::ThroughAll)?;
```

2. **Look up a thread and tap a hole.** `metric_coarse` turns a name into a `MetricThread`, which knows its `pitch` and its `tap_drill_diameter()`. `HoleKind::Tapped` takes it:

```
let m6 = metric_coarse("M6")?;
let tapped = k.hole(&csk, down_at(30.0, 15.0),
                    HoleKind::Tapped { thread: m6 },
                    HoleDepth::Blind(10.0))?;
```

> **TIP** **Sizes for a flat-head screw**
>
> The hole should be a little wider than the screw, and the cone at least as wide as the screw's head. The sample's 6.4 mm hole and 12 mm cone are close to the usual sizes for an M6 flat-head screw. Check the sizes your screws need.

> **CAUTION** **The tap drill comes from a rule of thumb**
>
> The tap-drill diameter here is the screw's diameter minus its pitch. Workshops sometimes use a slightly different drill, such as 6.8 mm rather than 6.75 mm for M8. If your shop or the screw's maker gives a size, use a simple hole of that size instead.

## 11.3 Common mistakes

- **A direction pointing away from the part.** The hole goes the wrong way and finds nothing: `invalid input: the hole misses the solid: it removes nothing`. Point the direction into the material.
- **A start point off the part.** The same message when the hole's line misses the part altogether. Check the coordinates against the part's bounding box (Section 19.4).
- **A counterbore no wider than its hole.** `invalid input: counterbore diameter 5 must be larger than the hole diameter 6`.
- **A blind hole no deeper than its counterbore.** `invalid input: hole depth 4 must be larger than the counterbore depth 6`. Make the hole deeper, or use a through hole.
- **A size of zero or less.** `invalid input: hole diameter must be finite and > 0, got 0` (or `hole depth`, `counterbore diameter`, `counterbore depth`).
- **No direction.** `invalid input: hole direction must be finite and non-zero, got [0.0, 0.0, 0.0]`.
- **A hole that takes the whole part.** `empty result: the hole removes the whole solid`: the hole is wider than the part.
- **A countersink no wider than its hole.** `invalid input: countersink diameter 6 must be larger than the hole diameter 6.4`.

- **A countersink angle in degrees, or of 180° or more.**
  `invalid input: countersink angle 3.141592653589793 (180°) must be strictly between 0 and π radians`. The message shows the angle you gave in radians, then the same angle in degrees, so a slip is easy to spot: an angle typed as `90` shows as 5156.620156°. Give the angle in radians, between 0 and π: `90f64.to_radians()`.
- **A blind hole no deeper than its countersink.**
  `invalid input: hole depth 2 must be larger than the countersink depth 2.8`. The cone's depth is (cone diameter − hole diameter) ÷ 2, divided by the tangent of half the angle; for 90°, just (cone diameter − hole diameter) ÷ 2. Make the hole deeper, or use a through hole.
- **A thread size that isn't in the table.** `invalid input: no metric coarse thread of size M7 (sizes: M2 to M24)`, or `thread designation "6" must be M followed by a size, e.g. "M6"` when the `M` is missing.
- **A home-made thread with a pitch as big as its diameter.**
  `invalid input: thread pitch 6 must be smaller than the nominal diameter 6`. Take sizes from `metric_coarse` rather than typing them.

CHAPTER 12

# Tilting faces with draft

> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

Plastic and cast parts are made in molds. Once the material has set, the part has to come out. If its sides are perfectly upright, they rub against the mold all the way out, and the part sticks or scrapes. So designers tilt the sides a little, a degree or a few, so that the part comes free as soon as it starts to move. That tilt is called *draft*.

Three things set a draft:

- **The *pull direction*:** the way the part leaves the mold.
- **The *neutral plane*:** a flat plane across the part, at right angles to the pull direction. Where each tilted face crosses it, the face stays where it was; the face turns about that line. On a part that sits on a mold's floor, the neutral plane is usually its bottom, so the bottom keeps its size.
- **The angle,** and which faces to tilt. A positive angle makes the part smaller as you go along the pull direction, which is what lets it come out. A negative angle makes it larger.

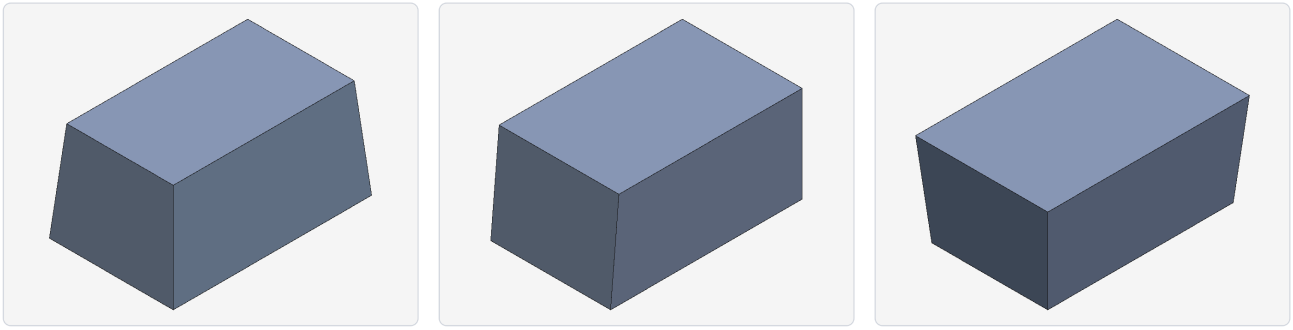



**Figure 12.1** A 40 × 25 × 20 mm block, drafted three ways. The neutral plane is the bottom and the part is pulled up, so the bottom keeps its size and the top changes: 1.75 mm in from each tilted side at 5°, 1.75 mm out at −5°.

**In Saulitwerx today:** the geometry engine can draft flat faces of a solid by one angle, about one neutral plane. You give the faces by their numbers in the solid (Section 6.3), and the neutral plane as a point and the pull direction. Every face in one call gets the same angle. A draft isn't recorded in a part's history yet.

## 12.1 Draft the sides of a block

| | |
|---|---|
| **YOU WILL** | Tilt the four sides of a block by 5°, then one side only, then the four sides outward. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `draft`:

```
TERMINAL
cargo run -p swx-kernel --features occt --example draft_block -- draft
```

It prints the block and its faces, one line for each draft with the size of the bottom and the top, four errors on purpose, then the files it wrote:

```
OUTPUT
block          20000.00 mm³  bottom 40.00 × 25.00 mm  top 40.00 × 25.00 mm
side faces [1, 0, 3, 2], top face 5
4 sides, 5°    17806.94 mm³  bottom 40.00 × 25.00 mm  top 36.50 × 21.50 mm
+X side, 5°    19562.56 mm³  bottom 40.00 × 25.00 mm  top 38.25 × 25.00 mm
4 sides, -5°   22356.35 mm³  bottom 40.00 × 25.00 mm  top 43.50 × 28.50 mm
top face -> error: kernel failure: draft failed
90° -> error: invalid input: draft angle must be finite, non-zero and
    below π/2 in size, got 1.5707963267948966
0° -> error: invalid input: draft angle must be finite, non-zero and
    below π/2 in size, got 0
no faces -> error: invalid input: the face list is empty
…  (the geometry engine's file reports)
wrote draft/draft-four.step
wrote draft/draft-one.step
wrote draft/draft-flared.step
```

Two long error lines are wrapped here to fit the page.

2. Read the second line. The four upright sides are faces 0 to 3, and the top is face 5.

3. Read the `4 sides, 5°` line. The bottom is still 40 × 25 mm. Each side leans in by 20 × tan 5° = 1.75 mm over the block's 20 mm height, so the top is 3.5 mm smaller each way: 36.50 × 21.50 mm.

4. Read the `+X side` line. Only the end facing +X leans, so only the length shrinks at the top.

5. Read the `-5°` line. A negative angle leans the sides out, so the top is 3.5 mm larger each way.

6. Read the `top face` line. The top is parallel to the neutral plane, so it doesn't cross it and has no line to turn about. The geometry engine can't draft it and says so. The block is unchanged.

7. Draw the drafted block:

```
TERMINAL
target/release/swx-qa render --step draft/draft-four.step -o draft-four.png
```

Open `draft-four.png`. You should see the first block of Figure 12.1. Draw the other files the same way.

**RESULT** A block drafted three ways, each by one call. The block itself is left as it was.

### How the sample is built

The program is `crates/swx-kernel/examples/draft_block.rs`. It does two things.

1. **Find the faces.** The sample finds each side by the way its outward normal points, as Section 10.1 found the top of a block, with `face_plane` (Section 7.6).

2. **Draft them.** `draft` takes the solid, the list of faces, the neutral plane and the angle in radians. The neutral plane is an `Axis`: a point on the plane, and the pull direction, which is also the plane's normal:

```rust
let pull_up = Axis { origin: [0.0, 0.0, 0.0], dir: [0.0, 0.0, 1.0] };
let tapered = k.draft(&block, &sides, pull_up, 5f64.to_radians())?;
```

> **TIP** **How much draft**
>
> One or two degrees is common for plastic parts, and more for textured surfaces or deep parts. Ask whoever makes the mold. Draft the sides before you round their edges: a fillet made first would have to be tilted too.

> **CAUTION** **Angles are in radians**
>
> `5.0` means 5 radians, which is refused. Write `5f64.to_radians()`. The angle must not be zero, and must be less than $\pi/2$ (90°) either way.

## 12.2 Common mistakes

- **A face parallel to the neutral plane.** A top or a bottom can't be drafted: `kernel failure: draft failed`. Draft the faces that run along the pull direction.
- **An angle of zero, 90° or more, or in degrees.** `invalid input: draft angle must be finite, non-zero and below π/2 in size, got 0`.
- **No faces.** `invalid input: the face list is empty`.
- **An edge or a vertex in the list, or a face number from another solid.** `invalid input: expected a face, got edge 0`, or `face index 6 is out of range: the shape has 6 faces`. Find the faces in the solid you are drafting.
- **No pull direction.** `invalid input: draft pull direction must be finite and non-zero, got [0.0, 0.0, 0.0]`.
- **The wrong sign.** If the part grows where it should shrink, the pull direction points the wrong way, or the angle's sign is wrong. Flip one of them.

CHAPTER 13

# Lofting between sections

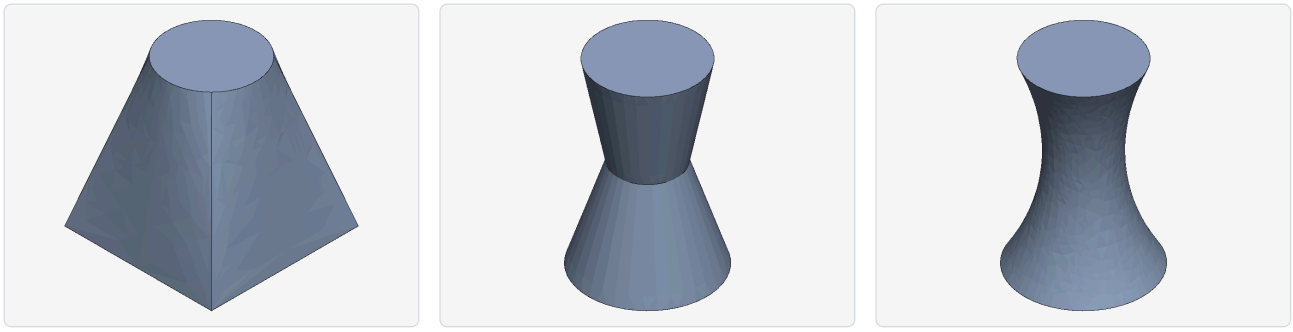
> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

An extrude pushes one outline straight, so the part has the same cross-section all the way along. A revolve turns one outline, so the part is round. Many real shapes are neither: a duct that starts square and ends round, a bottle that swells and narrows, a handle that tapers. For those you draw the shape at a few places along the part and let the computer fill in between.

That is a *loft*. Each outline you draw is a *section*: a flat face, like a slice through the finished part. The loft joins the sections in the order you give them, and closes the two ends with the first and last sections. It can join them two ways:

- **Ruled:** straight surfaces from each section to the next, like a lampshade made of straight strips. Each pair of neighbors gets its own surface, so you see a crease at every middle section.
- **Smooth:** one curved surface that passes through every section with no crease, like a vase thrown on a wheel.

With only two sections, ruled and smooth give the same shape.



Square to round | Three circles, ruled | Three circles, smooth

**Figure 13.1** Three lofts. Left: a 40 × 40 mm square at the bottom, a circle 24 mm across 40 mm higher. Middle and right: circles 40, 20 and 32 mm across at heights 0, 30 and 60 mm, joined ruled and smooth. The faint ripples on the curved faces come from the picture's coarse mesh, not from the part.

**In Saulitwerx today:** the geometry engine can loft through two or more flat sections, ruled or smooth. Each section is one face from `make_face` (Chapter 7), on any plane, with no holes. You give the sections in order; the loft has no guide curves yet, and no control over which point of one section joins which point of the next. A loft makes a new solid; to add it to a part, join it with `union` (Section 4.9). It isn't recorded in a part's history.

## 13.1 Loft through sections

| | |
|---|---|
| YOU WILL | Loft a square into a circle, then a vase through three circles, ruled and smooth. |
| YOU NEED | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| TIME | 5 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `loft`:

   TERMINAL

   ```
   cargo run -p swx-kernel --features occt --example loft_sections -- loft
   ```

   It prints one line for each loft, four errors on purpose, then the files it wrote:

   OUTPUT

   ```
   square to round  38889.24 mm³  7 faces  height 40.00 mm
   vase, ruled      38201.77 mm³  4 faces  height 60.00 mm
   vase, smooth     31508.61 mm³  3 faces  height 60.00 mm
   one section -> error: invalid input: a loft needs at least 2 sections,
       got 1
   hole in top -> error: invalid input: loft section 1 has a hole; sections
       with holes are not supported
   solid section -> error: invalid input: loft section 1 is not a single
       planar face
   same section twice -> error: kernel failure: loft failed
   …  (the geometry engine's file reports)
   wrote loft/loft-transition.step
   wrote loft/loft-ruled.step
   wrote loft/loft-smooth.step
   ```

   Four long error lines are wrapped here to fit the page.

2. Read the `square to round` line. The part is 40 mm tall, from the square up to the circle.

3. Read the `vase, ruled` line. It is two cones stacked at the waist: 4 faces, a bottom, a top and one side for each pair of sections. Its volume is the two cones' volumes added up.

4. Read the `vase, smooth` line. It has one side face, curving through all three circles. Because the side bows inward between the sections instead of running straight, it holds less: 31508.61 mm³ against 38201.77 mm³.

5. Read the errors. A loft needs two sections or more; each section must be one flat face without a hole; and two sections in the same place give nothing to fill, so the geometry engine refuses them.

6. Draw the lofts:

   TERMINAL

   ```
   target/release/swx-qa render --step loft/loft-smooth.step -o loft-smooth.png
   ```

   Open `loft-smooth.png`. You should see the right-hand part of Figure 13.1. Draw the other files the same way.

**RESULT** Three solids, each from one call, that no extrude or revolve could make.

### How the sample is built

The program is `crates/swx-kernel/examples/loft_sections.rs`. It does two things.

1. **Make the sections.** Each section is a face from `make_face` (Chapter 7): an outline on a workplane. The sample puts each one on the XY plane raised to its height with `offset` (Section 15.1):

```
let bottom = k.make_face(&Workplane::XY, &square, &[])?;
let top = k.make_face(&Workplane::XY.offset(40.0)?, &circle, &[])?;
```

2. **Loft them.** `loft` takes the sections in order, and `true` for ruled or `false` for smooth:

```
let transition = k.loft(&[bottom, top], true)?;
let vase = k.loft(&rings, false)?;
```

> **TIP** **Sections don't have to be parallel**
>
> Each section can sit on its own plane: a work plane tilted or turned (Chapter 15) gives a loft that bends. Keep the sections in the order the part runs, and don't let them cross each other.

> **TIP** **Ruled or smooth**
>
> Choose ruled when the part really is made of straight pieces, such as a sheet-metal transition, and smooth for anything that should flow, such as a handle or a nozzle. With a smooth loft, add a section where you want to control the shape: the surface passes through every section you give it.

## 13.2 Common mistakes

- **One section, or none.** `invalid input: a loft needs at least 2 sections, got 1`.
- **A section with a hole.** A ring-shaped outline is refused: `invalid input: loft section 1 has a hole; sections with holes are not supported`. Loft the outer outline, then cut the hole out with a second loft or an extrude.
- **A solid in place of a face.** `invalid input: loft section 1 is not a single planar face`. Pass faces from `make_face`, not the solids made from them.
- **Two sections in the same place.** `kernel failure: loft failed`. Move one section, or leave it out.
- **Sections out of order.** The loft joins them in the order of the list, so a list out of order makes a part that folds back on itself, or fails. Sort them along the part.
- **Very different outlines.** A star into a circle, say, may twist or fail, because the loft has to decide which point of one joins which point of the next. Give the outlines a similar number of pieces, starting at similar places.

CHAPTER 14

# Sweeping along a path

> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

Some parts have the same cross-section all the way along, but don't run straight: a bent rod, a pipe with an elbow, a handle that curves over the top of a box, a rib that follows the edge of a plate. An extrude can only push straight, and a revolve only turns around one axis. A *sweep* moves a flat outline along a line you draw, and the outline leaves a solid behind it as it goes.

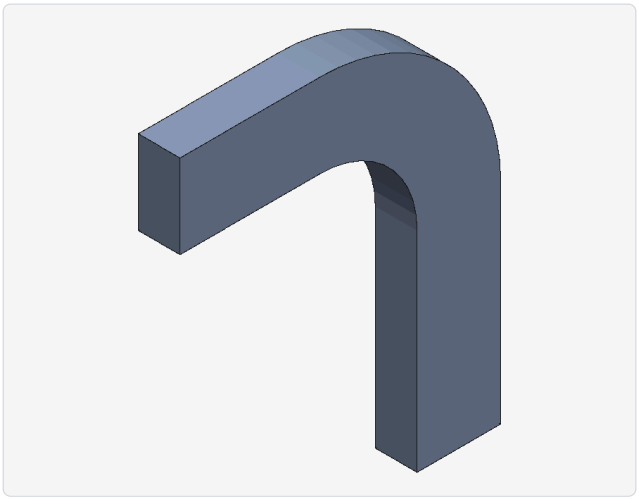
A sweep needs two things:

- **The profile:** the flat outline that gives the cross-section, such as a circle for a rod or a rectangle for a bar. It is a face from `make_face`, as for an extrude (Chapter 7).
- **The *path*:** the line the profile travels along, made of straight lines and arcs joined end to end. It is drawn on a plane of its own, and it must start on the profile's plane, usually at the profile's middle.

The profile keeps its size and shape all the way. Through each bend it turns with the path, so it stays square to the direction of travel and the part doesn't twist.



Circle: a bent rod



Rectangle: a bent bar

**Figure 14.1** Two profiles swept along one path: up 30 mm, a quarter circle of radius 20 mm, then 20 mm level. Left, a circle 10 mm across; right, a 12 × 6 mm rectangle.

**In Saulitwerx today:** the geometry engine can sweep one flat profile, with no holes, along an open path of lines and arcs that all lie in one plane, or around a helix (Section 14.2). The path can't be a closed loop, a full circle, a spline or any other curve in 3D yet, and the profile can't twist or change size along the way. A sweep makes a new solid; to add it to a part or cut it from one, use `union` or `cut` (Section 4.9). It isn't recorded in a part's history.

## 14.1 Sweep a profile along a path

| | |
|---|---|
| **YOU WILL** | Sweep a circle and a rectangle along a path with a bend, and see the errors. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `sweep`:

   TERMINAL
   ```
   cargo run -p swx-kernel --features occt --example sweep_pipe -- sweep
   ```

   It prints one line for each sweep, five errors on purpose, then the files it wrote:

   OUTPUT
   ```
   rod         6394.39 mm³  5 faces  box 45 × 10 × 55 mm
   bar         5861.95 mm³  14 faces  box 46 × 6 × 56 mm
   starts above -> error: invalid input: sweep path starts 5 off the
       profile's plane
   gap -> error: invalid input: sweep path: segment 1 doesn't meet segment 0
       within 0.0000001
   circle path -> error: invalid input: sweep path: circles (closed paths)
       are not supported
   profile with hole -> error: invalid input: sweep profile has a hole;
       profiles with holes are not supported
   tight bend -> error: invalid input: sweep path: arc segment 1 has radius
       4, not larger than the profile's distance 5.000000232604544 from the
       path's start; the swept solid would pass through itself
   …  (the geometry engine's file reports)
   wrote sweep/sweep-rod.step
   wrote sweep/sweep-bar.step
   ```

   Five long error lines are wrapped here to fit the page.

2. Read the `rod` line. The path is 30 mm + a quarter circle of 20 mm radius (31.42 mm) + 20 mm = 81.42 mm long. The circle's area is $\pi \times 5^2 = 78.54$ mm², and $78.54 \times 81.42 = 6394.39$ mm³: a sweep holds its profile's area times its path's length.

3. Read the `bar` line. The rectangle is 72 mm², so the bar holds $72 \times 81.42 = 5861.95$ mm³. Each of its four sides is split into three faces, one for each piece of the path, which with the two ends makes 14.

4. Read the errors. The path must start on the profile's plane, its pieces must meet end to end, it can't be a circle, and the profile can't have a hole. The last one bends the rod round a 4 mm radius, tighter than the rod's own 5 mm radius, so the sweep refuses it (see the caution below).

5. Draw the rod:

   TERMINAL
   ```
   target/release/swx-qa render --step sweep/sweep-rod.step -o sweep-rod.png
   ```

   Open `sweep-rod.png`. You should see the left-hand part of Figure 14.1.

**RESULT** A bent rod and a bent bar, each from one call.

### How the sample is built

The program is `crates/swx-kernel/examples/sweep_pipe.rs`. It does three things.

1. **Make the profile.** A circle 10 mm across on the XY plane, centered on the origin:

```
let round = k.make_face(&Workplane::XY, &circle, &[])?;
```

2. **Draw the path.** The path goes on the XZ plane, where u runs along X and v up along Z, so it starts at the origin, in the middle of the profile, and rises square to it. It is a list of `Segment2` pieces, as an outline is (Section 19.6), but open:

```
let path = [
    Segment2::Line { start: [0.0, 0.0], end: [0.0, 30.0] },
    Segment2::Arc { center: [20.0, 30.0], radius: 20.0,
                    start_angle: FRAC_PI_2, end_angle: PI },
    Segment2::Line { start: [20.0, 50.0], end: [40.0, 50.0] },
];
```

   An arc always runs counterclockwise from its start angle to its end angle, so this one runs from (20, 50) back to (0, 30). That's fine: the sweep follows the path from the first piece's start, and takes each next piece whichever way round meets the end of the one before.

3. **Sweep.** `sweep` takes the profile, the path's plane and the path:

```
let rod = k.sweep(&round, &Workplane::XZ, &path)?;
```

> **TIP** **Start square to the profile**
>
> Start the path at the profile's middle, heading straight out of the profile's plane, as the sample does. A path that starts at a slant leans the whole part over; one that starts flat along the profile's plane gives nothing to sweep, and fails.

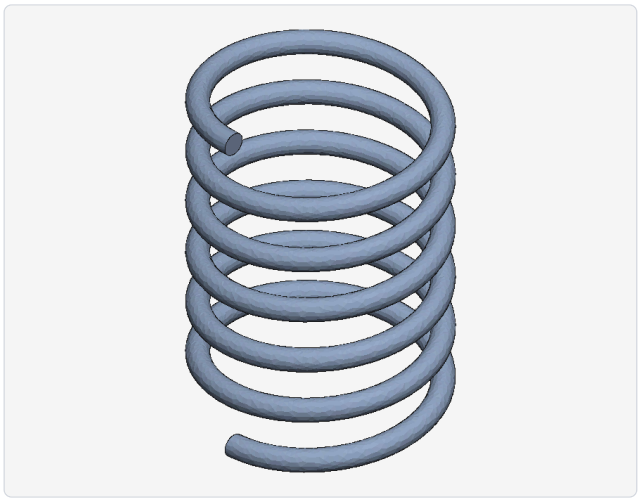
> **CAUTION** **Keep bends wider than the profile**
>
> Picture bending a thick rod round a tight corner: the inside of the bend gets squeezed until it would have to pass through itself. A sweep can't build that, so it refuses any arc whose radius isn't larger than the profile's reach: the distance from the path's start to the profile's farthest point, measured in the profile's plane. The message names the arc (pieces are counted from 0), its radius and that distance. In the sample, the rod's reach is its 5 mm radius (shown as 5.000000232604544, because it is measured on a fine mesh of the profile), so the 20 mm bend passes and the 4 mm bend is refused. For the 12 × 6 mm bar, the reach is its corner, $\sqrt{(6^2 + 3^2)}$ = 6.71 mm from the middle, so its bends must be wider than that. The check uses the farthest point in every direction, so it can refuse a bend that would just fit on the profile's narrow side. Make the bend wider, or the profile smaller.

## 14.2 Sweep around a helix: springs and coils

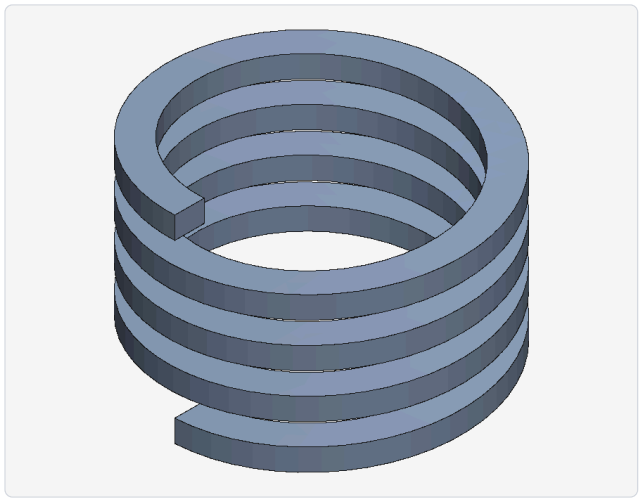
A coil spring is a round wire wound round and round while it climbs. The line its middle follows is a *helix*: the shape of a spiral staircase's handrail, which circles a center post and rises the same amount each time it goes round. How far it rises in one full turn is its *pitch*.

A *helical sweep* moves a flat profile along such a helix. You don't draw the path: you give the axis the helix turns about, the pitch and the number of turns, and the profile itself says how far from the axis the helix runs. As it goes

round, the profile turns with it and stays upright beside the axis, the way a door stays square to its hinge, so it doesn't twist. Springs, coils, and the winding ridge of a screw thread are all helical sweeps.



Circle: a coil spring



Rectangle: a square ridge

**Figure 14.2** Two profiles swept around the upright Z axis. Left, a circle 2 mm across, 10 mm from the axis: six turns at 5 mm a turn. Right, a 2 × 1.5 mm rectangle, 8 to 10 mm from the axis: four turns at 3 mm a turn.

**In Saulitwerx today:** the helix keeps one radius and one pitch all the way, and always winds right-handed, like a common screw. The profile is one flat face without holes, beside the axis. A helical sweep makes a new solid; to add it to a part or cut it from one, use `union` or `cut` (Section 4.9). Real screw threads, on a shaft or in a hole, aren't built yet (**Coming next**).

| | |
|---|---|
| **YOU WILL** | Wind a spring and a square ridge, and see the errors. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its parts in a folder called `helix`:

TERMINAL

```
cargo run -p swx-kernel --features occt --example helical_spring -- helix
```

It prints one line for each part, five errors on purpose, then the files it wrote:

OUTPUT

```
spring   1184.35 mm³  3 faces  box 22 × 22 × 32.0 mm
ridge     678.58 mm³  6 faces  box 20 × 20 × 13.5 mm
pitch 0 -> error: invalid input: helix pitch must be finite and > 0, got 0
axis along Y -> error: invalid input: helix axis (origin [0.0, 0.0, 0.0],
    direction [0.0, 1.0, 0.0]) is not in the profile's plane (through
    [0.0, 0.0, 0.0], normal [0.0, -1.0, 0.0])
on the axis -> error: invalid input: the profile crosses the helix axis:
    its distance from the axis spans [-1, 1]
pitch 1.5 -> error: invalid input: the profile's extent along the helix
    axis (2) must be less than the pitch (1.5), or neighboring turns overlap
tube -> error: invalid input: helical sweep profile has a hole; profiles
    with holes are not supported
…  (the geometry engine's file reports)
wrote helix/helix-spring.step
wrote helix/helix-ridge.step
```

Four long error lines are wrapped here to fit the page.

2. Read the `spring` line. The wire's middle is 10 mm from the axis, so one turn is about $2\pi \times 10 = 62.83$ mm long, and six turns about 377 mm. The wire's area is $\pi \times 1^2 = 3.14$ mm², and $3.14 \times 377 = 1184.35$ mm³. The spring stands $6 \times 5 = 30$ mm tall, plus the wire's own 2 mm. It is 22 mm across: twice 10 mm, plus the wire.
3. Read the `ridge` line. The rectangle's middle is 9 mm from the axis and its area is 3 mm², so four turns hold $4 \times 2\pi \times 9 \times 3 = 678.58$ mm³. It stands $4 \times 3 + 1.5 = 13.5$ mm tall.
4. Read the errors. The pitch must be more than 0. The axis must lie in the profile's plane: the profile is drawn on the XZ plane, which holds the Z axis but not the Y axis. The profile must sit to one side of the axis, not across it. It must be shorter, along the axis, than the pitch, or each turn would run into the next: the 2 mm wire can't climb only 1.5 mm a turn. And the profile can't have a hole.
5. Draw the spring:

```
TERMINAL
target/release/swx-qa render --step helix/helix-spring.step -o helix-spring.png
```

Open `helix-spring.png`. You should see the left-hand part of Figure 14.2.

**RESULT** A coil spring and a winding ridge, each from one call.

### How the sample is built

The program is `crates/swx-kernel/examples/helical_spring.rs`. It does three things.

1. **Choose the axis.** An `Axis` is a point and a direction (Section 19.7). Here it is the Z axis, through the origin and pointing up:

```
let z_axis = Axis { origin: [0.0, 0.0, 0.0], dir: [0.0, 0.0, 1.0] };
```

2. **Make the profile beside it.** The XZ plane holds the Z axis, with u running along X and v up along Z. A circle 2 mm across, centered 10 mm out along X, sits beside the axis:

```
let wire = k.make_face(&Workplane::XZ, &circle([10.0, 0.0], 1.0), &[])?;
```

3. **Sweep.** `helical_sweep` takes the profile, the axis, the pitch in mm and the number of turns:

```
let spring = k.helical_sweep(&wire, z_axis, 5.0, 6.0)?;
```

> **TIP** **Which way it winds**
>
> The helix starts at the profile and turns counterclockwise as you look down the axis from its tip, climbing towards the tip: a right-hand helix. To wind the other way up, point the axis the other way (`dir: [0.0, 0.0, -1.0]`); the part then climbs downwards from the profile. The number of turns needn't be whole: `0.25` makes a quarter turn.

> **NOTE** **Why the pitch doesn't change the volume**
>
> A spring with a steeper pitch has a longer wire, yet the same volume: 1184.35 mm³ for six turns whatever the pitch. The climb happens within the profile's own plane, so it sweeps out nothing; only going round the axis does. The volume is the number of turns, times $2\pi$, times the profile's area, times the distance of its middle from the axis, as for a revolve.

## 14.3 Common mistakes

- **A path that doesn't start on the profile.** `invalid input: sweep path starts 5 off the profile's plane`. The number is the distance, in mm. Start the path on the profile's plane, at its middle.
- **A gap in the path.** `invalid input: sweep path: segment 1 doesn't meet segment 0 within 0.0000001`. Segments are counted from 0. Reuse the same values for the end of one piece and the start of the next.
- **A closed path.** A circle is refused: `invalid input: sweep path: circles (closed paths) are not supported`. For a ring, revolve the profile instead (Chapter 8).
- **An empty path.** `invalid input: sweep path is empty`.
- **A profile with a hole.** `invalid input: sweep profile has a hole; profiles with holes are not supported`. To make a tube, sweep the outer circle, sweep the inner one along the same path, and cut the second from the first (Section 4.9). For the sample's path, circles 10 and 6 mm across give a tube of 4092.41 mm³.
- **A solid in place of the profile.** `invalid input: sweep profile is not a single planar face`. Pass the face from `make_face`.
- **A path that starts along the profile's plane.** The profile slides within its own plane and sweeps out nothing: `empty result: the sweep encloses no volume`. Draw the path on a plane that crosses the profile's plane.
- **A bend tighter than the profile.** The message names the arc, its radius and the profile's reach: `invalid input: sweep path: arc segment 1 has radius 4, not larger than the profile's distance 5.000000232604544 from the path's start; the swept solid would pass through itself`. Give the arc a radius larger than the distance shown (see the caution above).
- **A helix axis that leaves the profile's plane.** `invalid input: helix axis (origin …, direction …) is not in the profile's plane (through …, normal …)`. Draw the profile on a plane that holds the axis: XZ or YZ for the Z axis.
- **A profile across or touching the helix axis.** `invalid input: the profile crosses the helix axis: its distance from the axis spans [-1, 1]` (or `touches`). Move the profile to one side of the axis.
- **A pitch no larger than the profile is tall.** `invalid input: the profile's extent along the helix axis (2) must be less than the pitch (1.5), or neighboring turns overlap`. Raise the pitch, or make the profile shorter along the axis.
- **A pitch or a number of turns of 0 or less.** `invalid input: helix pitch must be finite and > 0, got 0` (or `helix turns`).

CHAPTER 15

# Work planes, axes and points

> **FOR DEVELOPERS**
>
> This chapter calls `swx-kernel` from Rust. Work features built from numbers need no geometry engine, so its first sample runs on a plain build; the others need OpenCascade and the `occt` build feature (Chapter 2).

Every sketch sits on a plane, every revolve turns about an axis, and every hole starts at a point. Often a face of the part gives you what you need: you sketch on the top, or drill into the side. But sometimes the plane, the line or the point you need isn't on the part at all: a plane 10 mm above the top, a plane tilted 30° to hold a sloping post, the line where two faces would meet. So you build it.

That reference geometry is called a *work feature*. It has no material of its own, so it never adds to or takes from the part; it is there to place other features. There are three kinds:

- A *work plane*: a flat plane to sketch on, extrude from or measure to.
- A *work axis*: a straight line, to revolve about or line things up on.
- A *work point*: one point in space, to start a hole at or measure from.

Every part starts with a set of them, all meeting at the *part origin*, the point (0, 0, 0): three planes, XY, XZ and YZ, and three axes, X, Y and Z, along their edges. You build new ones from those, or from each other (Figure 15.1).

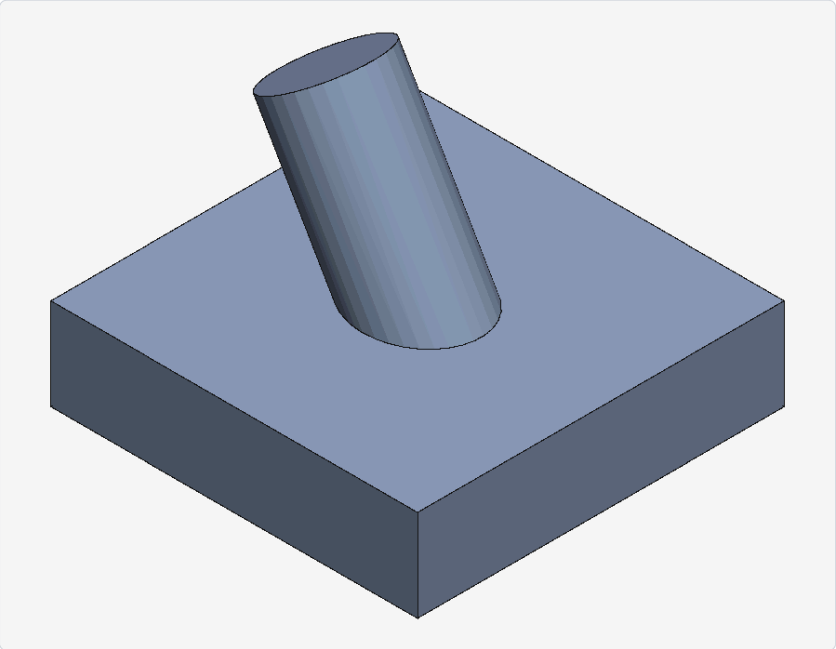


**Figure 15.1** A post that leans at 30°. Its circle is sketched on a work plane: the plate's top plane, made by offsetting the XY plane 10 mm, then turned 30° about a work axis across the middle of the top.

**In Saulitwerx today:** work planes, axes and points can be built from numbers, from the part origin and its planes and axes, and from each other. A work plane can be used wherever a `Workplane` goes (Section 19.6), and a work axis wherever an `Axis` goes: a revolve, a draft or a hole. They can also be taken from a part's own geometry: a plane on a flat face or touching a round one, an axis along a straight edge or through a round face, a point at a corner or halfway along an edge (Section 15.3, Section 15.4). A plane can also sit halfway between two others. You name that face, edge or corner by its number (Section 6.3); it can't be picked with the mouse yet, and the number doesn't follow the part through later changes. Work features aren't recorded in a part's history.

## 15.1 Build work features

The sample in this section builds one of each kind of work feature from the part origin and its planes, then shows the errors you can meet.

| | |
|---|---|
| **YOU WILL** | Build an offset plane, a plane through three points, a tilted plane, two axes and a point, and see the errors. |
| **YOU NEED** | Saulitwerx built from source (Chapter 2). OpenCascade isn't needed. |
| **TIME** | 5 minutes. |

1. Run the sample program:

   TERMINAL

   ```
   cargo run -p swx-kernel --example work_features
   ```

   It prints each work feature it builds, then five errors on purpose:

   OUTPUT

   ```
   part origin            (0, 0, 0)
   Z axis                 through (0, 0, 0)  along (0, 0, 1)
   XY plane               origin (0, 0, 0)  normal (0, 0, 1)
   XY offset 10           origin (0, 0, 10)  normal (0, 0, 1)
   through 3 points       origin (0, 0, 10)  normal (0, -0.447, 0.894)
   top turned 30°         origin (0, 2.679, 0)  normal (0, -0.5, 0.866)
   slope meets YZ         through (0, -4, 8)  along (0, 0.894, 0.447)
   through 2 points       through (0, 0, 0)  along (0.816, 0.408, 0.408)
   diagonal meets top     (20, 10, 10)
   two parallel planes -> error: invalid input: the planes are parallel:
       they have no line in common
   3 points in a line -> error: invalid input: the three plane points are
       collinear
   2 points the same -> error: invalid input: axis direction (the points
       coincide) must be non-zero
   X axis meets top -> error: invalid input: the axis is parallel to the plane
   turn about Z axis -> error: invalid input: the axis does not lie in the
       plane
   ```

   Four long error lines are wrapped here to fit the page.

2. Read the `XY offset 10` line. The plane has moved 10 mm up and still faces up. It is the top of a 10 mm plate.

3. Read the `through 3 points` line. The points (0, 0, 10), (40, 0, 10) and (0, 20, 20) make a slope that rises 10 mm over 20 mm, so its normal leans back from straight up: (0, −0.447, 0.894).

4. Read the `top turned 30°` line. The top plane turned 30° about a line along X through (0, 20, 10). Its normal now leans 30° from straight up: (0, −0.5, 0.866), since sin 30° = 0.5 and cos 30° = 0.866. Turning the plane moved its origin too, to (0, 2.679, 0); the turning line stays put.

5. Read the `slope meets YZ` line. It is the line where the slope meets the YZ plane. Its point is the one on the line nearest the origin.

6. Read the last two lines before the errors. The axis from (0, 0, 0) towards (40, 20, 20) crosses the top plane at (20, 10, 10), halfway along.
7. Read the errors. Each says why the work feature can't be built.

**RESULT** Six new work features, built from the origin's planes and axes and from each other.

Here are all the ways to build each kind. Every one checks its input and gives an error if the result wouldn't make sense, such as a plane through three points in a line.

| To get | Build it with |
|---|---|
| The origin and its planes and axes | `ORIGIN`, `Workplane::XY`, `::XZ`, `::YZ`, `Axis::X`, `::Y`, `::Z`. |
| A plane parallel to another | `plane.offset(distance)`: moved along its normal by `distance` mm; a negative distance goes the other way. |
| A plane through three points | `Workplane::through_points(p0, p1, p2)`. Its origin is `p0`; its normal follows the right-hand rule from `p0 → p1` to `p0 → p2`. |
| A plane through a point, square to a direction | `Workplane::from_point_normal(p, n)`. |
| A plane at an angle to another | `plane.angled(axis, angle)`: turned about an axis that lies in the plane, by an angle in radians (right-hand rule). |
| An axis through two points | `Axis::through_points(p, q)`, pointing from `p` to `q`. |
| The axis where two planes meet | `Axis::plane_intersection(a, b)`. |
| The point where an axis crosses a plane | `plane_axis_intersection(plane, axis)`. |
| A plane halfway between two parallel planes | `Workplane::midplane(a, b)`, facing the way `a` faces (Section 15.4). |

## 15.2 Place a feature on a work plane

A work plane is used like any other plane. The sample in this section builds the leaning post of Figure 15.1: it sketches a circle on a tilted work plane and extrudes it into the plate.

**YOU WILL** Stand a post at 30° on a plate, using a tilted work plane.
**YOU NEED** Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2).
**TIME** 5 minutes.

1. Run the sample program, and tell it to save its part in a folder called `work`:

TERMINAL

```
cargo run -p swx-kernel --features occt --example tilted_post -- work
```

It prints the tilted plane and the finished part:

OUTPUT

```
tilted plane: origin [0.0, 2.6794919243112254, 1.7763568394002505e-15],
    normal [0.000, -0.500, 0.866]
plate with post: 18261.95 mm³, 8 faces
…  (the geometry engine's file reports)
wrote work/tilted-post.step
```

The long first line is wrapped here to fit the page. The tiny number at the end of the origin is zero, give or take rounding.

2. Read the `plate with post` line. The plate is 40 × 40 × 10 mm, 16000 mm³. The post is 12 mm across and stands 20 mm out of the plate along its leaning axis, which adds $\pi \times 6^2 \times 20 = 2261.95$ mm³.

3. Draw the part:

```
TERMINAL
target/release/swx-qa render --step work/tilted-post.step -o tilted-post.png
```

Open `tilted-post.png`. You should see the part of Figure 15.1.

**RESULT** A post standing at an angle that no face of the plate could give you.

### How the sample is built

The program is `crates/swx-kernel/examples/tilted_post.rs`. It does three things.

1. **Build the work plane.** The plate's top plane is the XY plane moved up 10 mm. The work axis runs along X across the middle of the top, at y = 20. The tilted plane is the top turned 30° about it:

```
let top = Workplane::XY.offset(10.0)?;
let across = Axis { origin: [0.0, 20.0, 10.0], dir: [1.0, 0.0, 0.0] };
let tilted = top.angled(across, 30f64.to_radians())?;
```

2. **Sketch on it.** The circle goes on the plane at (u, v) = (20, 20): 20 mm along X, and 20 mm from the plane's origin, which is where the turning line is. The sample starts it on a plane 4 mm below the tilted one, so the whole foot of the post is buried in the plate:

```
let foot = tilted.offset(-4.0)?;
let face = k.make_face(&foot, &circle, &[])?;
```

3. **Extrude and join.** It extrudes 24 mm along the tilted normal and joins the post to the plate (Section 7.3):

```
let part = extrude_feature(&k, &face, Extent::Distance(24.0),
                           ExtrudeMode::Join, Some(&plate))?;
```

> **CAUTION** **Leave no gap under a leaning feature**
>
> A circle on a tilted plane rises on one side above the plane it turned from. Started right on the tilted plane, the post's high side would float above the plate, and the join would leave a gap. Start the sketch a little below, as the sample does, so the foot is inside the part.

## 15.3 Take work features from a part

Often the plane or line you need is already on the part: the face you want to sketch on, the edge you want to turn about, the middle of a round boss you want to drill down. Instead of working out its numbers, ask the geometry engine for it. The sample in this section takes all three kinds from a block with a round boss on top, then uses two of them (Figure 15.2).

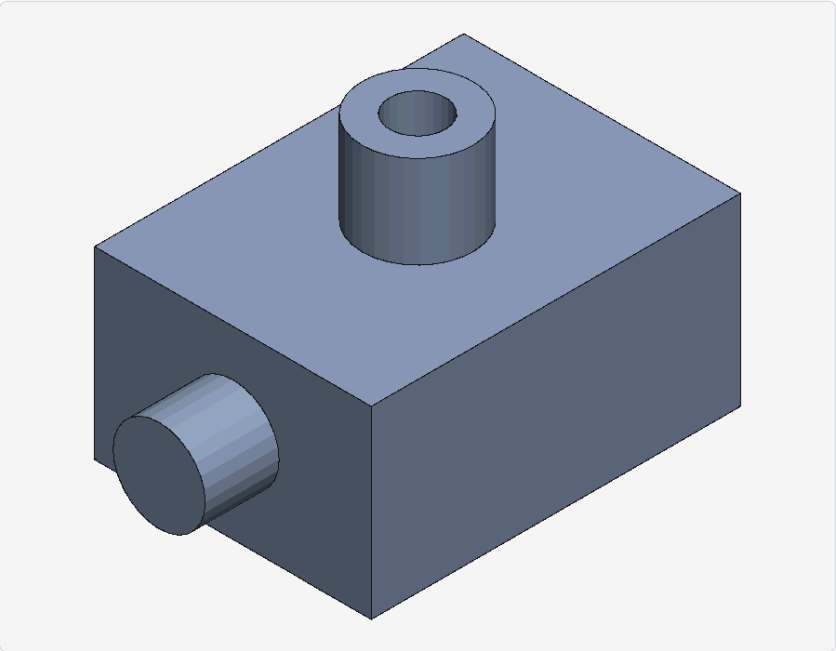


**Figure 15.2** A 40 × 30 × 20 mm block with a boss 12 mm across on top. The post on the side is sketched on a work plane taken from that face; the 6 mm hole runs down a work axis taken from the boss's round face.

| | |
|---|---|
| **YOU WILL** | Take a work plane, two work axes and a work point from a part, then sketch on the plane and drill along an axis. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its part in a folder called `work`:

TERMINAL

```
cargo run -p swx-kernel --features occt --example work_on_part -- work
```

It prints the numbers of the faces, edges and corner it uses, the work features it takes from them, four errors on purpose, and the finished part:

OUTPUT

```
side face 5, top edge 8, round face 6, rim edge 13, corner vertex 6
plane on side face     origin (40, 0, 0)  normal (1, 0, 0)  x_dir (0, 1, 0)
axis on top edge       through (40, 0, 20)  along (0, 1, 0)
axis on round face     through (20, 15, 0)  along (0, 0, 1)
point at corner        (40, 30, 20)
plane on edge -> error: invalid input: face_plane needs a face, got a
    Edge sub-shape
plane on round face -> error: invalid input: face 6 is not planar
axis on corner -> error: invalid input: work_axis_on needs an edge or a
    face, got a vertex
axis on rim -> error: invalid input: Edge 13 is not a straight edge or a
    cylindrical face
with post and hole: 24911.06 mm³, 11 faces
…  (the geometry engine's file reports)
wrote work/work-on-part.step
```

Three long error lines are wrapped here to fit the page.

2. Read the `plane on side face` line. The plane lies on the block's end at x = 40 and faces out of the part, along +X. Its origin is the point of the plane nearest the part origin, (40, 0, 0), and its u direction runs along +Y.
3. Read the two `axis` lines. The first runs along the top edge of that face, in the Y direction. The second is the boss's center line: straight up through (20, 15). Each passes through its point nearest the part origin, which for the boss is down at z = 0, on the bottom of the block.
4. Read the `point at corner` line: the top corner at the far end of the block.
5. Read the errors. A plane needs a flat face, and an axis needs a straight edge or a round (cylindrical) face. A corner has no line, and the boss's round top edge is a circle, not a line.
6. Read the last line. The block (24000 mm³) and the boss (1130.97 mm³) gained a post 10 mm across and 8 mm long (628.32 mm³), and lost a hole 6 mm across through 30 mm (848.23 mm³): 24911.06 mm³.
7. Draw the part:

   TERMINAL

   ```
   target/release/swx-qa render --step work/work-on-part.step -o work-on-part.png
   ```

   Open `work-on-part.png`. You should see the part of Figure 15.2.

**RESULT** Work features that sit exactly on the part, with no numbers worked out by hand, and two features placed on them.

### How the sample is built

The program is `crates/swx-kernel/examples/work_on_part.rs`. It does three things.

1. **Find the face, edge or corner.** Each one is a `SubShape`: its kind and its number in the part (Section 6.3). The sample finds each by its bounding box, with `sub_shape_bounding_box`.
2. **Take the work features.** One call for each kind:

   ```
   let plane = k.work_plane_on_face(&part, side)?;
   let along = k.work_axis_on(&part, edge)?;
   let boss_axis = k.work_axis_on(&part, round)?;
   let corner_point = k.vertex_point(&part, corner)?;
   ```

3. **Use them.** The post is a circle sketched on `plane` at (u, v) = (15, 10), the middle of the face, extruded 8 mm out and joined (Section 7.3). The hole is drilled through all along `boss_axis` (Chapter 11):

   ```
   let face = k.make_face(&plane, &circle, &[])?;
   let with_post = extrude_feature(&k, &face, Extent::Distance(8.0),
                                   ExtrudeMode::Join, Some(&part))?;
   let finished = k.hole(&with_post, boss_axis,
                         HoleKind::Simple { diameter: 6.0 }, HoleDepth::ThroughAll)?;
   ```

**CAUTION** **An axis from the part can point either way**

An axis taken from an edge or a round face runs along it, but which of its two directions you get isn't fixed. A hole or a revolve cares which way the axis points, so check it and flip it when needed, as the sample does before drilling: it makes the boss's axis point up, into the part, from its start on the bottom face.

**TIP** **A plane on a face faces out**

A plane taken from a face always faces out of the part, like the face itself. Sketch on it and extrude a positive distance to build outward from the face; extrude a negative distance, or cut, to go into the part.

## 15.4 Midplanes, tangent planes and midpoints

Some places on a part are easy to describe but awkward to work out by hand. The middle of a shaft. The flat that touches its round side. The middle of an edge. Three calls give you each one exactly:

- A *midplane* is the plane halfway between two parallel planes. Features placed on it sit in the middle of the part, such as a cross hole halfway along a shaft.
- A *tangent plane* touches a round face along one straight line without cutting into it, on the side you choose. Sketch on it to put a flat, a pad or a label on a shaft or a boss.
- An *edge midpoint* is the point halfway along an edge, measured along the edge. On a curved edge it lies on the curve.

The sample in this section uses all three on a shaft 16 mm across and 50 mm long (Figure 15.3).

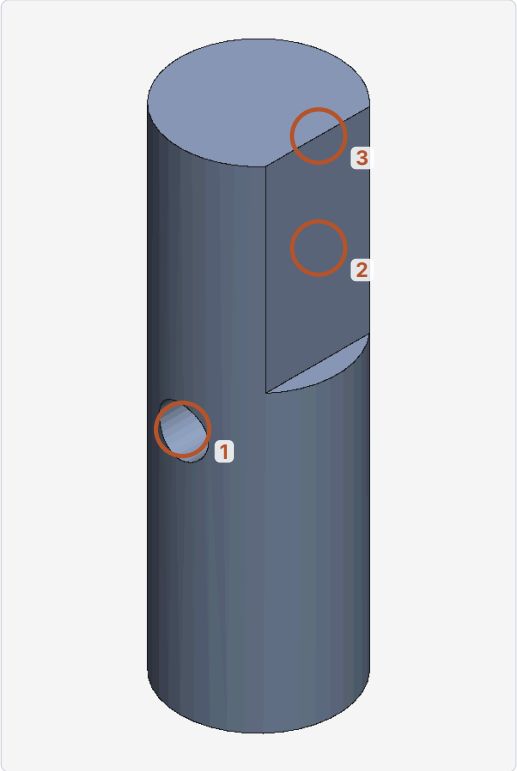



**Figure 15.3** A shaft with (1) a cross hole on the midplane between its ends, (2) a flat cut from a plane tangent to its side, and (3) the midpoint of the flat's top edge.

| | |
|---|---|
| **YOU WILL** | Take a midplane, a tangent plane and an edge midpoint from a shaft, and drill and cut on them. |
| **YOU NEED** | Saulitwerx built with OpenCascade, and `swx-qa` built with it too (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample program, and tell it to save its part in a folder called `work`:

   TERMINAL

   ```
   cargo run -p swx-kernel --features occt --example work_midplane -- work
   ```

   It prints the faces it uses, the two planes, four errors on purpose, the finished part and the midpoint:

   OUTPUT

   ```
   bottom face 2, top face 1, round face 0
   midplane               origin (0, 0, 25)  normal (0, 0, -1)
   tangent plane (+Y)     origin (0, 8, 0)  normal (0, 1, 0)  x_dir (1, 0, 0)
   midplane of XY and XZ -> error: invalid input: the planes are not
       parallel: they have no midplane
   tangent on top face -> error: invalid input: face 1 is not cylindrical
   tangent toward +Z -> error: invalid input: toward is parallel to the
       cylinder axis: it picks no side
   midpoint of a face -> error: invalid input: edge_midpoint needs an edge,
       got a Face sub-shape
   shaft with hole and flat: 9452.69 mm³, 6 faces
   flat's top edge 12: midpoint (0, 6, 50)
   …  (the geometry engine's file report)
   wrote work/work-midplane.step
   ```

   Three long error lines are wrapped here to fit the page.

2. Read the `midplane` line. It lies at z = 25, halfway up the 50 mm shaft. It was built from the planes of the bottom and the top faces, and it faces the way the first one does: the bottom face faces out of the part, down, so the normal is (0, 0, −1).

3. Read the `tangent plane` line. Asked for the +Y side, the plane touches the shaft at y = 8, its radius, and faces out along +Y.

4. Read the errors. A midplane needs two parallel planes; XY and XZ cross. A tangent plane needs a round face, and a side that points away from the axis, not along it. A midpoint needs an edge.

5. Read the `shaft with hole and flat` line. The shaft holds $\pi \times 8^2 \times 50 = 10053.10$ mm³. The flat, 2 mm deep along the top 20 mm, removes 290.12 mm³. The rest of the difference, 310.29 mm³, is the cross hole 5 mm across: a little less than $\pi \times 2.5^2 \times 16 = 314.16$ mm³, because the shaft's round side trims the ends of the hole.

6. Read the last line before the file. The flat's top edge runs straight across the shaft's top at y = 6. Halfway along, it crosses the Y axis: (0, 6, 50).

7. Draw the part:

   TERMINAL

   ```
   target/release/swx-qa render --step work/work-midplane.step -o work-midplane.png
   ```

   Open `work-midplane.png`. You should see the shaft of Figure 15.3.

**RESULT** A cross hole exactly halfway along the shaft, and a flat exactly 2 mm deep, with no positions worked out by hand.

### How the sample is built

The program is `crates/swx-kernel/examples/work_midplane.rs`. It does three things.

1. **The midplane and the cross hole.** The two end faces give two planes (Section 15.3), and `midplane` gives the one between them. The hole runs along the line where the midplane meets the XZ plane, starting outside the shaft at x = −10 and pointing into it (Chapter 11):

```
let mid = Workplane::midplane(&a, &b)?;
let across = Axis::plane_intersection(&mid, &Workplane::XZ)?;
```

2. **The tangent plane and the flat.** `work_plane_tangent` takes the round face and the side to touch. The sample moves the plane 2 mm into the shaft, sketches a rectangle on it over the top 20 mm, and cuts outward 5 mm, away from the axis (Section 7.3):

```
let tangent = k.work_plane_tangent(&shaft, side, [0.0, 1.0, 0.0])?;
let floor = tangent.offset(-2.0)?;
```

3. **The midpoint.** The sample finds the flat's top edge by its bounding box, then asks for its middle:

```
let middle = k.edge_midpoint(&finished, edge)?;
```

> **CAUTION** **Mind which way v runs**
>
> On a plane, u runs along `x_dir` and v along the normal × `x_dir` (Section 19.6). For the tangent plane here, that is −Z: down the shaft. So the top 20 mm of the shaft is v from −51 to −30, not 30 to 51. When a sketch on a work plane lands in the wrong place, check the directions the program prints.

> **TIP** **A midplane faces the way the first plane does**
>
> `midplane(a, b)` keeps `a`'s normal and directions, and swapping the two planes flips it. When a feature on the midplane goes the wrong way, swap them, or extrude a negative distance.

## 15.5 Common mistakes

- **Two parallel planes.** They never meet, so there's no line: `invalid input: the planes are parallel: they have no line in common`.
- **Three points in a line.** Many planes pass through them, so none is chosen: `invalid input: the three plane points are collinear`. The same if two of them are the same point.
- **Two points the same.** An axis needs two different points: `invalid input: axis direction (the points coincide) must be non-zero`.
- **An axis parallel to a plane.** It never crosses it: `invalid input: the axis is parallel to the plane`.
- **Turning a plane about an axis that isn't in it.** `invalid input: the axis does not lie in the plane`. The axis must lie on the plane: build it from two points on the plane, or where the plane meets another.
- **An angle in degrees.** `angled` takes radians: write `30f64.to_radians()`.
- **A plane on something other than a flat face.** An edge or a corner gives `invalid input: face_plane needs a face, got a Edge sub-shape` (or `Vertex`): the call is `work_plane_on_face`, though the message names the `face_plane` check it shares. A curved face gives `invalid input: face 6 is not planar`.
- **An axis on a corner, a curved edge or a face that isn't round.** `invalid input: work_axis_on needs an edge or a face, got a vertex`, or

`Edge 13 is not a straight edge or a cylindrical face`. Pick a straight edge, or a round face such as a boss, a pin or a hole.

- **A point on something other than a corner.** `invalid input: vertex_point needs a vertex, got a Edge sub-shape` (or `Face`).
- **A midplane of planes that cross.** `invalid input: the planes are not parallel: they have no midplane`. Take the planes of two opposite faces.
- **A tangent plane on a face that isn't round.** `invalid input: face 1 is not cylindrical`. Give the round side of a shaft, a boss or a hole.
- **A tangent side along the axis.** `invalid input: toward is parallel to the cylinder axis: it picks no side`. Point `toward` out from the axis, to the side you want.
- **A midpoint of something other than an edge.** `invalid input: edge_midpoint needs an edge, got a Face sub-shape` (or `Vertex`).
- **A number from another part, or from before a change.** `invalid input: face index 14 is out of range (the shape has 14 faces)`, or a work feature on the wrong face. Find the face, edge or corner again in the part you are building on.

CHAPTER 16

# Pictures of parts and sketches

`swx-qa render` draws a part to a PNG picture file, and `swx-qa sketch` draws a sketch. They need no graphics card and no window, and the same input always gives the same picture, byte for byte. The project uses them to show its work in reviews; this manual's pictures come from them too.

## 16.1 Draw a STEP file

**YOU WILL** Draw any STEP file to a PNG, at the size you want.
**YOU NEED** `swx-qa` built with OpenCascade (Chapter 2), and a STEP file. `plate.step` from Chapter 3 works.

1. Run `render` with the STEP file, a size and an output name:

   TERMINAL

   ```
   target/release/swx-qa render --step plate.step --size 1600x1000 -o plate.png
   ```

   It answers `wrote plate.png (1600x1000, …)`, with the file size in bytes.

2. Open `plate.png`. The part fills the picture, seen from the same corner as every other render.

**RESULT** A PNG of the part, ready to share or to put in a document.

The renderer chooses how finely to break curved faces into flat pieces for you: the mesh stays within 0.1 % of the part's overall size of the true surface.

## 16.2 Choose the size

`--size` takes the width and height in pixels, written `W` `x` `H`, for example `1600x1000`. The default is `800x600`; the largest is `8192x8192`. For a picture you will print, allow about 80 pixels per centimeter (200 per inch) of printed width: a picture printed 10 cm wide needs about 800 pixels.

## 16.3 Choose which edges are outlined

The renderer outlines the edges where faces meet at a sharp angle. `--crease` sets how sharp, in degrees, from 0 to 180; the default is 30. Where the surface bends by more than this angle from one face to the next, the edge gets a dark line. The edges of a box bend by 90°, so they are always outlined; the many small facets of a curved face bend by only a few degrees, so they are not. A smaller value outlines more edges; a larger value outlines fewer.

> **TIP**
>
> On a curved part that looks striped with lines, raise `--crease` (try 45). If an edge you want to see has no outline, lower it.

## 16.4 Draw a built-in sample

`--fixture` draws one of three built-in parts instead of a file: `cube` (10 mm), `cylinder` (5 mm radius, 10 mm tall) or `l-bracket` (30 mm legs, 8 mm thick, 20 mm deep). They need no OpenCascade, so they are a quick way to test the tool.

TERMINAL

```
target/release/swx-qa render --fixture l-bracket -o bracket.png
```

Section 19.1 lists every option.

## 16.5 Draw a sketch

`swx-qa sketch` solves a built-in sketch and draws it from above, with x to the right and y up. Lines, circles and arcs are dark blue; points and the ends of lines are red dots; each dimension's value is written in green beside what it measures.

| | |
|---|---|
| **YOU WILL** | Draw a built-in sketch to a PNG. |
| **YOU NEED** | `swx-qa` built (Chapter 2). OpenCascade is not needed. |
| **TIME** | 2 minutes. |

1. Run `sketch` with a sample name and an output name:

   TERMINAL

   ```
   target/release/swx-qa sketch --fixture rectangle -o rectangle.png
   ```

   It answers `wrote rectangle.png (800x600, …)`, with the file size in bytes.

2. Open `rectangle.png`. You should see a 40 × 25 mm rectangle, labeled `40` along the bottom and `25` up the right side.

3. Try the other sample, a right triangle with sides of 4 and 3:

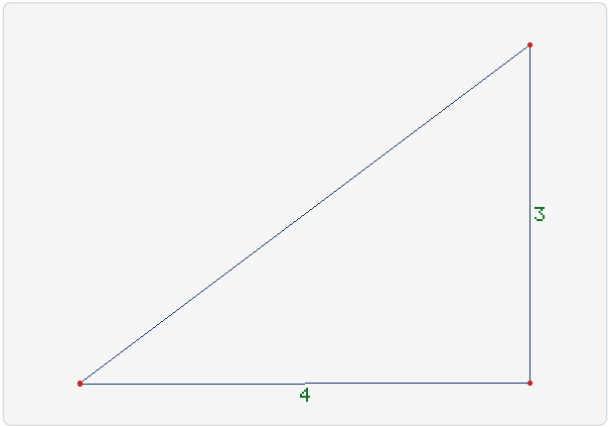
   TERMINAL

   ```
   target/release/swx-qa sketch --fixture triangle-345 -o triangle.png
   ```

**RESULT** A PNG of a solved sketch with its dimension values (Figure 16.1).




rectangle




triangle-345

**Figure 16.1** The two built-in sketches. The triangle's long side has no number: it has no dimension of its own, and comes out 5 because the other two sides are 4 and 3.

`--size` works as it does for parts; the default is `800x600`. The sketch is scaled to fill the picture with a margin, so a small sketch and a large one come out the same size. Labels are drawn at a fixed size in pixels, so they look smaller in a large picture.

**FOR DEVELOPERS**

To draw your own sketch, call `swx_qa::sketch::plot(&sketch, &PlotOptions::default())` and save the result with `to_png()`. It draws the sketch as it is, solved or not. The manual's sketch pictures come from `crates/swx-qa/examples/plate_sketch_plot.rs`, which does exactly this:

TERMINAL

```
cargo run -p swx-qa --example plate_sketch_plot -- <dir> 600x420
```

It writes `sketch-rough.png`, `sketch-shape.png` and `sketch-sized.png` in `<dir>`.

Only distances, radii and diameters are labeled. Other constraints, such as horizontal or fixed, don't show in the picture, and there are no dimension arrows yet.

CHAPTER 17

# Parameters and history

> **FOR DEVELOPERS**
>
> This chapter uses the document model, `swx-core`, and the sketcher, `swx-sketch`, from Rust. It needs no OpenCascade.

The document model holds what makes a part parametric: its named parameters and its ordered list of features (Section 4.8, Section 4.10). It doesn't make geometry itself. Its job is to keep the history valid and to work out what must be rebuilt after a change.

## 17.1 See what a change rebuilds

| | |
|---|---|
| **YOU WILL** | Build a four-feature history and ask which features a change affects. |
| **YOU NEED** | Saulitwerx built (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample:

   TERMINAL

   ```
   cargo run -p swx-core --example feature_history
   ```

   OUTPUT

   ```
   history:
     0  Sketch1
     1  Extrude1
     2  Sketch2
     3  Revolve1
   change thickness -> rebuild Extrude1
   edit Sketch2     -> rebuild Sketch2, Revolve1
   add Extrude2 with depth `height` -> error: unknown parameter `height`
   ```

2. Read the history. It is the table in Section 4.10: two sketches, an extrude that uses `thickness`, and a revolve that uses `sweep`.
3. Read the two rebuild lines. Changing `thickness` touches only Extrude1, the one feature that uses it. Editing Sketch2 rebuilds Sketch2 and Revolve1, which consumes it; Sketch1 and Extrude1 are left alone.
4. Read the last line. A feature may use only parameters that exist, so adding an extrude whose depth is `height` is refused.

**RESULT** You saw Saulitwerx decide, from the history alone, the smallest set of features a change needs rebuilt.

## 17.2 How the sample is built

The program is `crates/swx-core/examples/feature_history.rs`. A `Document` holds parameters and features:

```
let mut doc = Document::new();
doc.set_param("thickness", 5.0);
let plate_sketch = doc.add_feature("Sketch1", FeatureKind::Sketch)?;
doc.add_feature("Extrude1", FeatureKind::Extrude {
    profile: plate_sketch,
    depth: "thickness".into(),
})?;
```

Then two questions give the rebuild lists, in history order:

```
doc.dirty_after_param_change("thickness");   // [Extrude1]
doc.dirty_after_feature_edit(pin_sketch)?;    // [Sketch2, Revolve1]
```

The features that need rebuilding are called *dirty*. Today's feature kinds are `Sketch`, `Extrude` (with a depth parameter) and `Revolve` (with an angle parameter, in degrees). They are records in the history only; making their geometry comes with the part features on the roadmap.

## 17.3 Roll back and insert a feature

New features go at the end of the history. Sometimes a new feature belongs earlier: a boss that later features should build on, say, or a cut that should happen before the edges are rounded. Instead of rebuilding the part, you move the history's *end-of-part marker* up, above the features that should come after the new one. This is called *rolling back*. The features below the marker are kept but not used: the part is built as if they weren't there. While the history is rolled back, new features go in at the marker, and the marker moves down past each one. Move the marker back to the end, and the rolled-back features return, built on top of what you inserted.

The marker's position is a number: how many features are above it. In a history of four features, 4 means the end (nothing rolled back), and 2 means the last two are rolled back.

| | |
|---|---|
| **YOU WILL** | Roll a four-feature history back, insert two features above the rolled-back ones, and roll forward again. |
| **YOU NEED** | Saulitwerx built (Chapter 2). OpenCascade is not needed. |
| **TIME** | 5 minutes. |

1. Run the sample:

```
TERMINAL
cargo run -p swx-core --example roll_back
```

```
OUTPUT
history:
  0  Sketch1
  1  Extrude1
  2  Sketch2
  3  Revolve1
  -- end of part --
roll back to 2 -> rebuild nothing
rolled back:
  0  Sketch1
  1  Extrude1
  -- end of part --
  2  Sketch2
  3  Revolve1
change sweep -> rebuild nothing
after inserting Sketch3 and Extrude2:
  0  Sketch1
  1  Extrude1
  2  Sketch3
  3  Extrude2
  -- end of part --
  4  Sketch2
  5  Revolve1
add Extrude3 from Sketch2 -> error: feature 6 references
    feature 2 which is not earlier in the history
roll forward to 6 -> rebuild Sketch2, Revolve1
edit Sketch1 -> rebuild Sketch1, Extrude1
roll forward to 9 -> error: end-of-part position 9 is past the end of the history (6 features)
```

One long error line is wrapped here to fit the page.

2. Read the first history. It is the four features of Section 4.10, with the marker at the end.
3. Read `roll back to 2`. The marker now sits after Extrude1, and Sketch2 and Revolve1 are rolled back. Rolling back rebuilds nothing: the part simply stops at Extrude1.
4. Read `change sweep`. Revolve1 uses `sweep`, but while it is rolled back, changing `sweep` rebuilds nothing.
5. Read the third history. Sketch3 and Extrude2 went in at the marker, above Sketch2 and Revolve1, and the marker moved down past them.
6. Read the `Extrude3` line. A new feature can use only features above the marker, so an extrude of the rolled-back Sketch2 is refused, and the history is unchanged. The numbers in the message are the features' numbers, given when they were made (Section 19.13), not their places in the list: Sketch2 is feature 2, now in place 4.
7. Read `roll forward to 6`. With the marker back at the end, Sketch2 and Revolve1 return, and they must be rebuilt, now on top of Extrude2.
8. Read the last line. The history has six features, so the marker can't go to 9. It stays where it was.

**RESULT** Two features inserted in the middle of a history, without touching the features after them.

### How the sample is built

The program is `crates/swx-core/examples/roll_back.rs`. Two calls move the marker and read it:

```
doc.set_end_of_part(2)?;        // roll back: 2 features above the marker
doc.end_of_part();              // 2
doc.active_features();          // the features above the marker
let back = doc.history().len();
let rebuild = doc.set_end_of_part(back)?;   // [Sketch2, Revolve1]
```

`add_feature` needs no change: it always inserts at the marker. `history()` still lists every feature, rolled back or not; `active_features()` lists only those above the marker. `set_end_of_part` returns the features that came back and must be rebuilt, which is nothing when you roll back.

> **CAUTION** **Places and numbers**
>
> A feature's number never changes, but its place in the history does when a feature is inserted above it. To find a feature by its number, search the history for it, as the sample's `names` helper does; don't use the number as a place in the list.

## 17.4 Name and rename parameters

Every part keeps its parameters in one parameter table. Each row has a name, a value, a unit and an optional comment, and says where the parameter came from. A *model parameter* belongs to a dimension and gets the next free name in the series `d0`, `d1`, `d2`, … A *user parameter* is one you create yourself, with a name you choose, for a value you want to reuse or change in one place.

Names like `d0` are quick to make but say nothing. Renaming `d0` to `thickness` makes the part easier to read, and every feature that uses it follows the new name, so nothing breaks.

| | |
|---|---|
| **YOU WILL** | Build a small parameter table, rename a parameter a feature uses, and see which names are refused. |
| **YOU NEED** | Saulitwerx built (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample:

   TERMINAL

   ```
   cargo run -p swx-core --example parameter_table
   ```

   OUTPUT

   ```
   model parameters: d0, d1, d2
   after renaming d0 to thickness:
     name       kind   value  unit  comment
     thickness  model      5  mm
     d1         model     40  mm
     d2         model     10  mm
     clearance  user     0.2  mm    gap around the pin
   Extrude1 depth reads `thickness`
   next model parameter: d3
   add `2nd_hole` -> error: invalid parameter name `2nd_hole`: must start with a letter or `_`
   and contain only letters, digits and `_`
   rename d1 to thickness -> error: a parameter named `thickness` already exists
   ```

2. Read the first line. Three dimensions of a plate (its thickness, width and hole size) were given the model names `d0`, `d1` and `d2`, in the order they were made.

3. Read the table. `d0` is now called `thickness`; it is still a model parameter. `clearance` is a user parameter, with a comment saying what it is for. The rows stay in the order they were made.
4. Read the line about Extrude1. The extrude was made with depth `d0`. After the rename it reads `thickness`, with no change to the extrude itself.
5. Read `next model parameter: d3`. The name `d0` became free when it was renamed, but it is not handed out again. A model name always means one parameter, for the life of the part.
6. Read the last two lines. A name may not start with a digit, and two parameters may not share a name. In both cases the table is left as it was.

**RESULT** You have a parameter table with meaningful names, and a feature that kept working through a rename.

> **TIP** **Name what you will change**
>
> Rename the parameters you expect to edit later, such as a thickness or a hole size. Leave the rest as `d0`, `d1`, …

### How the sample is built

The program is `crates/swx-core/examples/parameter_table.rs`. Each call works on the document:

```
let depth = doc.add_model_param(5.0, "mm");        // "d0"
doc.add_user_param("clearance", 0.2, "mm")?;
doc.set_param_comment("clearance", "gap around the pin")?;
doc.rename_param(&depth, "thickness")?;
for p in doc.parameters() { /* name, kind, value, unit, comment */ }
```

The unit is free text, such as `"mm"` or `"deg"`, and may be empty; Saulitwerx doesn't check it yet. `set_param`, used in the history sample above, changes a parameter's value; given a new name, it makes a user parameter with no unit. Section 19.14 lists every call.

A parameter can also hold a formula instead of a plain number, so that it follows other parameters (Section 17.7).

## 17.5 Drive a sketch with parameters

A sketch dimension can be tied to a parameter, so the parameter table becomes the place where you change the part's sizes. Each tied dimension makes its own model parameter, `d0`, `d1`, … in the order you add them. Change the parameter, and the dimension follows the next time you apply the table and solve. The tie is called a *binding*.

| | |
|---|---|
| **YOU WILL** | Bind a rectangle's width and height to parameters, rename one, change it, and see the sketch follow. |
| **YOU NEED** | Saulitwerx built (Chapter 2). OpenCascade is not needed. |
| **TIME** | 5 minutes. |

1. Run the sample:

TERMINAL

```
cargo run -p swx-sketch --example driven_plate
```

OUTPUT

```
dimension 40 mm -> parameter d0
dimension 25 mm -> parameter d1
solved: 40.00 × 25.00 mm
renamed d0 to width (1 dimension follows)
width = 60: 60.00 × 25.00 mm
rename d1 without rename_bound -> error: a sketch dimension is bound to unknown parameter `d1`
height = 0 -> error: value must be > 0, got 0
sketch still 60.00 × 25.00 mm
```

2. Read the first two lines. The width dimension, 40 mm, made parameter `d0`; the height, 25 mm, made `d1`.
3. Read `renamed d0 to width`. The parameter got a meaningful name, and the sketch was told, so its width dimension now reads `width`.
4. Read `width = 60`. Setting `width` to 60 and applying the table made the rectangle 60 mm wide. The height stayed 25 mm.
5. Read the last three lines. Renaming `d1` in the table alone left the sketch looking for `d1`, which no longer exists. A height of 0 was refused, and the sketch kept its size.

**RESULT** A rectangle whose size you change from the parameter table.

### How the sample is built

The program is `crates/swx-sketch/examples/driven_plate.rs`. Instead of `add_constraint`, a dimension goes in with `bind_dimension`, which also adds its parameter:

```rust
use swx_sketch::params::{apply_params, bind_dimension, rename_bound};

let (_, name) = bind_dimension(&mut s, &mut doc, Constraint::Distance {
    a: PointRef::Start(bottom), b: PointRef::End(bottom), value: 40.0 })?;
// name == "d0"
doc.rename_param("d0", "width")?;
rename_bound(&mut s, "d0", "width");
doc.set_param("width", 60.0);
apply_params(&mut s, &doc)?;              // copy the values into the sketch
s.solve(SolveOptions::default())?;        // then move the geometry
```

**CAUTION** **Rename in both places**

The document and the sketch keep their own lists. `doc.rename_param` renames the parameter and the features that use it, but not the sketch's bindings. Call `rename_bound` with the same two names straight after, or `apply_params` fails with `a sketch dimension is bound to unknown parameter`.

**TIP**

Only distance, radius and diameter can be bound. A diameter's parameter holds the diameter, not the radius. `apply_params` checks every value before it changes anything, so one bad value leaves the whole sketch as it was.

## 17.6 Work out an expression

An expression lets one size depend on others: a hole that is always half the plate's width plus 5 mm is `d1 / 2 + 5 mm`. Saulitwerx reads the formula, looks up each name, and checks the units as it goes. Adding a length to an angle is an error, but a length times a length is an area.

| | |
|---|---|
| **YOU WILL** | Work out six expressions from a small parameter table, and see each error an expression can give. |
| **YOU NEED** | Saulitwerx built (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample:

   TERMINAL

   ```
   cargo run -p swx-core --example expressions
   ```

   The table holds `d0` = 5 mm, `d1` = 40 mm, `wall` = 2.5 mm and `angle` = 30 deg.

   OUTPUT

   ```
   d1 / 2 + 5 mm        = 25.00 mm
   (d1 - 2 * wall) / 3  = 11.67 mm
   2 in + d0            = 55.80 mm
   90 deg - angle       = 60.00 deg
   d1 * d0              = 200.00 mm²
   d1 / d0              = 8.00 (unitless)
   d1 + 5 deg
     error: dimension mismatch at byte 3: cannot apply `+` to length and angle
   5 mmm
     error: unknown unit `mmm` at byte 2 (expected mm, cm, m, in, ft, deg or rad)
   d9 * 2
     error: unknown name `d9` at byte 0
   d1 / (d0 - 5 mm)
     error: division by zero at byte 3
   d1 / (2
     error: syntax error at byte 7: expected `)`
   ```

2. Read the first line. Half of 40 mm is 20 mm, plus 5 mm is 25 mm. Multiplication and division come before addition and subtraction, as in school arithmetic; brackets change the order, as in the second line.
3. Read `2 in + d0`. Units can be mixed: 2 inches is 50.8 mm, plus 5 mm. Results are always worked out in millimeters and radians.
4. Read the next three lines. An angle minus an angle is an angle. A length times a length is an area, in mm². A length divided by a length is a plain number with no unit.
5. Read the errors. Each one says what is wrong and where, counting characters from 0: in `d1 + 5 deg`, byte 3 is the `+`.

**RESULT** You can work out a size from other sizes, in any of seven units, and trust that mixed-up units are caught.

## How the sample is built

The program is `crates/swx-core/examples/expressions.rs`. `parse` reads the text once; `eval` works it out, asking a function you supply for each name's value:

```rust
use swx_core::expr::{eval, parse, Quantity, Unit};

let resolve = |name: &str| -> Option<Quantity> {
    let p = doc.parameters().iter().find(|p| p.name == name)?;
    let unit = Unit::from_suffix(&p.unit)?;
    Some(Quantity::new(p.value * unit.factor(), unit.dim()))
};
let q = eval(&parse("d1 / 2 + 5 mm")?, &resolve, Unit::Mm)?;
// q.value == 25.0 (mm), q.dim == Dim::LENGTH
```

The parameter table stores a unit as plain text, so the sample turns it into a `Unit` itself. Section 19.15 lists the rules and every unit.

> **TIP** **Give every number its unit**
>
> `d1 + 5` is an error, because `d1` is a length and `5` is a bare number. Write `d1 + 5 mm`. A bare number is right only where it scales something, as in `d1 / 2`.

## 17.7 Make one size follow another

A parameter can hold an expression instead of a plain number. Then it is worked out from the parameters it names, and it changes whenever they do. Give a plate's hole the formula `d0 / 2 + 5 mm`, and when you make the plate wider, the hole grows with it. You change one number; the rest follow.

Saulitwerx works the formulas out in *dependency order*: every parameter after the ones its formula uses. It also refuses a formula that would make a loop, such as `a` from `b` while `b` comes from `a`, because a loop has no answer.

| | |
|---|---|
| **YOU WILL** | Give two parameters formulas, change and rename the one they depend on, and see which formulas are refused. |
| **YOU NEED** | Saulitwerx built (Chapter 2). |
| **TIME** | 5 minutes. |

1. Run the sample:

TERMINAL

```
cargo run -p swx-core --example parameter_formulas
```

The table starts with `d0` = 40 mm, the plate's width, and `wall` = 2.5 mm. `d1` gets the formula `d0 / 2 + 5 mm` and `d2` gets `(d0 - 2 * wall) / 3`.

OUTPUT

```
  name   value  unit  formula
  d0     40.00  mm
  d1     25.00  mm    d0 / 2 + 5 mm
  d2     11.67  mm    (d0 - 2 * wall) / 3
  wall    2.50  mm
worked out in order: d0, d1, wall, d2
change d0 -> rebuild Extrude1
after d0 = 60:
  name   value  unit  formula
  d0     60.00  mm
  d1     35.00  mm    d0 / 2 + 5 mm
  d2     18.33  mm    (d0 - 2 * wall) / 3
  wall    2.50  mm
after renaming d0 to width:
  name   value  unit  formula
  width  60.00  mm
  d1     35.00  mm    width / 2 + 5 mm
  d2     18.33  mm    (width - 2 * wall) / 3
  wall    2.50  mm
width = d1 - 5 mm -> error: parameter expressions form a cycle: width -> d1 -> width
d1 = width * wall -> error: expression for `d1` gives length^2, but its unit expects length
d1 = height / 2 -> error: expression for `d1`: unknown name `height` at byte 0
after d1 = 30 (typed as a plain number):
  name   value  unit  formula
```

```
width  60.00  mm
d1     30.00  mm
d2     18.33  mm    (width - 2 * wall) / 3
wall    2.50  mm
```

2. Read the first table. Each parameter with a formula shows it, and its value is the formula's result: 40 / 2 + 5 = 25 mm, and (40 − 5) / 3 = 11.67 mm.
3. Read `worked out in order`. `d2` needs `wall`, which was made after it, so `d2` is worked out last. Parameters that need nothing keep the order they were made in.
4. Read `change d0 -> rebuild Extrude1`. The sample's one extrude has depth `d1`, and `d1` comes from `d0`, so changing `d0` rebuilds the extrude, though the extrude never names `d0` (Section 4.10).
5. Read the table after `d0 = 60`. `d1` and `d2` followed on their own: 35 mm and 18.33 mm.
6. Read the table after the rename. The formulas now say `width`, and every value is the same.
7. Read the three errors. `width` from `d1` would make a loop, so it is refused, and the message traces it. `width * wall` is an area, which can't go in a length. `height` doesn't exist. Each time the table is left exactly as it was.
8. Read the last table. Typing a plain number into `d1` replaced its formula. `d1` is 30 mm from now on, and no longer follows `width`.

**RESULT** A parameter table where one change updates every size that depends on it.

### How the sample is built

The program is `crates/swx-core/examples/parameter_formulas.rs`. `set_param_expr` stores a formula and works out the whole table:

```rust
let width = doc.add_model_param(40.0, "mm");      // d0
let hole = doc.add_model_param(0.0, "mm");        // d1
doc.add_user_param("wall", 2.5, "mm")?;
doc.set_param_expr(&hole, "d0 / 2 + 5 mm")?;    // d1 is now 25
doc.set_param(&width, 60.0);                      // d1 follows: 35
doc.evaluation_order();                           // names, dependencies first
```

Each `Parameter` has an `expr` field holding the formula as typed, or `None` for a plain number. The result is stored in the parameter's own unit: a formula giving 1 inch, in a parameter whose unit is `mm`, stores 25.4. A bare number, such as `5`, is taken in the parameter's unit. Section 19.14 lists every call.

> **CAUTION** **A plain number replaces the formula**
>
> `set_param` stores a number and drops any formula the parameter had. To change how a parameter is worked out, give it a new formula with `set_param_expr` instead.

> **TIP** **When a change can't be worked out**
>
> If a new value would break a formula that depends on it, such as a count of 0 under `10 mm / n`, the value is still stored, but the formulas that depend on it keep their old results without an error. Check the values after such a change.

## 17.8 Common mistakes

- **Using a parameter before it exists.** Call `set_param` first; otherwise `add_feature` fails with `unknown parameter`.
- **Naming a parameter badly.** A name starts with a letter or `_` and holds only letters, digits and `_`: `hole_2` works, `2nd_hole` and `hole-2` don't. Names are case-sensitive.
- **Reusing a name.** Every parameter needs its own name, whether it is a model or a user parameter. To swap two names, rename one of them to a spare name first.
- **Renaming a bound parameter in one place only.** Call `rename_bound` after `rename_param` (Section 17.5).
- **Forgetting to apply and solve.** Changing a parameter changes the table only. Call `apply_params`, then `solve`, to move the sketch.
- **A formula that loops.** A parameter can't depend on itself, directly or through others. Break the loop by making one of them a plain number.
- **Replacing a formula by accident.** `set_param` drops the formula (Section 17.7).
- **Using a feature that comes later.** A feature may use only features already in the history and above the end-of-part marker. Otherwise `add_feature` fails with `feature N references feature M which is not earlier in the history`. If `M` is rolled back, roll forward first, or insert the new feature after it.
- **Using a feature's number as its place.** Once a feature has been inserted above others, numbers and places differ (Section 17.3). Search `history()` for the number.
- **Moving the marker past the end.** `end-of-part position 9 is past the end of the history (6 features)`. The end is `history().len()`.

CHAPTER 18

# Saulitwerx document files

> **FOR DEVELOPERS**
>
> This chapter calls `swx-io`, the crate that reads and writes Saulitwerx's own files, from Rust. It needs no OpenCascade.

A STEP file (Chapter 6) keeps only the finished shape. Open it again and you get a solid you can't edit: the sketches, dimensions, parameters and history that made it are gone. To come back to a design and change it, you need a file that keeps all of that. Saulitwerx has its own file for this, the *document file*, and its layout is published in full, so any program may read or write it.

There are three kinds of document, told apart by the end of their name:

| Kind | Name ends in | Holds |
|---|---|---|
| Part | `.sltwx` | One part: its sketches, features, parameters and history. |
| Assembly | `.sltwxa` | Parts put together (later milestones). |
| Drawing | `.sltwxd` | 2D drawings of parts and assemblies (later milestones). |

A document is a set of small text files in *JSON*, a plain-text way of writing down named values that people and programs can both read:

- **`manifest.json`** (*manifest*) says what the document is: its kind, the format version that wrote it, its id, its units, and which program made and last saved it.
- **`model.json`** holds the design itself, and is what Saulitwerx rebuilds the part from.
- **`properties.json`**, if present, holds facts about the document such as a part number or a material.

The files can live in one folder, the *unpacked form*, named like the document (`plate.sltwx/`). Each file is written the same way every time, with its names in alphabetical order and one value per line, so when you keep a design under version control a change shows up as a change to just the lines it touched.

**In Saulitwerx today:** `swx-io` reads and writes the unpacked form and checks the manifest. `model.json` and `properties.json` are read and written as plain JSON objects: Saulitwerx can't yet fill `model.json` from a part's history or build a part from it, so a part can't be saved and opened again yet. The single-file form (one zipped file), the stored pictures and previews, and the `sltwx` command that converts between the forms are planned (**Coming next**).

## 18.1 Write and read a document folder

| | |
|---|---|
| YOU WILL | Write a part document as a folder, read it back, and see the errors. |
| YOU NEED | Saulitwerx built from source (Chapter 2). OpenCascade isn't needed. |
| TIME | 5 minutes. |

1. Run the sample program, and tell it to write into a folder called `files`:

   TERMINAL

   ```
   cargo run -p swx-io --example unpacked_document -- files
   ```

   It prints what it wrote and read, the manifest, then four errors on purpose:

   OUTPUT

   ```
   wrote files/plate.sltwx
   files: manifest.json, model.json, properties.json
   read back: part 0.1.0 0b6d3a52-4c1e-4f7a-9e21-5d8c7f3b2a10, units mm / deg
       / kg
   same as written: true
   manifest.json:
   {
     "created_by": "saulitwerx 0.0.1",
     "document_id": "0b6d3a52-4c1e-4f7a-9e21-5d8c7f3b2a10",
     "format": "sltwx",
     "format_version": "0.1.0",
     "kind": "part",
     "saved_by": "saulitwerx 0.0.1",
     "units": {
       "angle": "deg",
       "length": "mm",
       "mass": "kg"
     }
   }
   wrong extension -> error: folder extension `.sltwxa` does not match the
       manifest kind `part` (expected `.sltwx`)
   missing folder -> error: files/missing.sltwx: No such file or directory
       (os error 2)
   version 1.0.0 -> error: unsupported format_version "1.0.0": only major
       version 0 is supported
   bad model -> error: invalid JSON: model.json: expected value at line 1
       column 17
   ```

   Five long lines are wrapped here to fit the page.

2. Read the first lines. The program wrote a folder `files/plate.sltwx` holding the three files, read it back, and got exactly what it wrote.

3. Read the manifest. Its names are in alphabetical order, nested values are indented by two spaces, and it ends with one empty line. It has no `model_hash`, because the program didn't give one.

4. Look at the model:

```
TERMINAL
cat files/plate.sltwx/model.json
```

The program built it with `name` before `expr`; the file has them the other way round, in alphabetical order:

```
OUTPUT
{
  "parameters": [
    {
      "expr": "10 mm",
      "name": "thickness"
    }
  ]
}
```

5. Read the errors. A part's folder must end in `.sltwx`, and nothing was written under the wrong name. A folder that isn't there can't be read. A manifest from a future major version is refused rather than misread. A model that isn't valid JSON is refused, with the file and the place named.

**RESULT** A part document on disk, read back unchanged, in files a person can read.

### How the sample is built

The program is `crates/swx-io/examples/unpacked_document.rs`. It does three things.

1. **Fill in the manifest.** Every field is required except `model_hash`; `extra` holds fields this version doesn't know (see the tip below):

```rust
let manifest = Manifest {
    format: "sltwx".into(),
    format_version: "0.1.0".into(),
    kind: DocumentKind::Part,
    document_id: "0b6d3a52-4c1e-4f7a-9e21-5d8c7f3b2a10".into(),
    units: Units { length: "mm".into(), angle: "deg".into(),
                   mass: "kg".into(), extra: BTreeMap::new() },
    created_by: "saulitwerx 0.0.1".into(),
    saved_by: "saulitwerx 0.0.1".into(),
    model_hash: None,
    extra: BTreeMap::new(),
};
```

2. **Make the document.** A `Document` is the manifest, the model and the optional properties, the last two as JSON objects:

```rust
let doc = Document {
    manifest,
    model: json!({ "parameters": [{ "name": "thickness",
                                    "expr": "10 mm" }] }),
    properties: Some(json!({ "part_number": "PL-001" })),
};
```

3. **Write it, and read it back.** The folder is made if it's missing:

```rust
write_unpacked(&folder, &doc)?;
let back = read_unpacked(&folder)?;
```

> **TIP** **Newer files keep their extra fields**
>
> A file written by a newer Saulitwerx may hold fields this one doesn't know. Reading keeps them in the manifest's `extra` (and in `units.extra`), and writing puts them back unchanged, so opening and saving a newer file loses nothing. Only a new major version (1.0.0 and up) is refused.

> **NOTE** **The document id**
>
> `document_id` is a *UUID*: 32 hexadecimal digits in groups of 8, 4, 4, 4 and 12, like the sample's. Give each new document a new one and never change it: assemblies will use it to find their parts again after a file is moved or renamed.

## 18.2 What is read and written

- **Reading** (`read_unpacked`) needs `manifest.json` and `model.json`; `properties.json` is optional. Anything else in the folder is left alone. The manifest is checked first, then the folder's name against its kind, then the other two files, which must each be a JSON object.
- **Writing** (`write_unpacked`) checks the name and the two objects before it writes anything, so it never writes a document it would refuse to read back. It writes the three files, deletes `properties.json` when the document has no properties, and leaves everything else in the folder as it was.
- **One file at a time.** `read_manifest` checks the text of a `manifest.json` on its own, and `write_manifest` writes one. `to_canonical_json` writes any JSON value the way every document file is written.

The manifest's checks, and the rest of the rules, are in Section 19.17.

## 18.3 Common mistakes

- **A folder name that doesn't match the kind.** `folder extension `.sltwxa` does not match the manifest kind `part` (expected `.sltwx`)`, or `folder has no extension; the manifest kind `part` needs `.sltwx``. The manifest decides what a document is; name the folder to match.
- **A missing file.** `missing required file `model.json`` (or `manifest.json`). Both are required, even for an empty model: write `{}`.
- **A model that isn't an object.** `invalid field `model.json`: must be a JSON object`. The model and the properties must each be a JSON object, in braces, not a list or a single value.
- **A broken file.** `invalid JSON: model.json: expected value at line 1 column 17`. The message names the file and the place where it stopped making sense.
- **A malformed id or hash.** `invalid field `document_id`: must be a UUID in 8-4-4-4-12 hex form, got "plate-1"`. A `model_hash` must be `sha256:` and 64 lowercase hexadecimal digits, or left out.
- **A file from a future version.** `unsupported format_version "1.0.0": only major version 0 is supported`. This Saulitwerx can't read it; use a newer one.

PART IV

# Reference

Every command, option, operation and setting that exists today, with its default and its allowed values.

CHAPTER 19

# Reference

## 19.1 `swx-qa render`

Draws a part as an isometric, flat-shaded PNG with its sharp edges outlined.

FORM

```
swx-qa render (--fixture <name> | --step <file.step>) -o <out.png> [options]
```

| Option | What it does | Default |
|---|---|---|
| `--fixture <name>` | Draws a built-in part: `cube`, `cylinder` or `l-bracket`. Give this or `--step`, not both. | — |
| `--step <file>` | Draws a STEP file. Needs a build with `--features occt`. | — |
| `-o`, `--output <file>` | The PNG to write. Required. An existing file is replaced. | — |
| `--size <W>x<H>` | Picture size in pixels. Each side from 1 to 8192. | `800x600` |
| `--crease <deg>` | Outline edges where the surface bends by more than this many degrees. 0 to 180. | `30` |
| `-h`, `--help` | Prints the help text and does nothing else. | — |

On success it prints `wrote <file> (<W>x<H>, <bytes> bytes)` and exits with status 0. On an error it prints `swx-qa:` and the message (Section 20.2) and exits with status 1, without writing a file.

| Built-in part | Shape |
|---|---|
| `cube` | A 10 mm cube with one corner at the origin. |
| `cylinder` | Radius 5 mm, height 10 mm, standing on +Z; 32 flat segments around. |
| `l-bracket` | An L profile with 30 mm legs and 8 mm thickness, 20 mm deep. |

## 19.2 `swx-qa sketch`

Solves a built-in sketch and draws it from above as a PNG, with its dimension values.

FORM

```
swx-qa sketch --fixture <name> -o <out.png> [--size <W>x<H>]
```

| Option | What it does | Default |
|---|---|---|
| `--fixture <name>` | The built-in sketch to draw: `rectangle` or `triangle-345`. Required. | — |
| `-o`, `--output <file>` | The PNG to write. Required. An existing file is replaced. | — |
| `--size <W>x<H>` | Picture size in pixels. Each side from 1 to 8192. | `800x600` |
| `-h`, `--help` | Prints the help text and does nothing else. | — |

Success and errors work as for `render`. `--step` and `--crease` belong to `render` only.

| Built-in sketch | Shape |
|---|---|
| `rectangle` | 40 × 25 mm, lower-left corner fixed at the origin; sides horizontal and vertical; bottom and right dimensioned. |
| `triangle-345` | A right triangle with its right angle fixed at the origin; a horizontal side of 4 and a vertical side of 3, both dimensioned. The long side comes out 5. |

| In the picture | Drawn as |
|---|---|
| Lines, circles, arcs | Dark blue lines, 1 pixel wide. |
| Points, line ends | Red dots, 5 pixels across. |
| Distance | Its value, in green, beside the middle of the two points, on the side away from the sketch's center. |
| Radius, diameter | Its value with `R` or `Ø` in front, outside the circle at the upper right, or outside an arc at its middle. |
| Values | Up to 3 decimals, with trailing zeros dropped: `40`, `12.5`, `R2.125`. |

## 19.3 Build options

| Command | What it builds |
|---|---|
| `cargo test --workspace` | Everything without OpenCascade, and runs every test. |
| `cargo build --release -p swx-qa --features occt` | The `swx-qa` tool with STEP input, at `target/release/swx-qa`. |
| `cargo run -p swx-sketch --example plate_sketch` | Runs the sample sketch (Chapter 5). |
| `cargo run -p swx-sketch --example sketch_dof` | Runs the degree-of-freedom sample (Section 5.3). |
| `cargo run -p swx-sketch --example centered_plate -- [<width> <height>]` | Runs the centered-hole sample (Section 5.5). Defaults: 40 and 25. |
| `cargo run -p swx-sketch --example slot_sketch` | Runs the slot sample (Section 5.6). |
| `cargo run -p swx-qa --example plate_sketch_plot -- <dir> [<W>x<H>]` | Draws the sample sketch in three stages, as PNGs in `<dir>` (Section 16.5). |
| `cargo run -p swx-qa --example constraint_plots -- <dir> [<W>x<H>]` | Draws the centered-plate and slot sketches as PNGs in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example plate_with_hole -- <file.step>` | Builds the plate and saves it (Chapter 6). The file defaults to `plate.step`. |
| `cargo run -p swx-kernel --features occt --example part_topology` | Counts the plate's faces, edges and vertices and lists their neighbours (Section 6.3). |
| `cargo run -p swx-kernel --features occt --example plate_stages -- <dir>` | Saves the block, the cylinder and the plate as three STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example extrude_plate -- <file.step>` | Extrudes the plate from its outline (Section 7.1). |
| `cargo run -p swx-kernel --features occt --example extrude_slot -- <file.step>` | Extrudes a slot three ways (Section 7.2). |
| `cargo run -p swx-kernel --features occt --example extrude_modes -- <dir>` | Joins, cuts and intersects extrudes with a block (Section 7.3). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example extrude_through_all -- <dir>` | Cuts two holes through a block (Section 7.4). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example extrude_history` | Extrudes the plate with history and prints which face each piece of the outline became (Section 7.5). |

| Command | What it builds |
|---|---|
| `cargo run -p swx-kernel --features occt --example extrude_to_face -- <dir>` | Extrudes up to and between the faces of a stepped block (Section 7.6). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example revolve_bushing -- <dir>` | Revolves a bushing and cuts a groove in it (Chapter 8). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example fillet_chamfer -- <dir>` | Fillets and chamfers the edges of a block (Chapter 9). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example chamfer_sizes -- <dir>` | Chamfers one edge of a block four ways (Section 9.2). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example boolean_history -- <out.step>` | Cuts a groove with history and prints what became of each face (Section 6.4). |
| `cargo run -p swx-kernel --features occt --example shell_parts -- <dir>` | Shells a block into a tray and a frame, and a cylinder into a cup (Chapter 10). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example fillet_variable -- <dir>` | Rounds one edge of a block from 2 mm to 8 mm (Section 9.3). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example hole_plate -- <dir>` | Drills four holes in a plate (Chapter 11). A STEP file in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example hole_screw -- <dir>` | Drills a countersunk hole and two tapped holes in a plate (Section 11.2). A STEP file in `<dir>`. |
| `cargo run -p swx-kernel --example thread_table` | Prints the metric coarse thread table and looks up sizes by name (Section 11.2). Needs no OpenCascade. |
| `cargo run -p swx-kernel --features occt --example draft_block -- <dir>` | Drafts the sides of a block three ways (Chapter 12). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example loft_sections -- <dir>` | Lofts a square into a circle, and a vase through three circles, ruled and smooth (Chapter 13). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example sweep_pipe -- <dir>` | Sweeps a circle and a rectangle along a bent path, and shows the errors, a too-tight bend among them (Chapter 14). STEP files in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example helical_spring -- <dir>` | Winds a circle into a coil spring and a rectangle into a square ridge around a helix, and shows the errors (Section 14.2). STEP files in `<dir>`. |
| `cargo run -p swx-io --example unpacked_document -- <dir>` | Writes a part document folder `<dir>/plate.sltwx`, reads it back, and shows the errors (Section 18.1). Needs no OpenCascade. |
| `cargo run -p swx-kernel --example work_features` | Builds work planes, axes and a point, and prints them (Section 15.1). Needs no OpenCascade. |
| `cargo run -p swx-kernel --features occt --example tilted_post -- <dir>` | Stands a post at 30° on a plate, from a tilted work plane (Section 15.2). A STEP file in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example work_on_part -- <dir>` | Takes work features from a block's faces, edges and a corner, then uses two of them (Section 15.3). A STEP file in `<dir>`. |

| Command | What it builds |
|---|---|
| `cargo run -p swx-kernel --features occt --example work_midplane -- <dir>` | Takes a midplane, a tangent plane and an edge midpoint from a shaft, and drills and cuts on them (Section 15.4). A STEP file in `<dir>`. |
| `cargo run -p swx-kernel --features occt --example multi_body -- <dir>` | Drills two holes in one combine call with history, then splits the plate into two bodies (Section 6.5). A STEP file in `<dir>`. |
| `cargo run -p swx-core --example feature_history` | Runs the history sample (Chapter 17). |
| `cargo run -p swx-core --example roll_back` | Runs the roll-back sample (Section 17.3). |
| `cargo run -p swx-core --example parameter_table` | Runs the parameter-table sample (Section 17.4). |
| `cargo run -p swx-sketch --example driven_plate` | Runs the bound-dimension sample (Section 17.5). |
| `cargo run -p swx-core --example expressions` | Runs the expression sample (Section 17.6). |
| `cargo run -p swx-core --example parameter_formulas` | Runs the stored-formula sample (Section 17.7). |
| `cargo run -p swx-sketch --example inferred_sketch` | Runs the inference sample (Section 5.8). |
| `cargo run -p swx-qa --example inference_plots -- <dir> [<W>x<H>]` | Draws the inference sketch as drawn and after inference, as PNGs in `<dir>`. |

The `occt` feature switches on OpenCascade. It is off by default, so a plain build needs only Rust.

## 19.4 Geometry kernel operations

| Operation | What it does and what it accepts |
|---|---|
| `make_box(corner, size)` | A box with its lowest corner at `corner` and sides `size` = [x, y, z]. Sizes > 0. |
| `make_cylinder(base, r, h)` | A cylinder along +Z with its base circle centered at `base`. Radius and height > 0. |
| `make_sphere(center, r)` | A sphere. Radius > 0. |
| `union(a, b)`, `cut(a, b)`, `intersect(a, b)` | Boolean operations (Section 4.9). Inputs unchanged. |
| `boolean_tracked(op, a, b)` | The same solid as `union`, `cut` or `intersect` (by `op`), with an `OpHistory` (Section 6.4). Same errors as the plain call. |
| `combine(op, target, tools)` | Applies `op` to `target` with each tool in `tools` in turn, the same as calling the Boolean once per tool (Section 6.5). At least one tool; the target and every tool must hold a solid body (not a lone face). A result with no volume is `EmptyResult`. Inputs unchanged. |
| `combine_tracked(op, target, tools)` | The same solid as `combine`, with a `CombineHistory`. Same errors. |
| `body_count(s)` | The number of separate solid bodies in `s`. 0 for a lone face. |
| `bodies(s)` | Each solid body of `s` as its own shape, ordered by the lowest corner of its bounding box (x, then y, then z). Empty for a lone face. `s` unchanged. |
| `volume(s)`, `surface_area(s)` | In $mm^3$ and $mm^2$. |
| `bounding_box(s)` | The smallest box, aligned with the axes, that holds the solid: its `min` and `max` corners. |
| `face_count(s)` | The number of faces. |
| `sub_shape_count(s, kind)` | The number of faces, edges or vertices (`kind`), each shared one counted once. Their indices run from 0 to the count minus 1 (Section 6.3). |
| `adjacent(s, of, kind)` | The sub-shapes of `kind` that touch `of`, sorted by index, without repeats: going down, the edges or vertices of a face, or the vertices of an edge; going up, the |

| Operation | What it does and what it accepts |
|---|---|
| | faces or edges that contain `of`. `kind` must differ from `of.kind`, and `of.index` must be in range. |
| `sub_shape_bounding_box(s, of)` | The smallest axis-aligned box around one face, edge or vertex `of` of `s`, as for `bounding_box`. `of.index` must be in range. |
| `face_plane(s, face)` | The plane of the flat face `face` of `s`: a point on it and its unit outward normal, pointing out of the solid (Section 7.6). `face` must be a face, in range, and flat. |
| `fillet(s, edges, radius)` | Rounds every edge in `edges` with one radius (Chapter 9). Radius > 0; the list not empty, edges only, each in range; an edge listed twice counts once. A radius that doesn't fit fails, and `s` is unchanged. |
| `fillet_variable(s, edge, start, r_start, r_end)` | Rounds one `edge` with a radius that changes evenly from `r_start` at the vertex `start` to `r_end` at the edge's other end (Section 9.3). Both radii > 0; `edge` an edge and `start` a vertex, each in range; `start` one of the edge's two vertices. The larger radius must fit as a constant fillet would, and the edge must not run smoothly into another; otherwise it fails, and `s` is unchanged. |
| `chamfer(s, edges, distance)` | Bevels every edge in `edges`, set back `distance` along both of its faces. Same rules as `fillet`. |
| `chamfer_asymmetric(s, edges, size)` | Bevels every edge in `edges`, a list of `EdgeOnFace`, by one `ChamferSize` measured from each edge's reference face (Section 9.2). The list not empty; each edge an edge and each face a face, in range; each face one of the two that bound its edge; an edge listed once, or twice with the same face. A size that doesn't fit fails, and `s` is unchanged. |
| `shell(s, open, thickness)` | Hollows `s` into walls of one `thickness` (mm, > 0), grown inward so the outside size is unchanged, with the faces in `open` removed as openings (Chapter 10). The list not empty, faces only, each in range; a face listed twice counts once. A thickness that doesn't fit fails, and `s` is unchanged. |
| `hole(s, at, kind, depth)` | Drills a hole of `kind` and `depth` into `s` (Chapter 11). `at` is an `Axis`: its `origin` the hole's center on the start face, its `dir` pointing into the material. Depths are measured from `at.origin` along `dir`, to a flat bottom. A hole that removes nothing is refused; one that removes everything is `EmptyResult`. `s` is unchanged. |
| `draft(s, faces, neutral, angle)` | Tilts every face in `faces` by `angle` radians about the line where it crosses the neutral plane (Chapter 12). `neutral` is an `Axis`: a point on the neutral plane, and the pull direction, which is the plane's normal. A positive angle makes the solid smaller along the pull direction. The angle not zero and less than π/2 either way; the list not empty, faces only, each in range; a face listed twice counts once. A face the engine can't draft, such as one parallel to the neutral plane, fails, and `s` is unchanged. |
| `loft(sections, ruled)` | A new solid through the flat `sections`, in the order given, capped with the first and last (Chapter 13). Each section a single flat face from `make_face`, without holes, on any plane; at least `MIN_LOFT_SECTIONS` (2). `ruled` true: straight surfaces between neighbors; false: one smooth surface through all. Sections the engine can't loft, such as two in the same place, fail. The sections are unchanged. |
| `sweep(profile, plane, path)` | A new solid made by moving the flat `profile` along the open `path`, a list of `Segment2` lines and arcs drawn on `plane` (Chapter 14). The path runs from the first piece's start; each next piece must meet the end of the one before within `LOOP_GAP_TOLERANCE`, either way round. The path's start must lie on the profile's plane. The profile a single flat face without holes; no circles in the path. A sweep that encloses nothing is `EmptyResult`. Every arc of the path must have a radius larger than the profile's farthest point from the path's start, measured in the profile's plane; a tighter bend is refused. The profile is unchanged. |

| Operation | What it does and what it accepts |
|---|---|
| `helical_sweep(profile, axis, pitch, turns)` | A new solid made by moving the flat `profile` along a helix about `axis` (an `Axis`; its direction is normalized), rising `pitch` mm along the axis per turn, for `turns` turns (Section 14.2). Right-handed: counterclockwise seen from the tip of `axis.dir`. The helix starts at the profile, which stays in the plane through the axis as it turns. Pitch and turns finite and > 0 (turns needn't be whole). The profile a single flat face without holes; the axis in its plane (within `PERPENDICULAR_TOLERANCE` for the direction, `LOOP_GAP_TOLERANCE` for the point); the profile wholly on one side of the axis; its extent along the axis less than the pitch. Volume: turns × 2π × area × the distance of the profile's centroid from the axis, whatever the pitch. The profile is unchanged. |
| `tessellate(s, deflection)` | Breaks the surface into triangles, no farther than `deflection` (mm, > 0) from the true surface. |
| `export_step(s, path)` | Writes a STEP file in millimeters, replacing any existing file. |
| `import_step(path)` | Reads everything in a STEP file as one shape. |

All positions must be finite numbers; all sizes finite and greater than zero.

| Type | Fields and rules |
|---|---|
| `TopoKind` | `Face`, `Edge` or `Vertex`. |
| `SubShape { kind, index }` | One face, edge or vertex: the `index`-th of its `kind`, counted from 0. Valid only for the shape it came from; ask again after each operation. |
| `EdgeOnFace { edge, face }` | An edge, and the reference face an uneven chamfer measures its first size on: one of the two faces that meet at the edge. |
| `ChamferSize::TwoDistances { d1, d2 }` | `d1` on the reference face, `d2` on the other face. Both finite and > 0, in mm. |
| `ChamferSize::DistanceAngle { distance, angle }` | `distance` on the reference face, finite and > 0, in mm. `angle` between the reference face and the new face, in radians, strictly between 0 and π/2. |
| `HoleKind::Simple { diameter }` | A plain round hole. Diameter > 0, in mm. |
| `HoleKind::Counterbore { diameter, cb_diameter, cb_depth }` | A hole with a wider, flat-bottomed recess at its mouth. All > 0, in mm; `cb_diameter` larger than `diameter`. |
| `HoleKind::Countersink { diameter, csk_diameter, csk_angle }` | A hole with a cone-shaped recess at its mouth, from `csk_diameter` at the start point down to `diameter`. Diameters > 0, in mm; `csk_diameter` larger than `diameter`. `csk_angle` is the angle across the cone, in radians, strictly between 0 and π (π/2 is 90°). The cone is ( `csk_diameter` − `diameter` ) ÷ 2 ÷ tan( `csk_angle` ÷ 2) deep. |
| `HoleKind::Tapped { thread }` | A hole for a screw thread, drilled at the thread's tap-drill diameter. The thread itself isn't modeled. `thread` is a `MetricThread` (Section 19.5). |
| `HoleDepth::Blind(depth)` | Stops `depth` mm from the start point, with a flat bottom. Depth > 0, and larger than a counterbore's `cb_depth` or a countersink's cone depth. |
| `HoleDepth::ThroughAll` | Through the whole solid, along the direction from the start point. |
| `BooleanOp` | `Union`, `Cut` or `Intersect`. |
| `CombineHistory { target, tools }` | `target`: for each face of the target, in index order, the result's faces it became. `tools`: the same for each tool, in order. Each list sorted: one face if it lives on, several if split, none if deleted. Faces only. |

| Type | Fields and rules |
|---|---|
| `OpHistory { modified, generated }` | `modified`: one entry `(operand, face, images)` for each face of `a` ( `Operand::A` ), then of `b` ( `Operand::B` ), in index order. `images` are the result's faces it became, sorted: one if it lives on (perhaps trimmed), several if split, none if deleted. `generated` is empty for a Boolean. Faces only. |

## 19.5 Thread sizes

`metric_coarse(name)` looks up a size by name, such as `"M6"`: an `M` (small or capital) and the size, with spaces around it ignored. It gives a `MetricThread { nominal, pitch }`, in mm. `designation()` gives its name back, `minor_diameter()` the inside of the thread's ridges (the diameter minus 1.082532 × the pitch, from the ISO basic profile), and `tap_drill_diameter()` the diameter minus the pitch. `METRIC_COARSE` is the whole table, smallest first. Pitches are the ISO metric coarse series (ISO 261).

| Size | Pitch | Minor | Tap drill | Size | Pitch | Minor | Tap drill |
|---|---|---|---|---|---|---|---|
| M2 | 0.40 | 1.57 | 1.60 | M6 | 1.00 | 4.92 | 5.00 |
| M2.5 | 0.45 | 2.01 | 2.05 | M8 | 1.25 | 6.65 | 6.75 |
| M3 | 0.50 | 2.46 | 2.50 | M10 | 1.50 | 8.38 | 8.50 |
| M4 | 0.70 | 3.24 | 3.30 | M12 | 1.75 | 10.11 | 10.25 |
| M5 | 0.80 | 4.13 | 4.20 | M16 | 2.00 | 13.83 | 14.00 |
| | | | | M20 | 2.50 | 17.29 | 17.50 |
| | | | | M24 | 3.00 | 20.75 | 21.00 |

All sizes in mm, as printed by `thread_table` (Section 11.2). Any other name is an error.

## 19.6 Profiles, extrude and revolve

`make_face(plane, outer, holes)` makes a flat face on `plane`, bounded by the `outer` loop, with each loop in `holes` cut out. `extrude(face, extent)` sweeps that face along the plane's normal into a new solid (Chapter 7).

| Type | Fields and rules |
|---|---|
| `Workplane` | `origin` (a point), `x_dir` (the plane's left–right direction) and `normal` (straight out of the plane). Both directions non-zero and at right angles; their length doesn't matter. A 2D point (u, v) sits at `origin + u·x_dir + v·(normal × x_dir)`. |
| `Workplane::XY`, `::XZ`, `::YZ` | Built-in planes through the origin, with normals +Z, −Y and +X (Section 7.2). |
| `Segment2::Line { start, end }` | A straight piece. Its ends must be apart. |
| `Segment2::Arc { center, radius, start_angle, end_angle }` | A circular arc, counterclockwise from `start_angle` to `end_angle`, in radians from the +u axis. Radius > 0; the two angles must give different end points. |
| `Segment2::Circle { center, radius }` | A full circle. Radius > 0. It must be the only piece of its loop. |
| `ProfileLoop { segments }` | Pieces in chain order, each meeting the next and the last meeting the first. Any piece may run backwards, and the loop may go either way round. |
| `Extent::Distance(d)` | `d` mm along the normal; negative goes the other way. Not zero. |
| `Extent::Symmetric(l)` | `l` mm in total, half on each side of the plane. Not zero. |

| Type | Fields and rules |
|---|---|
| `Extent::ThroughAll(dir)` | Through the whole target body (Section 7.4). Only `extrude_feature` with `ExtrudeMode::Cut` accepts it; `extrude` refuses it. |
| `ThroughDirection::Normal`, `::Reversed`, `::Both` | Which way a through-all cut goes: along the normal, against it, or both ways. |
| `Extent::FromTo(start, end)` | From `start` mm to `end` mm along the normal, in either order. Both finite, and not equal. |
| `Extent::ToFace(face)` | From the workplane to the plane of `face`, a flat face of the target body parallel to the workplane, on either side of it (Section 7.6). Only `extrude_feature` with join, cut or intersect accepts it. |
| `Extent::BetweenFaces(a, b)` | From the plane of face `a` to the plane of face `b`, both flat faces of the target body parallel to the workplane; the workplane's own position doesn't matter. Accepted as `ToFace`. |
| `LOOP_GAP_TOLERANCE` | 1e-7 mm: the largest gap between two ends that still counts as joined. Ends that close are moved to meet exactly. |
| `PERPENDICULAR_TOLERANCE` | 1e-9: how far from a right angle `x_dir` and `normal` may be. |
| `PARALLEL_FACE_TOLERANCE` | 1e-9 radians: how far a face's normal may tilt from the workplane's normal, or its reverse, and still count as parallel. |
| `FACE_DISTANCE_TOLERANCE` | 1e-7 mm: two planes closer than this count as one, so the extrude between them has no length and is refused. |
| `REVOLVE_AXIS_TOLERANCE` | 1e-7 mm: how far a revolve's axis may be off the face's plane, and how far the face may reach past the axis, and still pass (Section 8.3). |

All numbers must be finite. A face is refused if a loop is empty, has no area, doesn't close, or crosses itself, or if a hole isn't wholly inside the outer loop. Every refusal is an error (Section 20.2), never a crash.

`make_face_tracked(plane, outer, holes)` and `extrude_tracked(face, extent)` do the same as `make_face` and `extrude`, and also say where each piece went (Section 7.5). They take the same inputs and give the same errors; `extrude_tracked` refuses `Extent::ThroughAll`, `ToFace` and `BetweenFaces` as `extrude` does.

| Returns | What it holds |
|---|---|
| `make_face_tracked` → `(face, edges)` | `edges[l][i]`: the edge of `face` made from piece `i` of loop `l`, with loop 0 the outer loop and loop 1 onward the holes in order. Pieces in the order you listed them, whichever way round the loop goes. |
| `extrude_tracked` → `(solid, history)` | The solid, as `extrude` makes it, and its `ExtrudeHistory`. |
| `ExtrudeHistory::start_cap` | The face the sweep starts from: the input face moved by the extent's start, a face of the solid. For a positive distance it is on the workplane; for a negative distance or a symmetric extent it isn't. |
| `ExtrudeHistory::end_cap` | The face where the sweep ends: the start cap moved by the length along the normal. |
| `ExtrudeHistory::sides` | One `(edge, face)` pair per edge of the input face, in edge index order: an edge of the input face, then the side face of the solid it swept. |

`extrude_feature(k, face, extent, mode, target)` extrudes `face` like `extrude`, then combines the result with `target` as `mode` says (Section 7.3). It returns a new solid; `target` is left unchanged.

| Mode or value | Result and rules |
|---|---|
| `ExtrudeMode::NewSolid` | The extrusion alone. `target` is ignored and may be `None`. |
| `ExtrudeMode::Join` | Target ∪ extrusion. Needs a target. |
| `ExtrudeMode::Cut` | Target − extrusion. Needs a target. |
| `ExtrudeMode::Intersect` | Target ∩ extrusion. Needs a target. |
| `EMPTY_VOLUME_TOLERANCE` | 1e-9 mm³: a result with less volume than this counts as empty and is refused with `EmptyResult`. |

A through-all extent with any mode but cut, a face extent with the new-solid mode, and a missing target, are checked first, before any geometry is made. A face extent is then turned into a plain one: `ToFace` into `Distance(d)`, `BetweenFaces` into `FromTo(d_a, d_b)`, where each `d` is the signed distance of a face's plane from the workplane along its normal, so a face behind the workplane gives a negative distance. A through-all cut is made a little longer than the distance from the outline to the far side of the target, measured from their bounding boxes. If it leaves the target's volume unchanged (within a relative 1e-9), it is refused as a miss. Errors from `extrude` and from the Boolean operations come through unchanged.

`revolve(face, axis, angle)` turns a face from `make_face` about `axis` by `angle` radians into a new solid (Chapter 8). `revolve_feature(k, face, axis, angle, mode, target)` revolves it and combines the result with `target`, with the same modes and rules as `extrude_feature` (Section 8.2).

| Type or value | Fields and rules |
|---|---|
| `Axis { origin, dir }` | The line through the point `origin` along `dir`. `origin` finite; `dir` finite and non-zero, any length (it is scaled to length 1). |
| `angle` | Radians, greater than 0 and at most 2π (a full turn; up to 1e-12 over 2π also counts as a full turn). A positive angle turns counterclockwise looking against `dir`, from its tip (right-hand rule). |

The axis and the angle are checked before any geometry is made. Then `revolve` checks the face against the axis: the face must be one flat face, the axis must lie in its plane, and the face must not have points on both sides of the axis. Each check allows `REVOLVE_AXIS_TOLERANCE`, 1e-7 mm; a face that touches the axis passes (Section 8.3).

## 19.7 Work features

Work planes are `Workplane` values and work axes `Axis` values (Section 19.6), so they go wherever those do. Work points are `Point3` values, `[x, y, z]` in mm. The calls in the first table need no OpenCascade (Section 15.1); those in the second take work features from a part, and need it (Section 15.3).

| Call or value | What it gives and what it accepts |
|---|---|
| `ORIGIN` | The part origin, (0, 0, 0). |
| `Axis::X`, `::Y`, `::Z` | The origin axes: through the origin along +X, +Y and +Z. |
| `plane.offset(distance)` | The plane moved `distance` mm along its normal, keeping its directions. Any finite distance, negative included. |
| `Workplane::through_points(p0, p1, p2)` | The plane through three points: origin `p0`, `x_dir` towards `p1`, normal along (p1 − p0) × (p2 − p0). The points different and not in a line. |
| `Workplane::from_point_normal(p, n)` | The plane through `p` with normal `n`. Its `x_dir` is +X laid flat on the plane, or +Y if `n` is along X. `n` non-zero. |
| `plane.angled(axis, angle)` | The plane turned `angle` radians about `axis` (right-hand rule), origin included. The axis must lie in the plane. Any finite angle. |
| `Axis::through_points(p, q)` | The axis through `p` pointing towards `q`. The points different. |
| `Axis::plane_intersection(a, b)` | The line where two planes meet, along a.normal × b.normal, through its point nearest the origin. The planes not parallel. |

| Call or value | What it gives and what it accepts |
|---|---|
| `plane_axis_intersection(plane, axis)` | The point where the axis crosses the plane. The axis not parallel to the plane. |
| `Workplane::midplane(a, b)` | The plane halfway between two parallel planes, with `a`'s normal and `x_dir`; its origin is halfway from `a`'s origin to `b`. The planes parallel (they may face opposite ways); the same plane twice gives it back. |

| Kernel call | What it gives and what it accepts |
|---|---|
| `work_plane_on_face(s, face)` | The plane of the flat face `face` of `s`: normal the face's outward normal, origin the point of the plane nearest the part origin, `x_dir` as `from_point_normal` gives. `face` a face, in range, flat. |
| `work_axis_on(s, of)` | The line of a straight edge, or the center line of a cylindrical face, of `s`. `dir` a unit vector, either way along the line; origin the line's point nearest the part origin. `of` an edge or a face, in range; not a vertex, a curved edge or a face that isn't cylindrical. |
| `vertex_point(s, vertex)` | Where the vertex `vertex` of `s` is. `vertex` a vertex, in range. |
| `work_plane_tangent(s, face, toward)` | The plane that touches the cylindrical face `face` of `s` along one line, on the side `toward` points to (Section 15.4). Only the part of `toward` square to the cylinder's axis counts; it becomes the normal. Origin the point of the plane nearest the part origin, `x_dir` as `from_point_normal` gives. `face` a face, in range, cylindrical; `toward` non-zero and not along the axis. |
| `edge_midpoint(s, edge)` | The point halfway along the edge `edge` of `s`, measured along the edge, so on a curved edge it lies on the curve. `edge` an edge, in range. |

Every input must be finite, and every refusal is `InvalidInput` (Section 20.2). The checks use `DATUM_TOLERANCE`, 1e-9: directions this close count as parallel, points this close (in mm) as the same, and an axis this close to a plane as lying in it.

## 19.8 Sketch entities and constraints

| Entity | Defined by |
|---|---|
| `Point` | x, y. |
| `Line` | A start point and an end point. |
| `Circle` | A center point and a radius > 0. |
| `Arc` | A center, a radius > 0, and start and end angles in radians, running counter-clockwise; the end angle must be after the start. Solving can change all five; it keeps the arc's sweep more than 0 and at most one full turn. |

| Constraint | Rule |
|---|---|
| `Coincident(p, q)` | Points `p` and `q` are at the same place. |
| `Horizontal(line)` | The line's two ends are at the same height. |
| `HorizontalPoints(p, q)` | `p` and `q` are at the same height. |
| `Vertical(line)` | The line's two ends are directly above each other. |
| `VerticalPoints(p, q)` | `p` is directly above or below `q`. |
| `Fixed { point, x, y }` | The point stays at (x, y). |
| `Distance { a, b, value }` | Points `a` and `b` are `value` apart. Value > 0. |
| `Radius { geom, value }` | The circle or arc has this radius. Value > 0. |
| `Diameter { geom, value }` | The circle or arc has this diameter. Value > 0. |
| `Parallel(l1, l2)` | Lines `l1` and `l2` point the same way (or exactly opposite ways). |
| `Perpendicular(l1, l2)` | Lines `l1` and `l2` are at 90°. |

| Constraint | Rule |
|---|---|
| `Equal(g1, g2)` | Two lines of equal length, or two circles or arcs of equal radius (any mix of circle and arc). A line and a circle can't be made equal. |
| `Midpoint { point, line }` | The point is halfway between the line's two ends. |
| `Symmetric { a, b, line }` | Points `a` and `b` are mirror images across the line: the middle of `a` – `b` lies on the line (or on its extension), and `a` – `b` is perpendicular to it. |
| `Tangent(g1, g2)` | A line and a circle or arc, in either order, touch: the distance from the center to the line (extended without end) equals the radius. Or two circles or arcs touch, from outside or inside. |

A point `p` is written `PointRef::Point(id)` for a point entity, `Start(id)` or `End(id)` for a line's ends, `Center(id)` for a circle's or arc's center, or `ArcStart(id)` or `ArcEnd(id)` for the point of an arc at its start or end angle. Every constraint that takes a point accepts all six.

| Tangent detail | How it is decided |
|---|---|
| Line and circle | The circle stays on the side of the line its center was on when the constraint was added. |
| Two circles | Touching from outside (centers `r1 + r2` apart) or inside ( `\|r1 – r2\|` apart), whichever the drawing was closer to when the constraint was added. |
| Line end joined to arc end | When a coincident joins one of the line's ends directly to one of the arc's ends, tangent means the line is square to the arc's radius at that shared point. This holds whichever of the two constraints is added first; a join made through a separate point entity is not recognized. |
| Where they touch | Otherwise anywhere along the line. Tangent alone doesn't say where a line stops; add other constraints for its ends. |

Tangency uses the standard distance conditions for lines and circles; see A. Bowyer and J. Woodwark, *A Programmer's Geometry*, chapters 3 and 5.

## 19.9 Solver settings and report

| Setting | Meaning | Default |
|---|---|---|
| `tolerance` | Done when the remaining error is at most this. | `1e-10` |
| `max_iterations` | Give up after this many attempts. | `100` |
| `initial_damping` | How cautious the first steps are. | `1e-6` |

| Stop reason | Meaning |
|---|---|
| `Converged` | Every rule holds, within the tolerance. `converged` is `true`. |
| `MaxIterations` | The attempts ran out first. |
| `Stagnated` | No move makes it better: usually rules that conflict. |
| `NonFinite` | A value became infinite or not a number. |

The solver uses the Levenberg–Marquardt method, a standard way to solve many equations at once; see J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., chapter 10.

## 19.10 Profiles

`swx_sketch::profile::profile_report(&sketch, &options)` finds the regions a sketch encloses and the pieces left open (Section 4.7). `profiles(&sketch, &options)` returns the regions alone. Point entities are ignored.

| Setting | Meaning | Default |
|---|---|---|
| `weld_tolerance` | Ends of pieces closer than this count as one corner. `None` uses 0.000001 × the diagonal of the box around the sketch. Must be > 0 when given. | `None` |

| Result | Meaning |
|---|---|
| `regions` | Every region found. Each has an `outer` loop and a list of `holes`. |
| `Region::area()` | The region's area: the outer loop's area minus its holes', in mm². |
| `Loop` | The `edges` around it in order, each a piece's id and whether it is walked backwards ( `reversed` ), and its `signed_area`. |
| `signed_area` | Positive for an outer loop, which runs counter-clockwise; negative for a hole, which runs clockwise. Exact for arcs. |
| `open_edges` | Ids of the pieces in no loop: free-ended pieces, pieces that only link two loops, and pieces of zero length. Smallest id first. |

| Rule | What happens |
|---|---|
| Circle | A loop on its own. So is an arc that goes all the way round. |
| Loop inside a loop | Becomes a hole of the smallest loop around it. A loop inside that hole is a new region (an island), and so on, alternating. |
| Pieces that cross, overlap or touch away from their ends | An error, `SelfIntersecting`, naming the first two such pieces. No regions are returned. |
| No closed loop | No regions, and no error. |

Loops are found by walking around each corner in angle order, the standard method for the faces of a flat network of lines; see M. de Berg, O. Cheong, M. van Kreveld and M. Overmars, *Computational Geometry*, 3rd ed., §2.2. Areas use Green's theorem (J. Stewart, *Calculus*, 8th ed., §16.4).

## 19.11 Degree-of-freedom report

`sketch.dof(options)` checks a sketch as it is now (Section 4.6) and returns a `SketchDofReport`. Run it after `solve`. The lower-level `swx_sketch::dof::analyze(&sketch, sketch.params(), options)` returns the `DofReport` alone.

| Setting | Meaning | Default |
|---|---|---|
| `rank_tolerance` | How independent an equation must be, compared with the largest, to count as a new rule rather than a repeat. | `1e-9` |
| `residual_tolerance` | A sketch with a repeated rule is `Conflicting`, not `OverConstrained`, when its remaining error is more than this. | `1e-8` |

| Report field | Meaning |
|---|---|
| `dof` | Degrees of freedom left: the sketch's numbers minus its independent equations. |
| `rank` | The number of independent equations. |
| `redundant` | The numbers of the equations that repeat earlier ones, smallest first, counting from 0 in the order the constraints were added. |
| `residual_norm` | The remaining error: how far the rules are from being met, overall. |
| `state` | One of the states below. |

| State | When |
|---|---|
| `UnderConstrained` | No repeated rule, and `dof` is more than 0. |
| `FullyConstrained` | No repeated rule, and `dof` is 0. |
| `OverConstrained` | At least one repeated rule, and every rule is met. |
| `Conflicting` | At least one repeated rule, and the remaining error is more than `residual_tolerance`. |
| `NonFinite` | A value is infinite or not a number. Nothing else was worked out. |

| SketchDofReport field | Meaning |
|---|---|
| `report` | The `DofReport` above. |
| `redundant_constraints` | The constraints that own a repeated equation, as `ConstraintId`s, smallest first, each listed once. Of two rules that repeat each other, the one added later is listed. |

`sketch.constraint_of_row(n)` gives the constraint that made equation `n`, or `None` when there is no such equation.

| Entity or constraint | Numbers or equations |
|---|---|
| Point · Line · Circle · Arc | 2 · 4 · 3 · 5 numbers (degrees of freedom when free). |
| Coincident · Fixed · Midpoint · Symmetric | 2 equations each. |
| Every other constraint | 1 equation. |

The count uses a Householder QR factorization, taking the equations in the order they were added; see G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., §5.2.

## 19.12 Inference

`swx_sketch::infer` suggests the constraints a newly added entity already almost satisfies (Section 5.8). `infer(&sketch, new, &options)` returns them and changes nothing. `add_inferred(&mut sketch, new, &options)` adds them in order, then solves, and returns the new constraint ids. Both fail if `new` doesn't exist or a setting is invalid.

| Setting | Meaning | Default |
|---|---|---|
| `point_tol` | How close, in mm, two points must be to be joined, and how near a line or circle must come to touching to be made tangent. Finite and > 0. | diagonal ÷ 1000 |
| `angle_tol_deg` | How close, in degrees, a line must be to level, upright, parallel or perpendicular. Finite and > 0. | 1 |

`InferOptions::for_sketch(&sketch)` gives the defaults for that sketch. The diagonal is that of the smallest axis-aligned box holding the whole sketch, with circles and arcs counted as full circles; for an empty sketch `point_tol` is 0.001. The suggestions come in this order:

| Suggestion | When |
|---|---|
| Coincident | For each point of the new entity (a line's start, then end; a circle's center; an arc's center, start, then end), the nearest other point within `point_tol`, if any: a point, a line end, an arc end or a center. Ties go to the entity added first. |
| Horizontal, vertical | The new entity is a line within `angle_tol_deg` of level or upright. |
| Parallel, perpendicular | The new line is within `angle_tol_deg` of parallel or square to another line. At most one, the closest. When the new line got horizontal or vertical, lines that are themselves level or upright are skipped. |
| Tangent | A line whose distance to a circle's or arc's center is within `point_tol` of the radius (the line counted as endless); or two circles or arcs whose centers are within `point_tol` of touching, outside or inside. Circles with the same center are skipped. |

A constraint the sketch already holds, in either order, is never suggested, so inferring twice adds nothing. Inference never suggests a dimension.

| Error from add_inferred | Meaning |
|---|---|
| `Sketch` | The entity doesn't exist, or a setting is not finite or not > 0. Nothing is added. |
| `Solve` | The solver could not start. |
| `NotConverged` | The constraints were added, but the solve did not finish; the geometry is the solver's best attempt. |

## 19.13 History

A `swx_core::Document` keeps its features in history order, with the end-of-part marker between the active features and the rolled-back ones (Section 4.10, Section 17.3). Each feature has a `FeatureId`, a number given when it is made: 0, 1, 2, … in the order of making, never reused or changed. Once a feature is inserted above others, a feature's number and its place in the history differ.

| Call | What it does |
|---|---|
| `add_feature(name, kind)` | Inserts a feature at the end-of-part marker, moves the marker down past it, and returns its `FeatureId`. Fails if a parameter it uses doesn't exist, or a feature it uses isn't above the marker; then nothing changes. |
| `history()` | Every feature in history order, active or rolled back. |
| `end_of_part()` | The marker's position: how many features are above it, from 0 to the number of features. A new document has 0; while the marker is at the end, `add_feature` keeps it there. |
| `active_features()` | The features above the marker: the first `end_of_part()` features of `history()`. |
| `set_end_of_part(pos)` | Moves the marker so that `pos` features are above it, and returns the features that became active, in history order; they must be rebuilt. Rolling back, or staying in place, returns none. A `pos` past the number of features fails, and the marker stays. |
| `dirty_after_feature_edit(id)` | The active features to rebuild after feature `id` is edited, in history order: itself and every feature that uses it, directly or not. A rolled-back feature rebuilds nothing. Fails for an unknown `id`. |
| `dirty_after_param_change(name)` | See Section 19.14. Rolled-back features are never listed. |

| Feature kind | Uses |
|---|---|
| `FeatureKind::Sketch` | nothing |
| `FeatureKind::Extrude { profile, depth }` | the sketch `profile`, and the parameter named `depth` |
| `FeatureKind::Revolve { profile, angle_deg }` | the sketch `profile`, and the parameter named `angle_deg`, in degrees |

## 19.14 Parameters

A `swx_core::Document` keeps its parameters in creation order (Section 17.4). Every call that can fail leaves the table unchanged when it does.

| Call | What it does |
|---|---|
| `add_model_param(value, unit)` | Makes a model parameter named `d<N>` and returns the name. `N` starts at 0 and only goes up; a name already taken is skipped. |
| `add_user_param(name, value, unit)` | Makes a user parameter. Fails if the name is invalid or taken. |
| `rename_param(old, new)` | Renames a parameter, every feature that uses it, and every formula that names it (whole names only: renaming `d1` leaves `d10` alone). The kind is kept. Renaming to the same name does nothing. |
| `set_param_comment(name, comment)` | Sets the comment. Fails if there is no such parameter. |

| Call | What it does |
|---|---|
| `set_param(name, value)` | Sets the value and drops the parameter's formula, if it had one, then works out every formula that depends on it. If one of those fails, they keep their old values, with no error. An unknown name makes a user parameter with an empty unit; this name is not checked. |
| `set_param_expr(name, text)` | Gives the parameter a formula (Section 19.15) and works out the whole table. The result is stored in the parameter's unit; a unitless result is taken in that unit. Fails if the parameter or a name in the formula is unknown, the formula doesn't parse or can't be worked out, its result is the wrong kind of quantity for the unit, or formulas would form a loop. |
| `evaluation_order()` | Every parameter name, each after the parameters its formula names; ties keep creation order. |
| `dirty_after_param_change(name)` | The active features to rebuild when `name` changes, including those that use a parameter whose formula depends on it, directly or through other formulas. |
| `param(name)` | The value, or `None`. |
| `parameters()` | Every parameter, in creation order. |
| `is_valid_param_name(name)` | Whether a name is allowed. |

| Parameter field | Meaning | Allowed values |
|---|---|---|
| `name` | Unique within the part. Case-sensitive. | a letter or `_`, then letters, digits or `_` (ASCII) |
| `kind` | Where it came from. | `Model`, `User` |
| `value` | The number; for a parameter with a formula, its latest result. | any number |
| `unit` | Free text; not checked. Formulas understand only the units in Section 19.15. | any, may be empty |
| `comment` | A note for people. | any; empty by default |
| `expr` | The formula as typed, or `None` for a plain number. | `None` by default |

The dependency order uses Kahn's algorithm, always taking the earliest-made parameter that is ready; see A. B. Kahn, "Topological sorting of large networks", *Communications of the ACM* 5(11), 1962, and T. H. Cormen et al., *Introduction to Algorithms*, 3rd ed., §22.4.

A unit that is not one of the seven in Section 19.15, including an empty one, counts as unitless when a formula reads or stores the parameter.

`swx_sketch::params` ties a sketch's dimensions to a document's parameters (Section 17.5). A sketch remembers each binding as a pair of constraint and parameter name; `sketch.bindings()` lists them in the order they were made.

| Call | What it does |
|---|---|
| `bind_dimension(sketch, doc, c)` | Adds dimension `c` (a `Distance`, `Radius` or `Diameter`) to the sketch, and a model parameter with the same value and unit `mm` to the document; returns the constraint id and the parameter's name. A diameter's parameter holds the diameter. On error neither changes. |
| `apply_params(sketch, doc)` | Copies each bound parameter's current value into its dimension. Checks every value first, so on error nothing changes. Solve afterwards to move the geometry. |
| `rename_bound(sketch, old, new)` | Points the sketch's bindings from `old` to `new`; returns how many changed. Call it with every `doc.rename_param`. |
| `sketch.set_dimension_value(id, value)` | Changes one dimension's value directly. Value finite and > 0. |

## 19.15 Expressions

`swx_core::expr` reads and works out expressions (Section 17.6). `parse(text)` turns the text into an `Expr`; `eval(&expr, &resolve, default_length)` works it out, asking `resolve(name)` for each name's value. `expr.names()` lists the names an expression uses, each once, in first-use order. The result is a `Quantity`: a `value` in millimeters and

radians and a `dim` (its powers of length and angle). `q.in_unit(unit)` converts the value for display. `default_length` is the unit for showing lengths; it doesn't change the result.

| Part | Rule |
|---|---|
| Numbers | `5`, `2.5`, `.5`, `1e-3`. A bare number has no unit. |
| Names | A letter or `_`, then letters, digits or `_`, as for parameters. |
| Operators | `+ - * /`, unary `-`, and `( )`. `*` and `/` before `+` and `-`; left to right otherwise. |
| Units | Written straight after a number or a `)`: `5 mm`, `(2 + 3) mm`. After a name, a word is not a unit. |
| `+` and `-` | Both sides must have the same dimension: two lengths, two angles, or two bare numbers. |
| `*` and `/` | Any dimensions; they multiply or divide. A length times a length is an area. |
| Limits | At most 1024 pieces and 64 levels of brackets or minus signs. |

| Unit | Measures | Size |
|---|---|---|
| `mm` | length | 1 mm |
| `cm` | length | 10 mm |
| `m` | length | 1000 mm |
| `in` | length | 25.4 mm |
| `ft` | length | 304.8 mm |
| `deg` | angle | π/180 rad |
| `rad` | angle | 1 rad |

## 19.16 File formats and units

| Format | Read / write | Notes |
|---|---|---|
| STEP (`.step`, `.stp`), ISO 10303-21 | Both | Solids only. Written in millimeters. |
| PNG (`.png`) | Write | Pictures from `swx-qa render` and `swx-qa sketch`. |
| Saulitwerx document, unpacked form (a `.sltwx`, `.sltwxa` or `.sltwxd` folder) | Both | The manifest is checked in full; the model and properties are kept as JSON objects, not yet tied to a part (Chapter 18, Section 19.17). |
| Saulitwerx document, one file (`.sltwx`, zipped) | Not yet | Planned: the part with its full history in one file. |

| Quantity | Unit |
|---|---|
| Length | millimeter (mm) |
| Area, volume | $mm^2$, $mm^3$ |
| Angle | degrees for feature parameters such as a revolve's sweep; radians for arc angles in sketches and for the geometry engine's `revolve` and `revolve_feature` |

## 19.17 Document files

`swx-io` reads and writes Saulitwerx documents (Chapter 18). Every call returns a `FormatError` on failure; reading never panics.

| Call | What it does |
|---|---|
| `write_unpacked(dir, doc)` | Writes `doc` to the folder `dir`, made if missing: `manifest.json`, `model.json`, and `properties.json` when `doc.properties` is `Some` (deleted when `None`). Other entries in `dir` are left alone. Nothing is written when `dir`'s extension doesn't match the manifest's kind, or the model or properties isn't a JSON object. |
| `read_unpacked(dir)` | Reads the `Document` in the folder `dir`. `manifest.json` and `model.json` required, `properties.json` optional; other entries not read. Checks the manifest, then the extension against its kind, then that the model and properties are JSON objects. |
| `read_manifest(text)` | Reads and checks the text of a `manifest.json` (rules below). |
| `write_manifest(m)` | The text of a `manifest.json` for `m`, in canonical form, extra fields included, `model_hash` left out when `None`. |
| `to_canonical_json(v)` | Any JSON-representable value as canonical JSON: names in byte order at every level, 2-space indent, one member or element per line, `": "` after each name, `{}` and `[]` for empty ones, other characters kept as UTF-8, one newline at the end. The same value always gives the same text. |
| `Document` | `manifest` (a `Manifest`), `model` (a JSON object), `properties` (`Option`, a JSON object). |
| `DocumentKind` | `Part` (`"part"`, `.sltwx`), `Assembly` (`"assembly"`, `.sltwxa`), `Drawing` (`"drawing"`, `.sltwxd`). `as_str()` gives the name. |

| Manifest field | Required | Allowed values |
|---|---|---|
| `format` | Yes | `"sltwx"`. |
| `format_version` | Yes | `MAJOR.MINOR.PATCH` in decimal digits. Major must be 0; any minor or patch is read. |
| `kind` | Yes | `"part"`, `"assembly"` or `"drawing"`. |
| `document_id` | Yes | A UUID in 8-4-4-4-12 hexadecimal form, either case. Fixed for the document's life. |
| `units` | Yes | An object with `length`, `angle` and `mass` unit names (text, such as `"mm"`, `"deg"`, `"kg"`); other names kept in `Units::extra`. |
| `created_by`, `saved_by` | Yes | Text: the program and version that made, and last saved, the document. |
| `model_hash` | No | `sha256:` and 64 lowercase hexadecimal digits: the hash of the canonical `model.json`. Left out when `None`. |
| Any other field | No | Kept in `Manifest::extra` and written back unchanged. |

`FormatError` kinds: `Json` (not valid JSON; the message names the file), `MissingField` (a required field), `InvalidField` (a wrong value or type, or a model or properties that isn't an object), `UnsupportedVersion` (major version not 0), `Io` (a file or folder can't be read, written or removed, or isn't a folder), `MissingFile` (`manifest.json` or `model.json`), `ExtensionMismatch` (folder name and kind disagree). The messages are in Section 20.2.

## 19.18 Keyboard and mouse

There are no keyboard or mouse controls yet, because there is no app window. They arrive with the desktop app and will be listed here.

PART V

# Help

What to do when something goes wrong: symptoms, every error message you can see, common questions and known limitations.

CHAPTER 20

# Help

## 20.1 Troubleshooting

| Symptom | What to do |
|---|---|
| `cargo: command not found` | Rust isn't installed, or this terminal started before it was. Open a new terminal; if that doesn't help, repeat the Rust step in Chapter 2. |
| `could not find Cargo.toml` | The terminal isn't in the `saulitwerx` folder. Run `cd saulitwerx` and try again. |
| A build with `--features occt` fails with errors about `.hxx` files or `TK…` libraries | OpenCascade isn't installed. Install the `libocct-…-dev` packages from Chapter 2. |
| `swx-qa: cannot read … STEP input needs OCCT` | `swx-qa` was built without OpenCascade. Rebuild it: `cargo build --release -p swx-qa --features occt`. |
| `target/release/swx-qa: No such file or directory` | The tool isn't built yet. Run the build step in Chapter 2. |
| Rows of `*` and colored text appear when saving or reading STEP | Normal. The geometry engine prints its own report. Saulitwerx's own lines come before and after it. |
| The picture looks small or blurry when printed | Use a larger `--size`: about 80 pixels per centimeter of printed width. |
| A curved part is covered in lines | Raise `--crease`, for example to 45. |
| The numbers in a sketch picture are too small to read | Use a smaller `--size`. Labels have a fixed size in pixels, so they take up more of a small picture. |
| A sketch solves, but its sizes aren't what you expect | Some of it is still free to move. Check the degrees of freedom (Section 5.3) and add the missing dimensions. |
| A sketch looks closed, but has no region | Some ends don't meet. The open-edge list names the pieces (Section 5.7). Join every end, line or arc, to its neighbor with coincident (Section 5.6). |
| An arc flips or bulges the wrong way after solving | An arc runs counterclockwise from `ArcStart` to `ArcEnd`. Check that each line end is joined to the right end of the arc (Section 5.6). |
| A sketch dimension ignores a new parameter value | Copy the values into the sketch with `apply_params`, then solve again (Section 17.5). |
| Inference left a corner open | The ends were farther apart than `point_tol`. Draw the new piece from where the last one ended, or widen `point_tol` (Section 5.8). |
| Inference made a line level that you meant to lean | It was within `angle_tol_deg` of level. Lower the setting, or don't add the suggestion (Section 5.8). |
| A parameter stopped following its formula | `set_param` replaced the formula with a plain number. Give it the formula again with `set_param_expr` (Section 17.7). |
| A formula's value didn't change after its inputs did | The new inputs made the formula fail, for example by dividing by zero, so it kept its old value. Change the inputs back, or give a formula that works (Section 17.7). |
| The solver stops with `Stagnated` | Two or more rules contradict each other. Check the degrees of freedom: the report says `Conflicting` and gives the clashing equation's number. |

## 20.2 Error messages

### From `swx-qa`

Every message starts with `swx-qa:` . No picture is written when there is an error.

| Message | Meaning and fix |
|---|---|
| `missing command` | Nothing after `swx-qa` . Add `render` , `sketch` , or `--help` . |
| `unknown command `x`` | Only `render` , `sketch` and `help` exist. |
| `unknown option `x`` | A misspelled option, or one that belongs to the other command ( `--crease` and `--step` are for `render` only). The help text follows the message. |
| `` `--size` needs a value `` | An option was last on the line without its value. The same goes for the other options. |
| `missing input: give --fixture <name> or --step <file.step>` | Say what to draw. |
| `give exactly one of --fixture and --step` | Two inputs were given. Keep one. |
| `missing input: give --fixture <name>` | `sketch` needs the name of a built-in sketch. |
| `missing output: give -o <out.png>` | Say where to save the picture. |
| `unknown fixture `x`; available: cube, cylinder, l-bracket` | Use one of the listed names. |
| `unknown sketch fixture `x`; available: rectangle, triangle-345` | Use one of the listed names. |
| `invalid --size `x`: expected <W>x<H>, e.g. 800x600` | Write the size as two whole numbers joined by `x` , with no spaces. |
| `invalid --size `x`: width and height must be > 0` | Neither side may be 0. |
| `image size WxH must be between 1x1 and 8192x8192` | Use a smaller size. |
| `invalid --crease `x`: expected degrees` | Give a number, such as `45` . |
| `crease angle x must be between 0 and 180 degrees` | Use a value from 0 to 180. |
| `cannot read f: invalid input: no such file: f` | The STEP file isn't there. Check the name and the folder. |
| `cannot read f: kernel failure: import_step from f failed` | The file isn't valid STEP. The geometry engine prints the reason just above. |
| `cannot write f: …` | The picture can't be saved there, for example because the folder doesn't exist. |
| `mesh is empty` , `mesh has no visible extent` | The STEP file holds nothing that can be drawn. |

### From the geometry kernel

| Message | Meaning and fix |
|---|---|
| `invalid input: <name> must be finite and > 0, got <v>` | A size, radius or height is zero, negative or not a number. |
| `invalid input: <name> must have finite coordinates, got …` | A position contains an infinite value or not-a-number. |
| `invalid input: no such file: f` | The STEP file to read doesn't exist. |
| `invalid input: adjacent needs two different kinds, got edge and edge` | `adjacent` was asked for the same kind it starts from. Ask for another kind: the faces or vertices of an edge (Section 6.3). |
| `invalid input: face index 7 is out of range: the shape has 7 faces` | The index is past the last sub-shape of its kind. Indices start at 0, so the last is the count minus 1. An index from another shape, or from before a change, can also be out of range (Section 6.3). |
| `kernel failure: <operation> failed` | OpenCascade couldn't complete the operation. For a cut, check that the tool solid overlaps the part (Chapter 6). For a `fillet` or a `chamfer` , the size is too large for the faces next to the |

| Message | Meaning and fix |
|---|---|
| | edges: make it smaller (Section 9.4). For a `shell`, the walls are too thick for the part: keep them under half its narrowest width (Section 10.2). For a `draft`, a face can't be tilted, such as one parallel to the neutral plane (Section 12.2). For a `loft`, two sections may be in the same place (Section 13.2). |
| `invalid input: the edge list is empty` | A fillet or a chamfer was given no edges. Give at least one (Chapter 9). |
| `invalid input: the face list is empty` | A shell was given no faces to open, or a draft no faces to tilt. Give at least one (Chapter 10, Chapter 12). |
| `invalid input: vertex 1 is not on edge 11` | A variable fillet's start vertex isn't one of its edge's two ends. Ask `adjacent` for the edge's vertices (Section 9.3). |
| `invalid input: a variable fillet needs an edge and a vertex, got Face 0 and Vertex 2` | The first sub-shape must be an edge, the second a vertex (Section 9.3). |
| `invalid input: counterbore diameter 5 must be larger than the hole diameter 6` | A counterbore's recess must be wider than its hole (Section 11.3). |
| `invalid input: hole depth 4 must be larger than the counterbore depth 6` | A blind hole must go deeper than its recess. Make it deeper, or go through all. |
| `invalid input: countersink diameter 6 must be larger than the hole diameter 6.4` | A countersink's cone must be wider than its hole (Section 11.3). |
| `invalid input: countersink angle <v> (<d>°) must be strictly between 0 and π radians` | The angle across the cone is 0, 180° or more, not a number, or given in degrees. `<d>` is your angle in degrees. Give radians: `90f64.to_radians()` (Section 11.2). |
| `invalid input: hole depth 2 must be larger than the countersink depth 2.8` | A blind hole must go deeper than its cone. The cone depth is rounded to 6 decimal places. Make the hole deeper, or go through all. |
| `invalid input: no metric coarse thread of size M7 (sizes: M2 to M24)` | The size isn't in the thread table (Section 19.5). Pick one that is. |
| `invalid input: thread designation "6" must be M followed by a size, e.g. "M6"` | The name is missing its `M`, or isn't a number after it. |
| `invalid input: thread pitch 6 must be smaller than the nominal diameter 6` | A thread typed by hand has a pitch as large as its size. Take sizes from `metric_coarse`. |
| `invalid input: hole direction must be finite and non-zero, got …` | The hole's direction is missing or not a number. Point it into the material. |
| `invalid input: the hole misses the solid: it removes nothing` | The hole's line misses the part, or points away from it (Section 11.3). |
| `kernel failure: hole failed: the result is not a valid solid` | The geometry engine made the hole, but the result isn't a sound solid. Try a slightly different position or size. |
| `empty result: the hole removes the whole solid` | The hole is so large that nothing is left. |
| `invalid input: draft angle must be finite, non-zero and below π/2 in size, got <v>` | A draft angle is zero, 90° or more, not a number, or given in degrees. Give radians: `5f64.to_radians()` (Section 12.1). |
| `invalid input: draft pull direction must be finite and non-zero, got …` | The neutral plane's direction is missing or not a number. |
| `invalid input: a loft needs at least 2 sections, got 1` | A loft was given one section, or none (Section 13.2). |
| `invalid input: loft section 1 has a hole; sections with holes are not supported` | A loft section is ring-shaped. Loft the outer outline, and cut the hole separately. |
| `invalid input: loft section 1 is not a single planar face` | A loft section is a solid, or more than one face. Pass faces from `make_face`. |
| `empty result: the loft encloses no volume` | The sections enclose nothing between them. |
| `invalid input: sweep path is empty` | A sweep was given no path (Section 14.3). |

| Message | Meaning and fix |
|---|---|
| `invalid input: sweep path: circles (closed paths) are not supported` | The path holds a full circle. A sweep path must be open; for a ring, revolve instead (Chapter 8). |
| `invalid input: sweep path: segment 1 doesn't meet segment 0 within 0.0000001` | Two neighboring pieces of the path don't touch. Reuse the same end values (Section 14.3). |
| `invalid input: sweep path starts 5 off the profile's plane` | The path's start isn't on the profile's plane; the number is how far off it is, in mm. Start the path at the profile's middle. |
| `invalid input: sweep profile is not a single planar face`, `sweep profile has a hole; profiles with holes are not supported` | The profile is a solid, or has a hole. Pass one face from `make_face`; for a tube, cut an inner sweep from an outer one. |
| `invalid input: sweep path: arc segment 1 has radius 4, not larger than the profile's distance 5.000000232604544 from the path's start; the swept solid would pass through itself` | A bend of the path is tighter than the profile is wide: the inside of the part would fold through itself. Make the bend's radius larger than the distance shown, or the profile smaller (Section 14.3). |
| `empty result: the sweep encloses no volume` | The path runs along the profile's own plane, so the profile sweeps out nothing. Draw the path on a plane that crosses the profile's. |
| `invalid input: helix pitch must be finite and > 0, got 0`, `helix turns must be finite and > 0, got …` | A helical sweep was given a pitch or a number of turns of 0 or less, or not a number (Section 14.2). |
| `invalid input: helix axis direction must be non-zero`, `helix axis direction must be finite, got …`, `helix axis origin must be finite, got …` | The helix axis has no direction (all zeros), or a value that isn't a number. |
| `invalid input: helix axis (origin …, direction …) is not in the profile's plane (through …, normal …)` | The axis must lie in the profile's plane. Draw the profile on a plane that holds the axis (Section 14.3). |
| `invalid input: the profile crosses the helix axis: its distance from the axis spans [-1, 1]`, `the profile touches the helix axis: …` | The profile isn't wholly on one side of the axis. Move it away from the axis. |
| `invalid input: the profile's extent along the helix axis (2) must be less than the pitch (1.5), or neighboring turns overlap` | The profile is as tall as the pitch or taller, along the axis, so each turn would run into the next. Raise the pitch. |
| `invalid input: helical sweep profile is not a single planar face`, `helical sweep profile has a hole; profiles with holes are not supported` | Pass one flat face without holes from `make_face`. |
| `invalid input: work_axis_on needs an edge or a face, got a vertex` | A work axis was asked of a corner. Give a straight edge or a round face (Section 15.3). |
| `invalid input: Edge 13 is not a straight edge or a cylindrical face` (or `Face …`) | The edge is curved, or the face isn't round like a cylinder, so it has no single line. |
| `invalid input: vertex_point needs a vertex, got a Edge sub-shape` (or `Face`) | A work point was asked of an edge or a face. Give a vertex. |
| `invalid input: the planes are not parallel: they have no midplane` | A midplane needs two parallel planes (Section 15.5). |
| `invalid input: face 1 is not cylindrical` | A tangent plane was asked of a face that isn't round like a cylinder. Give the round side of a shaft, a boss or a hole (Section 15.4). |
| `invalid input: toward is parallel to the cylinder axis: it picks no side` | The direction given for a tangent plane runs along the cylinder, so it doesn't say which side to touch. Point it out sideways from the axis. |
| `invalid input: work_plane_tangent needs a face, got a Edge sub-shape` (or `Vertex`), `edge_midpoint needs an edge, got a Face sub-shape` (or `Vertex`) | A tangent plane needs a face, and a midpoint an edge. |

| Message | Meaning and fix |
|---|---|
| `invalid input: combine needs at least one tool body` | A combine was given an empty list of tools (Section 6.5). |
| `invalid input: the combine target has no solid body`, `combine tool 0 has no solid body` | The target, or a tool (counted from 0), is a flat face, not a solid. Extrude it first. |
| `empty result: the combined result encloses no volume` | A cut removed everything, or an intersect found no common volume. |
| `invalid input: the planes are parallel: they have no line in common` | Two parallel planes have no line where they meet (Section 15.5). |
| `invalid input: the three plane points are collinear` | Three points in a line don't fix a plane. Move one off the line. |
| `invalid input: axis direction (the points coincide) must be non-zero`, `plane points (two coincide) must be non-zero` | An axis or a plane was given the same point twice. |
| `invalid input: the axis is parallel to the plane` | The axis never crosses the plane, so there is no point (Section 15.5). |
| `invalid input: the axis does not lie in the plane` | A plane can turn only about an axis on it (Section 15.5). |
| `invalid input: <name> must be finite, got …`, `<name> must be non-zero` | A work feature was given a number that isn't finite, or a zero direction: for example `plane normal`, `axis direction` or `offset distance`. |
| `invalid input: shell thickness must be finite and > 0, got <v>` | A shell's wall thickness is zero, negative or not a number. Give a thickness in mm (Section 10.1). |
| `invalid input: chamfer angle must be finite and strictly between 0 and π/2, got <v>` | An uneven chamfer's angle is 0, 90° or more, not a number, or given in degrees. Give radians: `30f64.to_radians()` (Section 9.2). |
| `invalid input: face 4 does not bound edge 11` | An uneven chamfer's reference face isn't one of the two faces that meet at its edge. Ask `adjacent` for the edge's faces and pick one (Section 9.4). |
| `invalid input: edge 11 is listed with two different faces (3 and 5)` | An uneven chamfer lists the same edge twice, with different reference faces. Keep one. |
| `invalid input: expected a face, got edge 0` (or `vertex`) | An uneven chamfer's reference face, or one of a shell's open faces, is an edge or a vertex. Give a face (Section 10.2). |
| `invalid input: expected an edge, got face 0` (or `vertex`) | A fillet's or a chamfer's list holds something that isn't an edge. Give edges only (Section 9.4). |
| `invalid input: edge index 12 is out of range: the shape has 12 edges` | As for faces: the edge number is past the last edge, or comes from another solid. Find the edges in the solid you are softening. |
| `invalid input: face index 14 is out of range (the shape has 14 faces)` | `face_plane`, or a face extent, was given a face number past the last face of the target. |
| `invalid input: face_plane needs a face, got a Edge sub-shape` (or `Vertex`) | `face_plane` or `work_plane_on_face` was given an edge or a vertex. Give a face. |
| `invalid input: face 2 is not planar` | The face is curved, so it has no plane: a to-face extrude can't end on it, and no work plane can sit on it. Pick a flat face (Section 7.6, Section 15.3). |
| `invalid input: outer loop: consecutive segments don't meet within 0.0000001` (or `hole N loop: …`) | Two pieces of the loop, or its last and first, don't meet. Reuse the same corner values for both ends (Chapter 7). |
| `invalid input: outer loop is empty`, `… loop encloses no area` | The loop has no pieces, or they enclose nothing: a line there and back, or a figure eight whose halves are the same size. |
| `invalid input: … loop of one segment must be a circle`, `… a circle must be the only segment of its loop` | A loop of one piece must be a full circle, and a circle can't share a loop. Use arcs to round corners. |

| Message | Meaning and fix |
|---|---|
| `invalid input: line from … to … has no length`, `arc from … to … rad has no sweep; use a circle` | A piece starts and ends at the same point. |
| `invalid input: radius must be > 0, got <v>` | An arc or circle has a zero or negative radius. |
| `invalid input: line must be finite, got …` (also `arc`, `circle`, `workplane origin`) | A number is infinite or not a number. |
| `invalid input: workplane x_dir must be non-zero` (or `normal`), `workplane x_dir and normal must be perpendicular (dot …)` | The workplane's directions are missing or not at right angles (Section 19.6). |
| `kernel failure: make_face failed: the face is invalid (e.g. a self-intersecting loop or a hole outside the outer loop)` | The loops pass the checks above, but the outline crosses itself, or a hole is outside the outline or touches its edge. |
| `invalid input: extrude distance must be finite and non-zero, got <v>` (or `symmetric extrude length`) | The extent is zero or not a number. |
| `kernel failure: extrude failed` | The shape given to `extrude` or `extrude_tracked` isn't a face from `make_face` or `make_face_tracked`. |
| `invalid input: extrude mode "join" needs a target body` (or `"cut"`, `"intersect"`) | The mode combines with a part, but none was given. Pass the part as the target, or use `NewSolid` (Section 7.3). |
| `invalid input: a through-all extent works only with the cut mode, not "join"` (or another mode) | Only a cut can go through all. Give a distance instead (Section 7.4). |
| `invalid input: a through-all extent needs a target body to measure; use extrude_feature with the cut mode` | A through-all extent was given to `extrude` or `extrude_tracked`, which have no target to measure. Use `extrude_feature` with `ExtrudeMode::Cut`. |
| `invalid input: a to-face or between-faces extent needs a target body: it works with the join, cut and intersect modes, not "new solid"` | A face extent was given with the new-solid mode. Use join, cut or intersect, with the part as the target (Section 7.6). |
| `invalid input: a to-face or between-faces extent needs a target body to measure; use extrude_feature with the join, cut or intersect mode` | A face extent was given to `extrude` or `extrude_tracked`. Use `extrude_feature`. |
| `invalid input: target face 11 (normal …) is not parallel to the workplane (normal …)` | The face stands at an angle to the outline's plane. Pick a face parallel to the workplane (Section 7.6). |
| `invalid input: face 5 lies in the workplane: the to-face, extrude has zero length` `faces 7 and 9 are coplanar: the between-faces extrude has zero length` | The face is on the outline's own plane, or the two faces are on one plane, so the extrude has no length. Pick another face. |
| `invalid input: extrude start and end must be finite, got …`, `extrude length (end − start) must be finite and non-zero, got 0` | A from–to extent has a number that isn't finite, or the same start and end. |
| `invalid input: the through-all cut misses the target: it removes nothing` | The cut went the wrong way, or the outline isn't over the part. Change the direction or move the outline (Section 7.4). |
| `invalid input: revolve angle must be in (0, 2π] radians, got <v>` | The angle is zero, negative, more than a full turn, not a number, or given in degrees. Give radians: a full turn is 2π (Section 8.1). |
| `invalid input: revolve axis direction must be non-zero` (or `must be finite, got …`), `revolve axis origin must be finite, got …` | The axis is missing its direction, or has a number that isn't finite. |
| `invalid input: the profile crosses the revolve axis (origin …, direction …): it reaches <a> on one side and <b> on the other` | The axis runs through the outline. Move the axis or the outline so the outline is all on one side; touching the axis is fine (Section 8.3). |
| `invalid input: revolve axis (origin …, direction …) is not in the profile's plane (through …, normal …)` | The axis is tilted out of the outline's plane, stands straight out of it, or runs beside it. Use an axis that lies in the plane (Section 8.3). |

| Message | Meaning and fix |
|---|---|
| `invalid input: revolve needs a single planar face` | The shape given to the revolve isn't one flat face. Pass a face from `make_face`. |
| `kernel failure: revolve failed` | The geometry engine couldn't make the revolve, although its input passed every check above. |
| `invalid input: revolve mode "join" needs a target body` (or `"cut"`, `"intersect"`) | As for an extrude: pass the part as the target, or use `NewSolid` (Section 8.2). |
| `empty result: revolve new solid leaves no solid (volume …)` (or `join`, `cut`, `intersect`) | Nothing is left: for a cut or an intersect, as for an extrude. |
| `empty result: extrude cut leaves no solid (volume …)` (or `intersect`, `join`) | Nothing is left: the cut removed the whole part, or the intersect found no overlap. Make the outline smaller, or move it over the part. |

## From sketches

| Message | Meaning and fix |
|---|---|
| `geometry values must be finite` | A coordinate, radius or angle is infinite or not a number. |
| `radius must be > 0, got r` | A circle or arc needs a positive radius. |
| `value must be > 0, got v` | A distance, radius or diameter constraint needs a positive value. |
| `unknown geometry N` | The constraint names an entity that isn't in this sketch. |
| `geometry N has a different kind` | Wrong kind of point or entity, such as `Center` of a line, `ArcStart` of a circle, a circle in parallel, or a line made equal to a circle (Chapter 5). |
| `unknown constraint N` | No constraint with that number is in this sketch. |
| `constraint N is not a dimension` | Only a distance, radius or diameter has a value to change. |
| `only distance, radius and diameter can be bound, got …` | `bind_dimension` was given another kind of constraint. Add it with `add_constraint` instead (Section 17.5). |
| `a sketch dimension is bound to unknown parameter `p`` | The parameter was renamed or never existed. After `doc.rename_param`, call `rename_bound` with the same names (Section 17.5). |

## From profiles

| Message | Meaning and fix |
|---|---|
| `geometry A and B intersect` | Pieces A and B cross, overlap, or touch away from their ends. Move or shorten one so they meet only end to end (Section 5.7). |
| `weld tolerance must be > 0, got t` | Give a `weld_tolerance` greater than zero, or `None` for the default. |

Any sketch message above can also come from profiles, when the sketch itself is at fault.

## From the document model

| Message | Meaning and fix |
|---|---|
| `unknown parameter `p`` | Set the parameter before a feature uses it. Renaming or commenting on a parameter that doesn't exist also gives this. |

| Message | Meaning and fix |
|---|---|
| `a parameter named `p` already exists` | Every parameter needs its own name. Pick another, or rename the other parameter first (Section 17.4). |
| `invalid parameter name `p`: must start with a letter or `_` and contain only letters, digits and `_`` | Use a name like `hole_2`: no leading digit, no spaces, dashes, dots or accents. |
| `unknown feature N` | No feature with that number is in the history. |
| `feature N references feature M which is not earlier in the history` | A feature can use only features above the end-of-part marker. `M` is a feature number, not a place in the history: it may be rolled back (Section 17.3), or not made yet. |
| `end-of-part position P is past the end of the history (N features)` | The marker can go anywhere from 0 to the number of features, `N`. It stayed where it was (Section 17.3). |
| `expression for `p`: …` | The formula for `p` doesn't parse, names an unknown parameter, or can't be worked out. The rest is an expression message, below. |
| `parameter expressions form a cycle: a -> b -> a` | The formulas would depend on each other in a loop, traced in the message. Make one of them a plain number (Section 17.7). |
| `expression for `p` gives length^2, but its unit expects length` | The formula's result is the wrong kind of quantity for the parameter's unit, here an area for a length. A parameter with no unit, or a unit outside the seven, expects `unitless`. |

## From expressions

Each message gives the byte where the problem is, counting from 0 at the first character (Section 17.6).

| Message | Meaning and fix |
|---|---|
| `syntax error at byte n: …` | The text isn't a formula: an unexpected character, a missing `)` or number, or an expression too long or too deeply nested. The rest of the message says which. |
| `unknown unit `u` at byte n (expected mm, cm, m, in, ft, deg or rad)` | A word after a number isn't one of the seven units. Check the spelling. |
| `unknown name `p` at byte n` | No parameter has that name. Names are case-sensitive. |
| `dimension mismatch at byte n: cannot apply `+` to length and angle` | Both sides of `+` or `-` must measure the same thing. Give a bare number its unit, as in `d1 + 5 mm`. |
| `division by zero at byte n` | The part after `/` works out to zero. |

## From inference

For an unknown entity, `infer` and `add_inferred` give the sketch messages above. They can also give:

| Message | Meaning and fix |
|---|---|
| `geometry values must be finite` | A setting is infinite or not a number (Section 19.12). |
| `value must be > 0, got v` | A setting is zero or less. Both tolerances must be greater than zero. |
| `solve did not converge (residual r)` | From `add_inferred` only. The suggested constraints were added, but couldn't all be met. Check the degrees of freedom for a conflict (Section 5.3). A constraint can't be removed yet, so build the sketch again and add only the suggestions you want (Section 5.8). |

### From document files

`swx-io` messages have no prefix (Chapter 18). Fields are named by their path, such as `units.length`.

| Message | Meaning and fix |
|---|---|
| `invalid JSON: model.json: expected value at line 1 column 17` | The file isn't valid JSON. The message names the file (not for `read_manifest` alone) and where reading stopped. |
| `missing required field `units.length`` | A required manifest field is missing (Section 19.17). |
| `invalid field `kind`: must be "part", "assembly" or "drawing", got "widget"` | A manifest field has a value that isn't allowed, or the wrong type. The reason names what is allowed. The same form covers `format`, `format_version`, `document_id`, `model_hash` and `units`. |
| `invalid field `model.json`: must be a JSON object` | The model or the properties isn't a JSON object (also `properties.json`, and `manifest` for a manifest that isn't an object). |
| `unsupported format_version "1.0.0": only major version 0 is supported` | The document comes from a newer major version. Use a newer Saulitwerx. |
| `missing required file `model.json`` | The folder has no `model.json` or no `manifest.json` (Section 18.3). |
| `folder extension `.sltwxa` does not match the manifest kind `part` (expected `.sltwx`)`<br>`folder has no extension; the manifest kind `part` needs `.sltwx`` | Name the folder for its kind: `.sltwx` for a part, `.sltwxa` for an assembly, `.sltwxd` for a drawing. |
| `files/missing.sltwx: No such file or directory (os error 2)`, `…: not a directory` | The folder or a file can't be read, written or removed. The message starts with the path and ends with the system's reason. |

## 20.3 Questions

**Is there a version with windows and buttons?** Not yet. The desktop app is a later milestone; see **Coming next**.

**Can I open files from other CAD programs?** Yes, if you save them as STEP first. Saulitwerx doesn't read any program's own native file format, and won't.

**Can I save my design and edit it later?** Not yet. Today you can save solids as STEP, which keeps the shape but not the history. Saulitwerx can write and read its own document folders (Chapter 18), but can't yet store a part's history in them or rebuild a part from one.

**Does it cost anything?** No. Saulitwerx is free and open source, under the MIT or Apache-2.0 license, at your choice.

**Why millimeters?** Mechanical design works mostly in millimeters, so it is the default. Choosing other units per part is planned.

## 20.4 Known limitations

- No desktop app, 3D view, mouse or keyboard controls yet.
- A sketch's regions can't be extruded or revolved directly yet: type the outline's pieces into a program (Chapter 7, Chapter 8).
- An extrude can end on a face (Section 7.6) only when the face is flat and parallel to the workplane, and it ends on the face's whole plane: it doesn't stop at the face's edges, or follow a curved face.
- A fillet has one radius for all its edges, and a chamfer one size for all its edges: one distance, two distances, or a distance and an angle (Chapter 9). A fillet whose radius changes along the edge rounds one edge per call, changing evenly from one end to the other (Section 9.3).
- A shell has one wall thickness for every wall, and needs at least one open face (Chapter 10). A part hollow inside with no opening, or with walls of different thicknesses, can't be made yet.
- A revolve turns one way from the outline's plane by an angle; it can't yet turn both ways or symmetrically.

- An extrude's or a revolve's target is a solid you hold in your program. Faces and edges can be counted and numbered (Section 6.3), a plain extrude says which face each piece of the outline became (Section 7.5), and a Boolean what became of each face (Section 6.4), but they can't be picked, and their numbers don't last through a change; so the outline's workplane is given by its numbers (Section 7.3).
- Holes have flat bottoms, with no drill points. A tapped hole is drilled at its tap-drill size, but its thread isn't modeled, and only metric coarse sizes from M2 to M24 are built in (Section 11.2).
- A draft tilts flat faces by one angle about one neutral plane; it can't split a face or step its neutral line (Chapter 12).
- A part with several bodies (Section 6.5) is one shape: its bodies can be counted and taken apart, but not named, moved or kept apart in a part's history.
- Work planes, axes and points can be taken from a part's faces, edges and corners only by their numbers, not picked; an axis taken from the part may point either way (Section 15.3).
- A loft goes through flat sections without holes, in the order given, with no guide curves and no control over how the sections line up (Chapter 13).
- A sweep moves one flat profile without holes along an open path of lines and arcs in one plane, or around a helix of one radius and one pitch: no closed, spline or other 3D paths, no twist or change of size. A bend must be wider than the profile: the check uses the profile's farthest point from the path's start, so it can refuse a bend that would just fit on the profile's narrow side (Chapter 14).
- Extrudes, revolves, lofts, sweeps, fillets, chamfers, shells, holes, drafts and work features aren't recorded in a part's history, so they don't rebuild when a parameter changes.
- Document files exist only in the unpacked (folder) form, and `model.json` isn't yet filled from a part or read back into one, so work can't be saved with its history (Chapter 18). There is no single-file form and no `sltwx` command yet.
- `swx-qa sketch` draws only its two built-in sketches. Your own sketches need a short program (Section 16.5). Sketch pictures show no dimension arrows and no marks for constraints such as horizontal.
- A parameter's unit is free text. Only the seven units of Section 19.15 are understood by formulas; any other unit counts as none.
- When a new value breaks a formula that depends on it, the formula keeps its old value without an error (Section 17.7).
- A constraint can't be removed from a sketch once added; build the sketch again without it.
- Inference needs a program: there is no drawing with the mouse yet, so nothing is inferred as you move the pointer (Section 5.8).
- Renaming a parameter doesn't update the sketches bound to it by itself; call `rename_bound` too (Section 17.5).
- There are no angle dimensions and no point-to-line distance yet.
- Profiles don't split pieces where they cross; crossing pieces are an error.
- Only built on and tested on Ubuntu Linux.
- An imported STEP file becomes one shape, and can't be edited.

PART VI

# Back matter

Glossary, what changed, what comes next, a one-page card, credits and licenses, and the index.

# Glossary

**Adjacent**
Said of two sub-shapes when one bounds the other, such as a face and one of its edges.

**Axis of revolution**
The line a revolve turns its outline around.

**Between faces**
An extrude extent that runs from one flat face of the target body to another.

**Binding**
The link between a sketch dimension and the parameter that drives it.

**Blind hole**
A hole that stops at a set depth, with a flat bottom.

**Body**
One separate piece of solid. A shape can hold several bodies that don't touch.

**Boolean operation**
Combining two solids by union, cut or intersect.

**Bounding box**
The smallest box, with sides parallel to the axes, that holds a solid.

**CAD**
Computer-aided design: using a computer to design physical objects.

**Cap**
One of the two flat faces at the ends of an extrude: where the push starts and where it ends.

**Chamfer**
A flat, angled face cut along an edge in place of the sharp edge. It is set back the same distance on both faces, or by two distances, or by a distance and an angle.

**Coincident**
A constraint that puts two points at the same place.

**Conflicting**
Said of a sketch whose rules contradict each other, so no shape can meet them all.

**Constraint**
A rule about a sketch's shape, such as "horizontal" or "these points touch".

**Counterbore**
A wider, flat-bottomed recess at the mouth of a hole, so a screw head sits flush or below the surface.

**Countersink**
A cone-shaped recess at the mouth of a hole, so a screw with a cone-shaped head sits flush with the surface.

**Cut**
A Boolean operation that removes one solid from another.

**Degree of freedom (DOF)**
One way a piece of a sketch can still move. A free point has two.

**Dependency order**
The order formulas are worked out in: each parameter after the parameters its formula uses.

**Diameter**
The width of a circle through its center; twice the radius.

**Dimension**
A constraint with a number: a distance, radius or diameter.

**Dirty**
Said of a feature that must be rebuilt after a change.

**Document file**
A Saulitwerx file that keeps a design with everything needed to edit it: a part ( `.sltwx` ), an assembly ( `.sltwxa` ) or a drawing ( `.sltwxd` ).

**Draft**
A small tilt given to the sides of a molded or cast part so it comes out of the mold.

**Edge**
A curve where two faces of a solid meet, or where one round face meets itself (a seam).

**Edge midpoint**
The work point halfway along an edge, measured along the edge.

**End-of-part marker**
The point in a part's history after which features are set aside (rolled back). New features go in at the marker.

**Entity**
One piece of sketch geometry: a point, line, circle or arc.

**Equal**
A constraint that makes two lines the same length, or two circles or arcs the same size.

**Expression**
A small formula with units that works out a value, such as `d1 / 2 + 5 mm`.

**Extent**
How far an extrude goes: a distance one way, a length split evenly on both sides of the workplane, through all, or up to a face.

**Extrude**
Pushing a flat outline straight out into a solid with straight sides.

**Extrude mode**
What an extrude does with its shape: keep it as a new solid, or join, cut or intersect it with a target body.

**Face**
One flat or curved patch of a solid's surface.

**Feature**
One step in building a part, such as a sketch or a cut. Features are kept in the history.

**Feature tree**
Another name for a part's history.

**Fillet**
A rounded face along an edge in place of the sharp edge, with a set radius.

**Fixed**
A constraint that keeps a point at exact coordinates.

**From–to**
An extrude extent given by a start and an end distance along the workplane's normal.

**Fully constrained**
A sketch whose constraints and dimensions leave nothing free to move.

**Geometry kernel**
The part of a CAD program that does the 3D math. Saulitwerx uses OpenCascade.

**Helical sweep**
A sweep whose path is a helix, for springs, coils and threads.

**Helix**
A curve that circles an axis at a fixed distance while rising the same amount each turn, like the handrail of a spiral staircase.

**History**
A part's features, in the order they were made.

**Hole**
A feature that drills a round hole into a solid from a start point, in a direction, to a depth or through all.

**Inference**
Saulitwerx adding the constraints a newly drawn piece already almost satisfies, such as coincident or horizontal.

**Intersect**
A Boolean operation that keeps only what two solids share.

**Island**
A region inside a hole of another region.

**Isometric**
A view from a corner, looking down at an angle, so that three sides of a box show equally.

**JSON**
A plain-text way of writing down named values, readable by people and programs. Saulitwerx document files are written in it.

**Loft**
A feature that makes a solid through two or more flat sections, joining them in order, ruled or smooth.

**Loop**
A closed chain of sketch pieces, each ending where the next begins.

**Manifest**
The small file in a Saulitwerx document, `manifest.json`, that says what the document is, which format version wrote it, its id and its units.

**Midplane**
The work plane halfway between two parallel planes.

**Midpoint**
A constraint that keeps a point halfway along a line.

**Model parameter**
A parameter that belongs to a dimension, named `d0`, `d1`, … in turn.

**Multi-body part**
A part made of more than one body, such as a plate cut right across into two pieces.

**Neutral plane**
The plane a draft turns faces about. Where a drafted face crosses it, the face stays put.

**Open edge**
A sketch piece that isn't part of any closed loop.

**Open face**
A face that a shell removes, leaving an opening in the hollowed part.

**OpenCascade (OCCT)**
The open-source geometry kernel Saulitwerx uses.

**Over-constrained**
Said of a sketch with a rule that repeats what other rules already say.

**Parallel**
A constraint that makes two lines point the same way.

**Parameter**
A named number that features and dimensions refer to.

**Parametric**
Built from steps and numbers that can be changed later, with the design rebuilding to match.

**Part origin**
The point (0, 0, 0) of a part, where its three origin planes and axes meet.

**Path**
The line a sweep moves its profile along: straight lines and arcs joined end to end.

**Perpendicular**
A constraint that makes two lines meet at a right angle.

**Pitch**
How far a helix rises in one full turn; for a screw thread, the distance from one ridge to the next.

**Pre-alpha**
Early, unfinished software: the parts work, but it isn't ready for real use.

**Profile**
The regions of a sketch: the areas a feature can turn into a solid.

**Pull direction**
The way a part leaves its mold. Draft makes the part smaller along it.

**Redundant**
Said of a rule that repeats what other rules already say.

**Reference face**
For an uneven chamfer, the one of an edge's two faces that its first distance, or its distance and angle, is measured on.

**Region**
A flat area enclosed by an outer loop, minus any holes inside it.

**Revolve**
Turning a flat outline around an axis into a round solid.

**Revolve angle**
How far a revolve turns: a full turn ($2\pi$ radians, 360°) or part of one.

**Rolling back**
Moving the end-of-part marker up the history, so the features below it are kept but not used.

**Ruled**
Said of a loft joined by straight surfaces from each section to the next, with a crease at each middle section.

**Seam**
An edge where a closed round face, such as the wall of a hole, meets itself.

**Section**
One of the flat outlines a loft passes through, like a slice through the finished part.

**Shell**
A feature that hollows a solid, leaving walls of one thickness and openings where its open faces were.

**Side face**
A face of an extrude swept by one edge of its outline: a flat wall from a line, a round one from an arc or circle.

**Simple hole**
A plain round hole of one diameter.

**Sketch**
A 2D drawing on a plane, made of points, lines, circles and arcs.

**Smooth loft**
A loft with one curved surface through all its sections, with no creases.

**Solid**
A closed 3D shape with an inside and an outside.

**Solver**
The calculation that moves sketch geometry until every constraint holds.

**Split face**
A face that an operation cut into several faces of the result, such as a top divided by a groove.

**STEP**
An international standard file format (ISO 10303) for exchanging 3D parts.

**Sub-shape**
One face, edge or vertex of a solid.

**Surface area**
The total area of a solid's faces.

**Sweep**
A feature that moves a flat profile along a path, leaving a solid behind it.

**Symmetric**
A constraint that makes two points mirror images across a line.

**Tangent**
Touching without cutting in: a line and a circle, or two circles, that meet at one point.

**Tangent plane**
A work plane that touches a round face along one straight line without cutting into it.

**Tap drill**
The drill a hole is made with before a thread is cut in it: narrower than the screw, so there is material for the thread.

**Tapped hole**
A hole with a screw thread cut inside it, or drilled ready for one.

**Target body**
The solid an extrude or a revolve joins to, cuts from or intersects with.

**Thread**
The spiral ridge on a screw, or the matching groove inside a hole. Metric threads are named M and the diameter in mm, such as M6.

**Through all**
An extrude cut with no depth of its own, which goes right through the target body.

**Through hole**
A hole that goes all the way through a solid.

**To face**
An extrude extent that ends on the plane of a flat face of the target body.

**Under-constrained**
Said of a sketch with at least one degree of freedom left.

**Union**
A Boolean operation that joins two solids into one.

**Unit**
What a number measures in, such as millimeters.

**Unpacked form**
A Saulitwerx document kept as a folder of files, named like the document, rather than as one file.

**User parameter**
A parameter you create yourself, with a name you choose.

**UUID**
A 32-digit hexadecimal id, written in groups of 8, 4, 4, 4 and 12, made so that no two are ever the same.

**Variable fillet**
A fillet whose radius changes evenly from one end of its edge to the other.

**Vertex**
A corner point of a solid, where its edges meet. Plural: vertices.

**Volume**
How much space a solid fills.

**Work axis**
A straight line built to place features, such as an axis to revolve about. It has no material.

**Work feature**
Reference geometry built to place other features: a work plane, a work axis or a work point. It adds no material.

**Work plane**
A flat plane built to sketch on or place features from, such as one offset from a face or tilted at an angle. Used as a workplane.

**Work point**
One point in space, built to place features, such as where an axis crosses a plane.

**Workplane**
The flat plane an outline is drawn on, with its own directions and a normal pointing straight out of it.

# What's new

## 2026-11-12: springs and coils, and Saulitwerx document files

- **New:** sweep a flat profile around a helix, by pitch and number of turns, for coil springs, coils and thread-like ridges (Section 14.2).
- **New:** write and read Saulitwerx document folders ( `.sltwx` ), with a checked manifest and files written the same way every time (Chapter 18). The design itself can't be stored in them yet.
- **Changed:** hole errors now print the countersink depth without stray digits ( `2.8` , not `2.8000000000000003` ), and a bad countersink angle in degrees as well as radians (Section 11.3).
- **Pictures:** a coil spring and a square ridge wound around a helix.
- New chapter 18, Saulitwerx document files; Reference and Help are now chapters 19 and 20. Reference: `helical_sweep` , and the document file calls and manifest fields. Help: the helical sweep's and the document files' error messages, new common mistakes, and the known limitations updated. Glossary: seven new terms, from document file to UUID.

## 2026-11-11: countersunk and tapped holes, and a thread table

- **New:** drill a countersunk hole for a flat-head screw, and a tapped hole at the right drill size for a metric screw thread, blind or through all (Section 11.2).
- **New:** look up metric coarse thread sizes from M2 to M24 by name, with their pitch, minor diameter and tap drill (Section 19.5).
- **Pictures:** a plate with a countersunk hole and two tapped holes.
- Reference: the new hole shapes and the thread table. Help: six new error messages, new common mistakes, and the known limitations updated. Glossary: five new terms, from countersink to thread.

## 2026-11-10: several bodies, midplanes and tangent planes, safer sweeps

- **New:** count the separate bodies in a part, take each one on its own, and join, cut or intersect a part with a whole list of tools in one call, with the history of every face (Section 6.5).
- **New:** a work plane halfway between two parallel planes, a work plane touching a round face, and a work point halfway along an edge (Section 15.4).
- **Changed:** a sweep now refuses a bend tighter than its profile, with an error naming the bend, instead of returning a part that passes through itself (Chapter 14).
- **Pictures:** a plate drilled in one call and split into two bodies; a shaft with a cross hole on its midplane and a flat cut from a tangent plane.
- Reference: the new calls, `CombineHistory` , and the sweep's bend rule. Help: ten new error messages, new common mistakes, and the known limitations updated. Glossary: five new terms, from body to tangent plane.

## 2026-11-09: loft, sweep, and work features from the part

- **New:** blend a solid through several flat sections, ruled or smooth, such as a square into a circle (Chapter 13).
- **New:** move a flat outline along a path of lines and arcs, for bent rods, bars and pipes (Chapter 14).

- **New:** take a work plane from a flat face, a work axis from a straight edge or a round face, and a work point from a corner of a part (Section 15.3).
- **Pictures:** a square-to-round transition and a vase lofted ruled and smooth; a rod and a bar swept along one bent path; a block with a post on a face's plane and a hole down its boss's axis.
- Reference: the new calls and samples. Help: thirteen new error messages, new common mistakes, and the known limitations updated. Glossary: six new terms, from loft to sweep. Chapters 13 to 19 are renumbered: Work planes, axes and points is now Chapter 15.

## 2026-11-08: holes, draft, work features, and a fillet that changes size

- **New:** drill simple and counterbored holes, blind or through all, in any direction (Chapter 11).
- **New:** tilt the sides of a part with draft, so it comes out of a mold (Chapter 12).
- **New:** build work planes, axes and points from the part origin and from each other, and place features on them (Chapter 15).
- **New:** round one edge with a radius that changes from one end to the other (Section 9.3).
- **Pictures:** a plate with four holes; a block drafted three ways; a post leaning on a tilted work plane; a constant and a variable fillet side by side.
- Reference: the new calls and types, and a work-features section (Section 19.7). Help: sixteen new error messages, new common mistakes, and the known limitations updated. Glossary: fourteen new terms, from blind hole to work point.

## 2026-11-07: shells, and rolling back the history

- **New:** hollow a solid into walls of one thickness, open at the faces you choose: a tray, a frame or a cup in one call (Chapter 10).
- **New:** move the history's end-of-part marker to roll a part back, insert features in the middle of its history, and roll forward again (Section 17.3). A new reference section lists the history calls (Section 19.13).
- **Pictures:** a block shelled into a tray and a frame, and a cylinder into a cup.
- Help: three new error messages, seven new common mistakes, and the known limitations updated. Glossary: end-of-part marker, open face, rolling back, shell.

## 2026-11-06: uneven chamfers, and faces through a Boolean

- **New:** chamfer an edge by two different distances, or by a distance and an angle, measured from a face you choose (Section 9.2).
- **New:** combine two solids with history, and find out which faces of the result each input face became: kept, split or deleted (Section 6.4).
- **Pictures:** one edge chamfered four ways; a grooved block with its faces named.
- Help: four new error messages, four new common mistakes, and the known limitations updated. Glossary: reference face, split face.

## 2026-11-05: up to a face, fillets and chamfers

- **New:** extrude up to a flat face of the part, or between two of its faces, instead of by a distance; and give an extrude a start and an end distance (Section 7.6).
- **New:** round edges with a fillet, or bevel them with a chamfer, in a new chapter (Chapter 9).
- **New:** ask for the plane of a flat face (Section 19.4).
- **Pictures:** a stepped block extruded up to its faces; a block with filleted and chamfered edges.
- Help: thirteen new error messages, eight new common mistakes, and the known limitations updated. Glossary: between faces, chamfer, fillet, from–to, to face.

## 2026-11-04: which face is which

- **New:** extrude with history, and find out which face of the solid each piece of your outline became: the two caps and every side face (Section 7.5, Section 19.6).
- **New:** ask for the bounding box of a single face, edge or vertex (Section 19.4).
- **Pictures:** the extruded plate, with its faces named by their history.
- Help: two new common mistakes; two error messages and a known limitation updated. Glossary: cap, side face.

## 2026-11-03: safer revolves

- **New:** a revolve now checks its axis first. An axis that runs through the outline, or doesn't lie in the outline's plane, gets a clear message saying which, instead of a broken solid or a vague failure (Section 8.3).
- The revolve sample shows both new errors (Section 8.1).
- Help: three new error messages; the revolve rows and the known limitation about unchecked axes updated.

## 2026-11-02: faces, edges and vertices

- **New:** count a solid's faces, edges and vertices, and list which ones touch: the edges around a face, the faces along an edge (Section 6.3, Section 19.4).
- Concepts: vertices join faces and edges in Section 4.1, with the plate's figure now marking one of each.
- Help: two new error messages; the known limitation about faces and edges updated.

## 2026-11-01: revolve, and cuts through all

- **New:** revolve a flat outline around an axis into a round solid, by a full turn or part of one, as a new solid or joined to, cut from or intersected with a part (Chapter 8, Section 19.6).
- **New:** an extrude cut can go right through a part, one way or both, however thick the part is (Section 7.4).
- **Pictures:** a flanged bushing at three quarters of a turn and at a full turn, the bushing with a groove, and a block with two through holes.
- Help: nine new error messages, eight new common mistakes, and known limitations updated. The limitations that there is no revolve and no "through all" are gone.

## 2026-10-31: extrude modes

- **New:** an extrude can join its shape to a part, cut it out, or keep only the overlap, in one step (Section 7.3, Section 19.6).
- **Pictures:** one block with a boss, a pocket and a trimmed shape made from it.
- Help: two new error messages, three new common mistakes, and known limitations updated. The limitation that an extrude always makes a new solid is gone.

## 2026-10-30: extrude

- **New:** extrude a flat outline into a solid. Outlines are made of lines, arcs and circles, can have holes, and can sit on any plane; the extrude goes a distance one way or splits evenly on both sides (Chapter 7, Section 19.6).
- **Pictures:** a profile and the plate extruded from it, and an extruded slot.
- Concepts: the profile and feature sections now point to the extrude.
- Help: eleven new error messages, and known limitations updated. The limitation that nothing turns outlines into solids is gone.

## 2026-10-29: formulas in parameters, and inference

- **New:** a parameter can hold a formula, such as `d0 / 2 + 5 mm`, and follows the parameters it names. Formulas are worked out in dependency order, loops and wrong units are refused, and renaming a parameter rewrites the formulas that use it (Section 17.7, Section 19.14).
- **New:** inference adds the coincident, horizontal, vertical, parallel, perpendicular and tangent constraints a newly drawn piece already almost satisfies (Section 5.8, Section 19.12).
- **Pictures:** a rough rectangle and circle before and after inference.
- Help: new error messages for formulas and inference, four troubleshooting rows, new common mistakes, and known limitations updated. The limitation that parameters hold only plain numbers is gone.

## 2026-10-28: dimensions as parameters, expressions, and joined arcs

- **New:** constraints can name an arc's two ends (`ArcStart`, `ArcEnd`), so lines and arcs join into closed outlines. The slot task is rewritten: it now starts from a careless drawing and solves to a fully defined slot (Section 5.6, Section 19.8).
- **New:** a sketch's distance, radius and diameter dimensions can each make a model parameter, `d0`, `d1`, …, and follow it when it changes (Section 17.5).
- **New:** work out expressions with units, such as `d1 / 2 + 5 mm`, with unit checking (Section 17.6, Section 19.15).
- **Pictures:** the slot before and after solving replaces the slot with a gap.
- Help: new error messages for bound dimensions and expressions; troubleshooting and known limitations updated. The limitation on arc ends is gone.

## 2026-10-26: parameter table, and named conflicts

- **New:** a part's parameters form a table: model parameters named `d0`, `d1`, … in turn, user parameters with names you choose, units and comments. Renaming a parameter updates every feature that uses it (Section 4.8, Section 17.4, Section 19.14).
- **Changed:** the degree-of-freedom report now names the redundant or conflicting constraints, not just equation numbers (Section 5.3, Section 19.11). The matching known limitation is gone.
- Help: two new parameter error messages, and two new common mistakes.

## 2026-10-25: more constraints, and profiles

- **New:** six constraints: parallel, perpendicular, equal, midpoint, symmetric and tangent (Section 4.4, Section 19.8).
- **New:** keep a hole centered whatever the plate's size (Section 5.5), and draw a slot with tangent, equal-sized ends (Section 5.6).
- **New:** find the regions a sketch encloses, with their holes and areas, and the pieces left open (Section 4.7, Section 5.7, Section 19.10).
- **Pictures:** the centered plate at two widths, and a slot with a gap, ringed by the template.
- Help: profile error messages, two new troubleshooting rows, and known limitations updated, including arcs whose ends can't be constrained.
- Corrected: midpoint and symmetric add two equations each to the degree-of-freedom count.

## 2026-10-24: degrees of freedom and sketch pictures

- **New:** check how fully a sketch is defined: degrees of freedom left, and whether it is under-, fully or over-constrained, or conflicting (Section 4.6, Section 5.3, Section 19.11).
- **New:** draw a sketch to a PNG with `swx-qa sketch` (Section 16.5, Section 19.2).
- **Pictures:** the sketch sections of **Concepts** and **Sketching** now show the plate sketch, rough and solved.

- Help: new troubleshooting rows and `swx-qa sketch` error messages; known limitations updated.

## 2026-10-22: first edition

The first edition of this manual. It covers everything on `main` today:

- Installing from source, and a first tutorial that builds, measures, saves and draws a plate with a hole.
- The ideas of parametric CAD: solids, units, sketches, constraints, dimensions, parameters, features and history.
- Sketching with the coincident, horizontal, vertical, fixed, distance, radius and diameter constraints, and the solver.
- Making and combining solids, and saving and reading STEP files.
- Drawing parts with `swx-qa render`.
- The feature history, and finding what a change rebuilds.
- Reference tables, every error message, troubleshooting, and known limitations.

# Coming next

These are the next items on the public roadmap. None of them works yet. When one arrives, it moves into the chapters of this manual and into **What's new**.

| Soon | What you'll be able to do |
|---|---|
| Extrude from a sketch | Hand a sketch's region straight to the extrude and the revolve. |
| More part features | Shells with walls of different thicknesses; fillets, chamfers, shells, holes, drafts, lofts, sweeps and work features kept in a part's history. |
| Threads | Model real screw threads, outside on a shaft or inside a hole, built on the helical sweep. |
| Lasting references | A feature built on a face or an edge keeps finding it when earlier features change. |
| Part files | Save a part with its full history in a document file and open it again; one zipped file as well as a folder; the `sltwx pack` and `unpack` commands; stored previews. |
| Desktop app | A window with a 3D view, a model browser, sketch mode and property panels. |

Later milestones add assemblies, 2D drawings and exploded views. The full plan is in `docs/ROADMAP.md` in the source code.

# Quick reference



## BUILD

TERMINAL

```
cargo test --workspace
cargo build --release -p swx-qa \
  --features occt
```

## DRAW A PART

TERMINAL

```
swx-qa render --step part.step -o part.png
swx-qa render --fixture cube -o cube.png
  --size 1600x1000   (default 800x600)
  --crease 45     (default 30)
```

## DRAW A SKETCH

TERMINAL

```
swx-qa sketch --fixture rectangle -o r.png
  (or triangle-345)
```

## SAMPLES

TERMINAL

```
cargo run -p swx-sketch \
  --example plate_sketch
cargo run -p swx-sketch \
  --example centered_plate -- 60 25
cargo run -p swx-kernel --features occt \
  --example plate_with_hole -- plate.step
cargo run -p swx-kernel --features occt \
  --example extrude_plate
cargo run -p swx-kernel --features occt \
  --example revolve_bushing -- revolve
cargo run -p swx-core \
  --example feature_history
cargo run -p swx-core \
  --example parameter_table
cargo run -p swx-sketch \
  --example driven_plate
cargo run -p swx-core \
  --example parameter_formulas
cargo run -p swx-sketch \
  --example inferred_sketch
```

Every sample, with what it does: Section 19.3.

## UNITS

Millimeters. Angles: degrees in feature parameters; radians in sketch arcs and kernel revolves.

## SKETCH STATES

| State | Means |
|---|---|
| Under-constrained | can still move: add rules |
| Fully constrained | done: 0 DOF left |
| Over-constrained | a rule repeats others |
| Conflicting | rules contradict |

## SKETCH CONSTRAINTS

| Constraint | Keeps |
|---|---|
| Coincident | two points together |
| Horizontal | a line level |
| Vertical | a line upright |
| Fixed | a point at (x, y) |
| Distance | two points a length apart |
| Radius / Diameter | a circle's size |
| Parallel · Perpendicular | two lines' directions |
| Equal | two lengths or radii alike |
| Midpoint | a point mid-line |
| Symmetric | two points mirrored |
| Tangent | a line touching a curve |

## SOLIDS

| Call | Gives |
|---|---|
| `make_box` | box from corner + sizes |
| `make_cylinder` | cylinder on +Z |
| `make_sphere` | sphere |
| `union` · `cut` · `intersect` | Boolean result |
| `make_face` · `extrude` | outline ⟶ face ⟶ solid |
| `extrude_feature` | extrude and combine |
| `revolve_feature` | revolve and combine |
| `fillet` · `chamfer` | round / bevel edges |
| `chamfer_asymmetric` | bevel by 2 sizes |
| `shell` | hollow, open at faces |
| `fillet_variable` | round, 2 radii |
| `hole` | drill any kind of hole |
| `metric_coarse` | thread size by name |
| `draft` | tilt faces for a mold |
| `loft` · `sweep` | blend / follow a path |
| `helical_sweep` | spring, coil |
| `offset` · `midplane` | work planes |
| `work_plane_on_face` | plane on a flat face |
| `work_axis_on` | axis on edge or round face |
| `boolean_tracked` | Boolean + face history |
| `export_step` · `import_step` | STEP file out / in |

# Credits and licenses

Saulitwerx is written by its contributors and is licensed under either the MIT license or the Apache License 2.0, at your choice. It is an independent project, not affiliated with or derived from any commercial CAD product.

| Component | License | Notes |
|---|---|---|
| OpenCascade Technology (OCCT) 7.6 | LGPL-2.1 with the OCCT exception | The geometry kernel. Used as a separate, dynamically linked library; not included in Saulitwerx's source. |
| `png`, `flate2`, `miniz_oxide`, `fdeflate`, `crc32fast`, `adler2`, `simd-adler32`, `bitflags`, `cfg-if` | MIT, Apache-2.0, Zlib or 0BSD | Write the PNG pictures of `swx-qa`. |
| `cc`, `shlex`, `find-msvc-tools` | MIT or Apache-2.0 | Build the bridge to OpenCascade. |
| Typst 0.15.1 | Apache-2.0 | Typesets this manual. |
| Inter 4.0, by The Inter Project Authors | SIL Open Font License 1.1 | Headings and labels in this manual. License in `docs/manual/fonts/OFL-Inter.txt`. |
| Libertinus Serif, DejaVu Sans Mono | SIL Open Font License 1.1; Bitstream Vera license | Body text and code in this manual, as bundled with Typst. |

Every picture of a part in this manual was drawn by Saulitwerx's own renderer, `swx-qa render`, and every picture of a sketch by `swx-qa sketch`'s plotter, from geometry made by Saulitwerx.

# Index


**A**

adjacent 31
analyze 126
arc ends 26
axis of revolution 51

**B**

between faces 47
between-faces 47
bind_dimension 129
binding 104
blind hole 67
body 34
Boolean history 33
boolean operations 16
bounding box 12

**C**

CAD 2
cap 45
chamfer 56, 58
coincident 14
conflicting 15
constraint 13
constraint id 22
counterbore 67
countersink 67, 70
cut 16

**D**

degree of freedom 15
degree-of-freedom report 21
dependency order 107
diameter 14
dimension 14
dirty 101
distance 14
Document 128
document file 110
dof 126
doubly connected edge list 126
draft 75

**E**

edge 12
edge midpoint 93
end-of-part marker 101, 128
entities 13
equal 14, 24
expression 16, 129
extent 38
extrude 38
extrude history 45
extrude mode 41

**F**

faces 12
feature 16
feature tree 17
fillet 56, 60
fixed 14
from–to 49
fully constrained 15

**G**

geometry 13
geometry kernel 5, 30

**H**

helical sweep 83
helix 83
history 17
hole 67
horizontal 14
Householder QR 127

**I**

inference 27, 127
intersect 16
isometric 8

**J**

json 110

**K**

Kahn's algorithm 129

**L**

Levenberg–Marquardt 125
loft 78
loop 15

**M**

manifest 110
midplane 93
midpoint 14, 22
model parameter 103
multi-body part 34

**N**

neutral plane 75

**O**

occt feature 118
open edges 16
open face 64
OpenCascade 5
over-constrained 15

**P**

parallel 14, 23
parameter 16
parameter table 103
parametric 2
part origin 87
path 81
perpendicular 14, 23
pitch 70, 83
PointRef 20
profile 15
profile_report 125
pull direction 75

**R**

radius 14
rebuild 101
redundant 15
reference face 58
region 15
revolve 51
revolve angle 51
revolve mode 53
rolling back 101

**S**

seam 32
section 78
shell 64
side face 45
simple hole 67
sketch 13
solid 9, 12
solver 14
split face 33
STEP 10
sub-shape 31
sub-shapes 31
surface area 12
sweep 81
swx-qa render 115
swx-qa sketch 115
symmetric 14, 24

**T**

tangent 14, 24, 125
tangent plane 93
tap drill 70
tapped hole 67, 70
target body 41
thread 70, 121
through all 43
through hole 67
through-all 43
to face 47
to-face 47

**U**

under-constrained 15
union 16
unit 13
unpacked form 110
user parameter 103
uuid 113

**V**

variable fillet 60
vertex 12, 60
vertical 14
volume 12

**W**

work axis 87
work feature 87
work plane 87
work point 87
workplane 38